

VECTOR

Conference Bumper Issue

Including ...

• J Release 3 reviewed	48
• Dyalog 7.2/8 reviewed	54
• Eric Iverson at the BAA	71
• Parallel Processing in J	86
• Solving Nonograms with APL	102
• Lescasse on APL and Delphi	122



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.13 No.1 July 1996

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1996-97

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

		Page
Editorial	Duncan Pearson	3
APL NEWS		
Quick Reference Diary		5
News from Sustaining Members	Gill Smith	12
The Education Vector	Ian Clark	17
APL Product Guide	Gill Smith	33
PRODUCT REVIEWS		
J3 - J Grows Up!	John Baker, Martin Neitzel	48
Dyalog APL/W 7.2 and 8.0	Bob Byers	54
Fractals, Visualization, and J	Nicholas Cox	64
RECENT MEETINGS		
IBM GSE and APL Club Germany	Adrian Smith	68
Eric Iverson at the BAA	Adrian Smith	71
BAA Vendor Forum	Jon Sandles	76
GENERAL ARTICLES		
Parallel Processing with J	Brisson and Reiter	86
APL and J (1) - Function Rank	Chris Burke	96
Solving Nonogram Puzzles with APL	Alan J Wilson	102
TECHNICAL SECTION		
Hacker's Corner: A Word6 APL Keyboard	Ajay Askoolum	116
Technical Correspondence		119
Advanced Windows Programming with APL and Delphi	Eric Lescasse	122
APL Operators and J	Norman Thomson	143
The J DLL and Extended Integers	Chris Burke	149
At Play with J: Riding a Unicycle	Gene McDonnell	154
Index to Advertisers		159



SOLITON ASSOCIATES

Your Software Solutions Company

Enterprise software for today's business needs.

Providing high performance systems and applications software designed to meet the changing needs of customers and the changing face of Information Technology.

Historically based in mainframe technologies, today Soliton's product line is integrated with industry standard operating environments using both standard and proprietary network solutions.

SHARP APL — a highly productive application development environment for MVS mainframes and UNIX platforms

ViewPoint — a powerful, flexible, yet easy-to-use data manager and report generator

TimeSquare — a UNIX-based database management system, optimized for financial timeseries data, integrated with standard UNIX and PC desktops and with popular relational database managers

Visit our booth and attend our forum at APL96

For more information, telephone:

Soliton Associates Limited

44 Victoria Street, Suite 2100

Toronto, Ontario, Canada M5C 1Y2

Tel: (416) 364-9355 Fax: (416) 364-6159

In Europe

Groot Blankenberg 53
1082 AC Amsterdam
The Netherlands
Tel: + 31-20-646-4475

In the U.S.

951 President Street, #3F
Brooklyn
New York, USA 11215
Tel: (718) 783-9330

1100 University Avenue, Suite 111
Rochester
New York, USA 14607
Tel: (716) 256-6466

sales@soliton.com

Editorial

by Duncan Pearson

We are at an interesting time in the development of computer software. For the first time in ages the users of compiled languages such as C and Pascal have better facilities for the production of good user-interfaces than the users of APL.

It is frightening to compare the development of an environment such as Microsoft C++ with that of the competing APL products over the last three years. The reason it is frightening is that it implies an erosion of the advantage that APL developers have traditionally enjoyed over their counterparts in the mainstream of the software industry. To a certain extent our customers were prepared to put up with a product being a little slower and a little lacking in fancy features if it were delivered on time and if during its development it could change to reflect myriad changes in specification. These things are both valuable to customers, whose requirements change and who need the software for tasks today, and difficult to achieve. If at any stage it is seen that an ordinary developer using Visual Basic or Delphi can be as fast and as flexible in development and yet produce a faster and fancier system, then a number of APL developers will find themselves either out of jobs or using a different language — one more suited to the computer than to the problem solver.

Martyn Adams in an article in *Vector* 9.4 explained why he had had to abandon APL in favour of Visual Basic. That was over three years ago when VB was slower and offered a truly horrendous programming environment in which to organise a large system. When in April I looked at Visual Basic v.4 (as part of some work in developing a Causeway DLL for J) I was staggered by the range of controls available and the extent to which the developer could customise their behaviour. When as part of the same work I compiled and ran my first C++ project (created from a wizard and at that stage with none of my own code) I was in for more surprises. I had something that handled a menu, a docking toolbar, all the horrible file menu functionality and that even registered a file-type so that I could double-click one of my files on the Windows 95 desktop and start the application. *Why can't we have these things in APL? Should we want them anyway?*

In his article in this issue about the use of Delphi from APL, Eric Lescasse examines ways of including some of this kind of interface functionality in an APL system but he also hints that there are ways in which a Delphi application can call routines in an APL server to do manipulations of data. Also for those who use J there is support for this kind of thing built into the language. As Eric

Iverson demonstrated at the Association meeting in April it is possible to use the interface development tools of VB, Delphi or C++ and to call J in a dynamic link library to manipulate data. The resulting system appears as a small fast executable with a small J DLL in tow, is easy to install and uses the buzzword "OLE" technology which makes it even more attractive to clients. Furthermore it allocates memory dynamically so that one doesn't have to guess the size of the user's data in order to set the workspace size.

It is not clear whether the APL interpreter developers are challenged by what ISI have done with J. It is quite possible that the majority of APLers are happy in an entirely APL programming environment. That is clearly what the interpreter developers think and indeed the meeting of GSE (the European IBM user group — reported in this issue) seemed to bear this out. The users of APL2 said that they would prefer Windows features in the APL to the kind of DLL support I am describing. Only time will tell who is right but it is clear that the APL development environments have much that J does not offer. To those used to developing a system in a workspace the J method of defining functions and data in scripts comes as a surprise. In APL to change a function I double-click its name with the mouse, moving to any called function in the same way. In J I have to know in which script the "verb" was defined and then open that script file and find it. The APL session from either Dyadic or from APL2000 offers significant advantages over the J session, APL+Win in the area of syntax colouring and debugging facilities, and Dyalog in the area of workspace exploration (see the review of Dyalog 8 by Bob Byers).

There are challenges here for both APL and J. If half the time that is spent on developing the APL GUI facilities were put into providing the kind of OLE integration that J enjoys then I would never have considered changing to J. These facilities should be available to all APL programmers and if we were given them I believe that the community would be more productive as a whole. We would stop comparing the size of each other's interfaces and get on with designing and talking about fast powerful server systems.

All this playing with J has made me realise that while the languages share a common heritage there is more to the difference than meets the eye. It is not in the spelling or character set, which is mere syntax, but is deeper in the semantics. To help others like me to approach J from the standpoint of an experienced APLer, and to give the pure J folk a flavour of what all these funny APL symbols mean, we have the first of what I hope will be a series of articles exploring common programming problems from the point of view of both languages. There is an article from Norman Thomson in which he applies some of the operator-based thinking of J to APL2 programming and there is an article by

Chris Burke on the use of the Rank conjunction (operator) and how it simplifies some common APL problems.

Since the inception of J in 1990 there has been a debate within the APL community about whether J belongs or not. The annual conferences when they have included J have tended to split into two groups of delegates and I know many readers of *Vector* who completely ignore one or other half of the magazine. The users of one dialect have in general little or no interest in the use of or developments in the other. However I am a firm believer that APL and J belong together. The dialects share a great deal more than divides them. For a user of one dialect, who has already conditioned himself to think in a way that suits the array basis of the language, the main hurdle in the learning of the other dialect is over. I also value *Vector's* position as being one of the few forums in which users of both J and APL can share opinions and read about developments in each other's worlds. Long may it continue as such.

Quick Reference Diary 1996

Date	Venue	Event
July 28 - August 2 1996	Lancaster University UK	APL96 - Designing the Future
October 23 to 25th	Orlando Florida	APL2000 User Conference

Dates for Future Issues of VECTOR

	Vol.13 No.2	Vol.13 No.3	Vol.13 No.4
Copy date	6th Sept 96	6th Dec 96	8th March 97
Ad booking	13th Sept 96	13th Dec 96	15th March 97
Ad Copy	20th Sept 96	20th Dec 96	22nd March 97
Distribution	October 96	January 97	April 97



**Renaissance Data Systems
P. O. Box 421 - V
Georgetown, CT 06829
(212) 864-3078**

**Books on APL and J and other curiosities
of merit!**

Renaissance Data Systems announces a change in its mailing address. Please note that the telephone number remains the same.

If you would like a copy of our latest **catalog**, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as: **APL is EASY**, **APL - An Interactive Approach**, **APL2 at a Glance**, **APL - the Language and its Actuarial Applications**, **APL as a Tool of Thought** proceedings, **I-APL** publications and software, **Boolean Functions and Techniques**, **The FinnAPL Idiom List**, **The Toronto APL Toolkit**, **Mathematical Experiments on the Computer**, **Probability in APL**, **APL - Stat: Do it yourself guide to computational statistics**, **A Source Book in APL** - Approximately 80 titles in all!

We also carry J publications, including **Programming in J**, **An Implementation in J (structure and source code)**, **Arithmetic**, and **Calculus**.

Shareware and commercial APL interpreters and J2 interpreters are available as well.

Vector on the Internet: we're Getting There

from Adrian Smith (Vector Production)

Everyone seems to agree that *Vector* could be a useful Internet resource. Things like the *Product Guide* already have Email addresses included for virtually all the vendors — this is in exactly the right shape to be loaded up onto the Web with all the addresses tagged as 'mailto'. The index to back numbers is useful as it is, but how much better if it was composed of hotlinks to articles! Much of the APL and J code could be copied straight from Netscape, or downloaded as zipped workspaces or .ATF files for APL2.

The British APL Association has agreed to lease the necessary web space, and to have it called 'www.vector.org.uk'. By the time you read this, there should at least be an index page and a 'site under construction' message. Meanwhile, APL-385 is happy to continue to maintain current material on our site at www.demon.co.uk/apl385/vector.htm. At the moment you will definitely find an up-to-date copy of the index (including this Vector) and any workspaces which were supplied with articles.

<h2>Where are we Going?</h2>

We would very much like to post the entire contents of each Vector to the BAA site as soon as it is published on paper. Authors can help us a lot here, as the HTML preparation could be a significant task otherwise. Some of the most helpful things you could do are:

- use plain text where at all possible, with a small subset of HTML tags to indicate formatting. A selection of tags is listed below.
- use an APL font with the same mapping as APL2741. APLPLUS is fine, the new Dyalog fonts are OK apart from the θ character which I will need to watch for and move. This will allow Netscape users to view all the APL code having installed only one of the above as their 'fixed pitch' font.

<p>HTML tags come in pairs! Enclose ordinary paragraphs in <p>para</p> (like this one) and subheadings in <h2>Subhead</h2>. Wrap APL code in <apl>1.12</apl> and J code in <j>1.12</j> so it is easy to spot. Use italic for emphasis (most browsers render this as italic) and for bold. This should get the bulk of the text up to the web with minimal hassle, and I can easily write the Word macros to find the tags and set the appropriate style. Doing it the other way around is much harder!</p>

Causeway

Product Status and Availability Dates

Product	Price (£)	Interpreter	
		Dyalog	+ Win
NewLeaf – desktop publishing for APL	400	Shipping now	Testing at customer sites
RainPro – high quality graphics with 3D capability	250	Pre-release will be available at APL96	October 1996 at Orlando
CausewayPro – development environment	400	Design work complete – coding on hold until after APL96	Design work complete; development to begin after September. Targeted for April 1997
Set of 3	750		

Note that Rain professional will be available to existing registered users at a special price of £100. Causeway professional will similarly be reduced by £50 to £350 for registered users.

Maintenance on any of the individual components is £50 per annum. Maintenance on the set of three taken together is £100.

This advertisement was prepared entirely in NewLeaf in 26 lines of simple APL code. It uses a shadowed frame, some paragraph text and a table with tree-structured headings.

Causeway Graphical Systems Ltd
The Maltings, Castlegate
MALTON, North Yorkshire
YO17 0DP

British APL Association Minutes of AGM

*Held on 17 May 1996 at 1.30 pm
in the Royal Statistical Society*

1. The Minutes of the Annual General Meeting of Friday 19 May 1995 were agreed.
2. The Chairman reported that 1995/96 had been a difficult year because the appeal for new blood to join the Committee had met with a poor response and, in the absence of an Activities Officer, the meetings had had to be suspended. Only one meeting was arranged, that of 19 April at which Eric Iverson gave a J presentation and demonstration. The Committee hoped that Internet communication would facilitate the Committee work and Dave Ziemann and Ray Cannon had agreed to be co-opted to advise on the technical aspects. Vector continued as indisputably the premier APL journal.

The contents that elicited a reader response were the Editorials of both the Vector and Education Vector and the Chairman thanked both editors. Suggestions as to how to reinstate meetings would be welcome. The possibility of evening meetings would be explored. The Chairman noted that David Eastwood was leaving the Committee after 10 years' continuous service and thanked him warmly for his support in many rôles. There were no questions from the floor.

3. Nicholas Small presented the Accounts. He said that there was nothing of particular note but that he was somewhat disappointed with the number of advertisements carried by Vector.
4. Sylvia Camacho named the new Committee which is the same as last year except that David Eastwood has resigned as Publicity Officer and Anthony Camacho has stepped down after 3 years as Vector Editor. Duncan Pearson joins the Committee as Vector Editor and Anthony has undertaken Projects and will help with Publicity. Jon Sandles has agreed to be Activities Officer provided that the job is to co-ordinate some meetings given help from other Committee members to make contact with speakers and to investigate venues.
5. The current auditor, John Sullivan, agreed to continue acting for the Association and this was passed nem con.

British APL Association

Summary of Annual Accounts 1995/6

Summary of income and expenditure/receipts and payments:

	1995/96 (R&P) £	1995/96 (I&E) £	1994/95 (I&E) £
Income/Receipts			
Subscriptions	10741	11871	12425
BCS services	317	317	344
Bank interest	2900	2900	2650
Vector advertising (incl. VAT)	4077	3240	6257
Conferences & seminars	100	0	2522
Other	540	200	1890
Total receipts	18677	18529	26088
Expenditure/Payments			
Meetings	1086	1086	1039
Administration	2340	2340	2486
BCS services	317	317	344
Vector production and despatch	13472	15458	15650
Education Vector supplement	3007	3462	3516
Outstanding Achievement Award	500	0	500
Projects	472	729	2078
Conferences and seminars	0	0	4229
Other	650	488	1680
Total payments	21843	23880	31522
Assets summary:			
Bank and other balances		55771	58938
Debtors		1217	3063
Creditors		(5230)	(4640)
Written off		(250)	(1117)
Net assets		51759	57360

Notes

Pence figures have been omitted, so columns may not add exactly.

The value of stocks of Vector and the Software Library have not been assessed, nor has the value of the Association's computing hardware and software.

For 1995/96, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 1994/95 relate to income and expenditure.

The bad debts written off relate to Vector advertising by APL People.

News from Sustaining Members

Compiled by Gill Smith

APL2000 Inc (Bloomsbury Software Ltd)

APL+Win Release 1.8

Release 1.8 of APL+Win is now available to APL Windows developers. This release adds the popular Windows95 controls on all three Windows platforms: NT, Win95 and Windows 3.*. These include a Treeview control for managing hierarchical lists, a Listview control to help organize columnar data, a Selector control to provide Property Sheets and Wizards as well as Statusbar, Spinner, Trackbar, Imagelist, Progressbar, and Owner Drawn Buttons. These controls can enhance your application and give it the new interface clients expect. This release also adds: tooltips, an unlimited number of user-defined data properties, the ability to suppress events from firing when changing properties under program control, and many other useful properties and events.

To enhance your application when run under Win95, we have added the ability to call 16-bit DLLs and long file name support. Long file names may also be used in NT. None of your existing code needs to change, but using these extended file functions you will get better file error messages (in your native language), wildcard searching for filenames, as well as allowing meaningful names. While these extended file functions work under Windows 3.* (so you only need one workspace) the 8.3 character limit still applies. In addition to all of the above, they also allow you to give APL component files any extension you wish.

As if all this wasn't enough, we're also including a set of APL widget workspaces and utilities. These include an AVI movie player, calendar and date controls, and a workspace to demonstrate the new controls and widgets.

APL+Win Release 2.0: Free to 1996 Purchasers of APL+win 1.8

Later this year we will be releasing version 2.0 to take further advantage of Windows95 and NT. This newer version will include all of the above as well as adding support for OCX/ActiveX controls. ActiveX is the newest standard for adding custom third-party controls to your application. We will also be including new and enhanced widgets including a RichEdit control and an APL forms browser. To give you the opportunity to get started NOW with Windows95, version 2.0 will be free to all users who purchase version 1.8 this year.

APL+UNIX: Release 5.2: Improving Compatibility with Other Platforms

A new release of APL+UNIX is also available. This maintenance release adds control structures (:IF,:WHILE,:FOR, etc.) and numerous small enhancements and corrective services. The addition of control structures makes the UNIX and Windows systems even more compatible, and will enhance your ability to write APL functions which will work on either system.

APL2000 on the World Wide Web

Check out the Web site at [HTTP://MEMBERS.AOL.COM/APL2000](http://MEMBERS.AOL.COM/APL2000). The site includes general information about APL2000, news on upcoming events and releases, technical tips and demo workspaces.

APL2000 Conference

APL2000 will be holding its first user conference in Orlando, Florida from October 23 through 25 1996. The conference will include application developer presentations and training sessions on new and existing products.

Soliton Associates Ltd

General distribution of Release 22 of SHARP APL for the IBM MVS mainframe environment is planned for 4th quarter 1996. Release 22 is Year 2000 compliant. The emphasis of Release 22 development was on performance and operability.

The first customer shipments of the SQAPL Client for SHARP APL UNIX were made in 1st quarter 1996. The SQAPL Client provides SHARP APL with access to standard SQL database products on a wide range of server platforms.

Soliton has announced TimeSquare, a time series database manager for the open systems environment. TimeSquare manages the flow of time-based financial market data through a financial organization, from the data feeds provided by commercial data vendors through to the delivery of market data to end-users.

TimeSquare is an ODBC-compliant SQL server based upon the SQAPL Server for SHARP APL. TimeSquare supports both standard SQL and TimeSquare Extended SQL (a set of proprietary SQL extensions allowing sophisticated time series analysis at the TimeSquare Server).

The open systems networking and interfacing capabilities of SQAPL are provided by DataDirect SequeLink Client/Server Middleware from INTERSOLV Inc. Soliton Associates distributes and supports DataDirect SequeLink for use with SHARP APL.

Dyadic Systems Limited

Dyadic is pleased to announce that the long awaited Version 8 for UNIX is now on Beta Test.

With a few restrictions due to differences in the environments, Version 8 for UNIX is fully compatible with Version 8 for Windows 95 and NT and with Version 7.2 for Windows 3.1. This means that you can develop a GUI application under Windows and deliver it to customers running on UNIX platforms. Indeed, as the workspaces are compatible at the binary level, you have only a single set of application code to support and maintain; Dyalog APL takes care of the rest for you. Under UNIX, your application will have a Motif look and feel, but will otherwise behave the same as it does under Windows.

The beta test release of Dyalog APL Version 8 for UNIX is currently available on IBM RISC System/6000. Versions for SUN SPARC (both Solaris and SunOS), DEC Alpha and HP9000 will be available in due course.

The current release of Dyalog APL/W Version 8 supports OLE Controls (OCX's). In OLE terminology, Dyalog APL (and a Dyalog APL application) acts as an OLE *Client*. In this context, the *Server* is the .OCX module in which the object (or control) resides. In practical terms, this means that you can use OLE custom controls in your Dyalog APL application.

At APL96, Dyadic will demonstrate the next phase of its OLE strategy, namely the ability for Dyalog APL, and more importantly a Dyalog APL application, to act as an OLE Server. Firstly, this means that you will be able to call Dyalog APL directly from another application such as Visual Basic. For example, you could build the user interface of an application in Visual Basic but perform the calculations using Dyalog APL. Secondly, it means that you will be able to build your own OLE Objects in Dyalog APL. These objects will be based upon Namespace technology and be manipulated by other applications through their properties (variables), events and methods (functions).

Causeway Graphical Systems Ltd

It is hard to believe that three months have elapsed since the last time we sat down to compose our *Vector* news. During that time we have made solid progress on our two main consultancy projects, as well as bringing our APL product list significantly nearer to completion.

Our major project of 1996 is to extend and integrate the "Organisational Architect" tool fully into the SAP implementation process. Two months ago, we

reluctantly made a major design decision — we are moving the implementation from Dyalog APL/W to a mixed platform with Microsoft C++ for the visual components and J for the calculation and logic engine. We will continue to use Dyalog to prototype and design new objects and interfaces.

The reasons for making this move are complex, and partly technical, for example we need to support double-byte characters (Japanese) and a multiplicity of keyboards and fonts, including Unicode. We also felt that we were constantly 'pushing the envelope' of what Dyadic offer in the Gui, and that by moving to C++ we could be sure that we had access to **everything** that Microsoft offer. Having decided to re-implement the Gui, we then had a choice of only 1 APL interpreter that can run as an in-process OLE2 server, and that interpreter was J! I think it is fair to say that (a) it was the right decision anyway and (b) we would never have moved if the same facility had been available in symbolic APL.

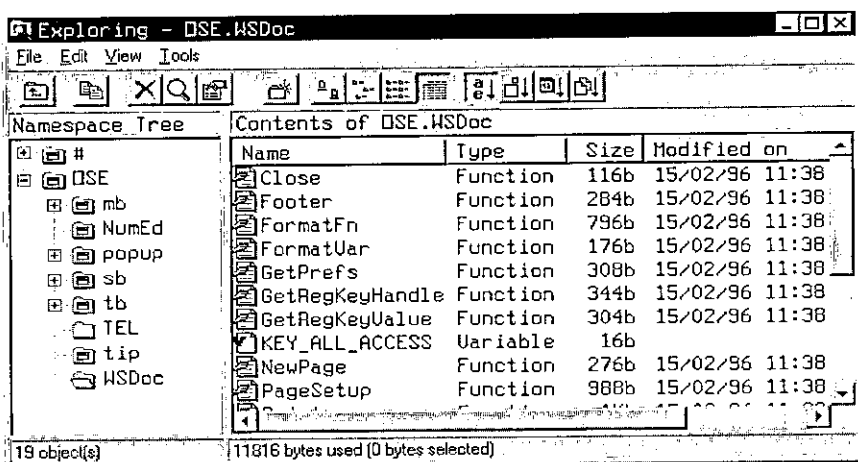
Needless to say, part-1 of the project was to code the core of the Causeway 'change-notify-refresh' logic as a DLL and to ensure that we could continue to benefit from the design principles that have given us such an edge in APL application development. It is interesting to speculate on the possible application of these ideas completely outwith the APL world, particularly given the move to Java which does at least have a partially interpreted approach.

The majority of the product development work has been focussed on getting *Rain* and *NewLeaf* up to a common level of functionality and reliability on Dyalog and APL+Win. *Rain* has already proved to be a major asset in our other consultancy project — migrating VDI-based graphics from APL*PLUS II to APL+Win for a logistics planning package with some very complex charting requirements. Both *Rain* and *NewLeaf* are now with APL2000 customers for pre-release testing, and we are confident that we will have shaken out any remaining problems and deficiencies by APL96.

We will continue to ship Causeway as shareware for Dyalog versions 7.2 and 8, and we are targeting the APL2000 user conference as our release date for Causeway for APL+Win as well as the professional version of *Rain* (including 3D charting) and *NewLeaf*. Now that we have a solid platform for 95/NT we can begin to move ahead with Causeway/Pro for Dyalog — we see Causeway, *Rain* and *NewLeaf* as strongly complementary products and we urgently want all three on both the major APL platforms, sharing common documentation and support.

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- **Organise** your workspace into self-contained sub-systems
- **Encapsulate** functions and variables *within* GUI objects
- **Isolate** utilities from your application code

Plus The new Version 8 **Explorer** lets you browse namespaces, drag-drop objects from one to another, and a whole lot more.

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



THE EDUCATION VECTOR

July 1996

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.13 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Sylvia Camacho, 11 Auburn Road, Redland, BRISTOL, BS6 6LS. Tel: 0117-973 0036.

Contents

Editorial	Ian Clark	18
Functional Programming in J	Howard Peelle	20
J-ottings 10	Norman Thomson	29

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 01388-527190

Email: 100021.3073@compuserve.com

Editorial

The last editorial did a demolition job on OFSTED, not undeserved, but maybe unfair on its hard-working inspectors. What was not said (and what should have been said) was that OFSTED is an attempt to replace the irreplaceable — like trying to grow corn on cleared rainforest to provide ground-cover. When the local authorities were rolled back as part of Thatcher's drive to 'roll back socialism' some priceless things got rolled back with them. For instance the network of advisory units on IT. The advice dispensed was free, useful, supplier-independent, unobtainable elsewhere and locally-based. When you destroy a distributed resource you have to put something centralised in its place, even if it's there only to deflect the blame onto broader shoulders for the consequences of what you have done. Hence OFSTED. End of story. Only it isn't, because the editorial in question threw down a gauntlet and vowed to take it up in this issue.

This publication is about APL (which we take to include its new dialect, J). What it is not about is peddling a nostrum for all the ills of the educational world. Have we ever said: "APL is the solution — now what's your problem?" No, but we have explored why this country seems to be such poor soil to nourish the seeds of excellence in Mathematics, Science, Technology and Engineering, and proposed solutions. And we have an idiom for discussing these solutions. It happens to be APL.

But it might have been BASIC. What with the breakneck pace of computer technology, we are all inclined to forget what an achievement the BBC Microcomputer was. For roughly a seven-year span, 1983 to 1989 say, entrants into further education could be relied upon to possess at least one formalism in which computer ideas could be precisely discussed, viz. BBC BASIC. It made the development of effective courseware in technical subjects remarkably easy, in spite of the severe disadvantages from which BASIC suffers as a language. BASIC is too voluminous. Anything beyond the simplest arithmetic needs scores of lines of text to represent, each line by itself superficially clear, but taken as a whole so dilute that the student has difficulty seeing the wood for trees, let alone typing it all in.

Nevertheless, if you're asked for an example of where a nationwide initiative gave a real boost to education, point to the foresight of the BBC to sponsor a standard vehicle for its planned series of educational programmes. Why did it work? Not because of any unique virtue of BASIC. They used the wrong language (but we would say that, wouldn't we?).

Yet the dear old 'Beeb' was a catalyst of innovation, instead of laying a burden of paperwork upon teachers to smother initiative. Well before the National Curriculum (NC) came along, whilst the benefits of the BBC initiative were still making themselves felt, teachers were taking introductory IT courses, in their own time and sometimes at their own expense, because they felt it was empowering. What I mean is that it looked as if it might be good for their work, their careers and good for the pupils too. But there were anxious questions. What exactly was the role of computers in classroom teaching supposed to be? Was the teacher destined to become obsolete?

On my introductory IT course to practising teachers, I once opined that the real place of a computer in the classroom was not as a robot teacher, but as an administrative slave, taking the strain of routine tasks like attendance, drill, assessment, progress tracking and reporting, "leaving you, the teacher, free to do what you're good at."

One student nearly fell off his chair. "To think we've got to come on a computer course to be told that!"

The huge expenditure of time, effort and money that went into the creation of OFSTED, the NC, key stage examinations and published league-tables, not only by the government on coercion, but by every school in the country on compliance, was a total waste, at least prior to Sir Ron Dearing's revision. It remains largely futile because it makes a mockery of the teaching profession in England and Wales. What about other professions? The law of the land requires no doctor to cure me if I'm ill, no lawyer to win my case in court. But if I am a child my teachers are required by law to train me and rate me in a tightly prescribed set of attainment targets, which are supposed to define a sound education. (What is a sound education? Discuss.)

The resources to erect this monstrosity would have been better spent on commissioning and deploying a standard computer system to take the waste of teacher time out of class record keeping. Or, if that's too hard (the Inland Revenue seems to have found it so, with its PC program to do self-assessment) we could have had a national library of software to support all aspects of a recommended, or even a commended curriculum, seamlessly spliced with its delivery vehicle in schools. Much as the French government installed MINITEL and the US Defense Department installed ARPANET in their respective countries. If only we'd gone the whole hog with an empowering infrastructure like MINITEL, and tinkered with a prescriptive straitjacket like the NC, instead of doing it the other way round. Then we wouldn't have needed to take two bites of the cherry and call a moratorium on future changes to stop the project foundering. Just the opposite. The vehicle would have been a living organism, its growing tips practising teachers, instead of the withering of buds and the lopping of sound wood which has been the fruit of the NC and its gendarmerie, OFSTED.

In APL (and J) we have an empowering idiom to formulate such a vehicle. It's time we used it.

Introduction to Functional Programming in J (Part I)

by Howard A. Peelle (hapeelle@educ.umass.edu)

This is the first of a three-part article. Part I presents fundamentals of J in a framework of functional programming strategies, with examples aimed toward developing an illustrative program to generate prime numbers. Part II (to appear in the next issue) will demonstrate how to develop the program directly, characterizing a J programmer's thinking along the way, and will continue development of the same program with higher-level strategies. Part III (in a following issue) will present alternative algorithms for iterative, recursive, and array-processing programs and will end with meta-strategies for packaging and documentation.

0. Think Arrays

As an *a priori* programming strategy, a programmer must decide how to represent data for a particular task. (This strategy is numbered 0 to indicate that it may well occur before actual programming begins.) It involves thinking ahead about what structures are best suited for processing the given data and for reporting results. In J, data structures are "arrays" — that is, single values (scalars), lists (vectors), tables (matrices), 3-D and higher-dimensional arrays. To 'think arrays' means to plan to process data as a whole array where possible. In this article we will use lists primarily.

A list of numbers is represented with a space between each number. For example:

2 3 5 7 11

NB. A list of integers

Other kinds of numbers (not needed in this article) are represented as follows: Negative numbers are preceded by an underbar, as in 3. Real numbers use a decimal point and leading zero, if necessary, as in 0.5. Very small and large numbers are represented in scientific notation, as in 1e6 for one million. Complex numbers use engineering notation, as in 3j4 for 3 plus 4 times the square root of negative 1.

Note that comments (following NB.) can be used for explanation.

A list of characters is represented within single quote marks. For example:

'primes' NB.

A list of characters

A table is represented in rectangular rows and columns. For example:

2	3	4	5
6	7	8	9
10	11	12	13

NB. A 3 by 4 numeric table

(Ways to create tables will be presented in Part II and Part III.) Note that no declarations and no data typing are necessary in J. Different data types may be mixed together in a "boxed" array. For example:

primes	2 3 5 7 11
--------	------------

NB. A list of two boxed items
NB. each containing items

This enables representation of non-rectangular arrays. (See [1] for an introduction to boxed arrays, which are not needed in this article.)

1. Select Primitive Functions

A first strategy for programming in J is to select primitive functions appropriate for the task at hand. J provides over a hundred mathematical functions as primitives, that is, symbols on the keyboard. Included are: arithmetic functions Plus, Minus, Times, Divide, and Power; relational functions Equal, NotEqual, LessThan, GreaterThan, etc.; logical functions And, Or, Not, etc.; standard functions such as SquareRoot, random numbers, and greatest integer part; handy functions such as Square, Increment, Halve, and Factorial; powerful functions such as Matrix Inverse and Matrix Divide for linear algebra and statistical regression; permutation and sorting functions; trigonometric and circular functions; and many other functions for manipulating arrays, formatting output, complex arithmetic, etc.

Only a dozen or so primitive J functions are introduced here in this article. (See [1] for a complete list.) Note that symbols can be utilized alone or with a dot (.) or a colon (:) as a suffix to denote different functions, as in * (Times) or *. (And) or *: (Square).

In J, functions are called "verbs", and they use "nouns" (data) or "pronouns" (variables) for arguments (inputs). A student may explore the behaviour of any verb by entering expressions interactively, where an expression is indented automatically and the computer prints the result on the next line. For instance, consider the Remainder verb (|) which uses a noun on the left and on the right:

0	3 9	NB. 3 Remainder 9 NB. Result is 0
1	3 10	NB. 3 Remainder 10 NB. Result is 1
2	3 11	NB. 3 Remainder 11 NB. Result is 2
0	3 12	NB. 3 Remainder 12 NB. 3 goes into 12 evenly

These expressions may be combined using a list:

0 1 2 0	3 9 10 11 12	NB. 3 Remainder of a list NB. Result is a list
---------	----------------	---

A longer list reveals more of a pattern:

0 1 2 0 1 2 0 1 2 0 1 2 0	3 0 1 2 3 4 5 6 7 8 9 10 11 12
---------------------------	----------------------------------

The primitive verb *i.* (called Integers) is convenient here:

0 1 2 3 4 5 6 7 8 9 10 11 12	<i>i.13</i> NB. Nonnegative Integers up to 13
------------------------------	--

Substituting this illustrates how verbs may be used together:

0 1 2 0 1 2 0 1 2 0 1 2 0	3 <i>i.13</i> NB. Remainders of Integers 0 to 12
---------------------------	---

Note that here *|* is a "dyad" verb (binary function) which — like Plus (+) — uses two inputs (here a single number on the left and a list on the right); whereas Integers (*i.*) is a "monad" verb (unary function) which uses one input on the right — like -4 and $\sqrt{4}$ and $f(x)$ in conventional notation. All verbs in J are either monadic or dyadic. Many use the same symbol for both a monad and a dyad, which can be distinguished by use (with one input or two inputs), as in $7--4$ which is 7 Minus Negate 4.

Further, note that the rightmost verb is executed first in order to produce the right input for the next verb. For instance, above, Negate before Minus, as in $7-(-4)$ and, earlier, Integers (*i.*) before Remainder (*|*), as in $3|(i.13)$. Indeed, J adopts the following general rule: *every verb uses the (noun) result of the entire expression on its right*. This eliminates the need for a hierarchy (such as exponentiation before multiplication and division before addition and subtraction) but appears controversial (see [2] for discussion of maths teachers' reactions to learning J notation). Nevertheless, the issue can be avoided by using parentheses redundantly to indicate order of operations (in the normal way), as shown in this article.

2. Select Primitive Operators

In addition to primitive functions (verbs), J provides operators which modify verbs. Operators effectively create new verbs which often enable programming tasks to be accomplished more directly. For instance, Insert (/) is an "adverb" operator which inserts a verb between successive items of an array. It requires a verb as an input (on its left). For example:

```

      +/ 2 3 5 7 11      NB. Plus-Insert a list
28                                NB. Sum

```

Here the verb + (Plus) is the input for / (Insert). The two symbols +/ together produce a new verb (called Sum) which then applies to the given numbers, in effect producing $2+3+5+7+11$, which is 28. Programmers might compare this with how it is done in other programming languages and note that J allows direct use of an operator instead of iteration with explicit control structures. (See forthcoming Part III for more discussion.)

Other dyadic verbs may be used with Insert, so the programmer can produce more new verbs. For example:

```

      */ 2 3 5 7 11      NB. Times-Insert a list
2310                               NB. Product

```

In effect, this repeats * as many times as needed to insert it between all items in the input, and then $2*3*5*7*11$ is evaluated as $2*(3*(5*(7*11)))$. Other operators will be introduced, as needed, in the following sections. (See [1] for a complete list and note that all operators are performed before functions.)

3. Use Parallel Processing

A natural programming strategy in J uses "parallel processing", where a verb applies automatically to every item in an array. Recall the example of Remainder (!) from Section 1: $3 \mid 9 \ 10 \ 11 \ 12$ is equivalent to $(3 \mid 9), (3 \mid 10), (3 \mid 11), (3 \mid 12)$ where a single number on the left is used with each number on the right. In general, the verb applies every item on the left in one-to-one correspondence with every item on the right — in parallel. For example, when both inputs have the same number of items:

```

      1 2 3 4 | 9 10 11 12      NB. (1|9),(2|10),(3|11),(4|12)
0 0 2 0                               NB. Parallel remainders

```

This pertains to other primitive verbs as well, such as:

```

      1 2 3 4 + 9 10 11 12    NB. (1+9),(2+10),(3+11),(4+12)
10 12 14 16                  NB. Parallel sums

```

```

      1 2 3 4 # 9 10 11 12    NB. (1#9),(2#10),(3#11),(4#12)
9 10 10 11 11 11 12 12 12 12    NB. Parallel copies

```

Here the verb # (called Copy) makes copies of each item respectively. It is also particularly useful for selecting items:

```

      1 0 0 1 # 9 10 11 12    NB. (1#9),(0#10),(0#11),(1#12)
9 12                          NB. Select 9 and 12

```

Each 1 selects its corresponding item, and each 0 eliminates an item. (This is often called a "bit mask".) Another example:

```

      0 1 1 0 1 0 1 # 1 2 3 4 5 6 7    NB. 0s and 1s Copy 1 2 3...7
2 3 5 7                          NB. Select primes

```

4. Name Utility Functions and Variables

Programmers often enter or retrieve utility functions (programs) and variables they need as tools for tackling a programming task. J uses =. for local assignment and =: for global assignment of variables (called "pronouns"). For example:

```

p =: i.12                      NB. p is Integers 0 1 2 3...11

```

To print out the contents of a pronoun, simply type its name:

```

p                               NB. What is in p?
0 1 2 3 4 5 6 7 8 9 10 11     NB. The assigned integers

```

A pronoun may be changed, of course. For example, use the primitive verb Increment (>:) to add 1 to each item in the list:

```

p =: >:p                       NB. p is Increment p
p                               NB. What is in p now?
1 2 3 4 5 6 7 8 9 10 11 12    NB. New integers

```

Asking for the value of a pronoun which has not been assigned results in an error message. For example, note that J is case sensitive:

```

P                               NB. What is in (capital) P?
value error                     NB. P has no value

```

This is one of about a dozen possible error messages, including "domain error", "syntax error" etc. J reports an error message whenever it cannot interpret an expression but lets you enter another expression immediately.

New functions ("pro-verbs") in J are defined just as easily using `=.` or `=:` which can be read as 'is'. For instance, suppose you want to use a name instead of a symbol for a verb:

Remainder `=.` | NB. Remainder is |

Now this name can be used instead of the | symbol. For example:

3 Remainder 12 NB. Same as 3 | 12
0 NB. 3 divides into 12 evenly

It can be used as a function in any expression. For example:

(3 Remainder 12) = 0 NB. Is (3 | 12) Equal to 0?
1 NB. Yes

Note that the result of Equal (`=`) is either 1 (representing "true") or 0 ("false"). Other relational functions, such as LessThan (`<`) and GreaterThan (`>`), return Boolean results also. See [1].

Since some programmers may prefer using the name "mod" (short for "modulo") with the inputs switched (as in 12 modulo 3), a slightly different verb could be defined:

Mod `=.` |~ NB. Mod is Remainder-Commute

The symbol `~` is an adverb (called Passive or Commute) which switches the inputs of a verb. For example:

12 Mod 3 NB. 12 |~ 3 (same as 3 | 12)
0

Recall the Copy verb (`#`) from the previous section, which used 0s and 1s on the left to select items from an array. We may use `#~` to select items, where 0s and 1s are given on the right side instead:

1 2 3 4 5 6 7 #~ 0 1 1 0 1 0 1 NB. Integers selected by 1s
2 3 5 7

It is convenient to give this verb a name:

If `=.` #~ NB. If is Copy-Commute

Now you can use the name as a utility function:

1 2 3 4 5 6 7 If 0 1 1 0 1 0 1 NB. Select integers If 1
2 3 5 7

5. Compose Functions (Define Programs)

J provides facilities for defining traditional multi-line programs with variables and control structures (see Part III). But these are not needed here since we can define short (one-line) programs without control structures by composing functions. This is a quintessential strategy in J programming. Furthermore, J allows definition of a program ("pro-verb") without explicit reference to inputs or variables. This is sometimes called "pure functional programming".

J offers several kinds of functional compositions. To begin with, the operator @ (called Atop) composes one verb after another — like function composition in mathematics. For example, let's define a simple utility verb to generate positive integers:

```
Pos =. >: @ i.      NB. Pos is Increment Atop Integers
                    NB. (Extra spaces for readability)
```

Atop (@) is a "conjunction" operator which requires two verb inputs. It causes the verb on the left (here >:) to be done after the verb immediately on the right (here i.), that is, Increment after Integers. Now execute this new verb with an input:

```
Pos 12      NB. Positive integers 1 to 12
1 2 3 4 5 6 7 8 9 10 11 12
```

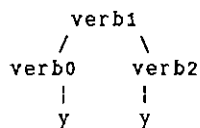
Another convenient form of composition — called a "fork" in J — occurs when three verbs are written in an isolated sequence. The verb in the middle uses the results of the verb on the left and the verb on the right. For example, (>: * <:) is a fork which executes Times with the results of Increment and Decrement. (<: is the primitive verb Decrement which subtracts one from its input.) Give this fork a name:

```
F =. >: * <:      NB. F is Increment Times Decrement
```

Test it with one input:

```
F 4 NB. (>: 4) * (<: 4) or 5*3
15
```

In general, for one input y , a fork (verb0 verb1 verb2) y is equivalent to (verb0 y) verb1 (verb2 y), as seen in the following diagram:

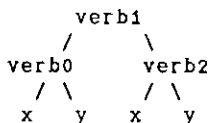


A fork may be given two inputs. For example:

F =. + * -
7 F 4
33

NB. F is Plus Times Minus
NB. (7+4) * (7-4)

In general, for two inputs x and y , fork x (verb0 verb1 verb2) y is equivalent to $(x \text{ verb0 } y) \text{ verb1 } (x \text{ verb2 } y)$. Diagram:



Now consider the fork ($| = 0$) which executes Equal with the results of Remainder and Zero. (0 : is a constant verb which returns a 0 result.) Give this fork a name:

Divides =. | = 0: NB. Divides is Remainder Equal Zero

This program uses two inputs. For example:

3 Divides 12
1

NB. (3|12)=(3 0: 12) or (3|12)=0
NB. True (0=0)

It also works with a list input — by parallel processing — to test the divisibility of each item:

(Pos 12) Divides 12
1 1 1 1 0 1 0 0 0 0 1

NB. (1 2 3...12 | 12) = 0
NB. 1s indicate divisible items

This suggests defining another program using a fork within a fork:

Divisible =. Pos Divides] NB. Divisible is
NB. Pos Divides input

Note that $]$ is a primitive verb which returns the input itself. This program uses one input:

Divisible 12
1 1 1 1 0 1 0 0 0 0 1

NB. (Pos 12) Divides 12

Forks may be composed into increasingly complex sequences. This next program uses another fork:

Divisors =. Pos If Divisible

This is equivalent to the following definition, where the rightmost sequence of three verbs is composed first:

```
Divisors =. Pos If Pos Divides ] NB. Divisor is Pos If
                                NB. (Pos Divides Input)
```

(See Part II for how to display the structure of such a program.) Examples of its use:

```
Divisors 12      NB. (Pos 12) If ((Pos 12) Divides 12)
1 2 3 4 6 12     NB. Divisors of 12
Divisors 13      NB. (Pos 13) If ((Pos 13) Divides 13)
1 13            NB. Divisors of 13
```

This program is a new verb which may be combined usefully with other verbs. For instance, the primitive verb Tally (# used with one input) counts the number of items:

```
#Divisors 12      NB. Tally Divisors of 12
6
#Divisors 13      NB. Tally Divisors of 13
2
```

Now compare the results with two:

```
(#Divisors 12) = 2  NB. Is (Tally Divisors 12) Equal 2?
0                  NB. No
(#Divisors 13) = 2  NB. Is (Tally Divisors 13) Equal 2?
1                  NB. Yes
```

Let's compose the verbs and define a program to detect a prime number (which has exactly two divisors):

```
Prime =. #Divisors = 2:NB. Prime is
                        NB. (Tally Atop Divisors) Equal Two
                        NB. where verb 2: returns 2
```

Examples of its use:

```
Prime 12          NB. Is 12 prime?
0                 NB. No
Prime 13          NB. Is 13 prime?
1                 NB. Yes
```

Now we are ready to develop a program for generating a list of primes. This will be presented (from the start) in Part II, along with higher-level programming strategies: testing, debugging, modularizing, simplifying, optimizing, and generalizing.

References

- [1] Iverson, K. E. *J Introduction and Dictionary*, Iverson Software Inc., Toronto 1995
- [2] Peelle, H. A. "Teaching a Computer Notation for Secondary Mathematics" submitted to *Journal of Computing in Teacher Education*, International Society for Technology in Education, Eugene, OR

Professor Howard A. Peelle
University of Massachusetts
Furcolo #10
Amherst, MA 01003
(413) 545 — 1114

J-ottings 10

by Norman Thomson

The last article in this series was very powerful; this one, for reasons which will soon be apparent, is going to be full of interest.

First, I want to develop the theme of "little things that do so much". Take the hook

++:

which realises in J the phrase in algebra $k(k+1)$.

In mathematics it is often desirable to sum series of such terms from 1 to n , say, so define an adverb

```
upto=. (5>.)@1.
```

and a verb of the sort which seems to make things more acceptable to general readers and sceptical colleagues:

```
sum=: +/
```

so that

```
sum (++) upto 5
70
```

gives the sum of the series $1.2 + 2.3 + 3.4 + 4.5 + 5.6$.

You can now slot between the parentheses a description of an individual term of a series of your choice. For example

```
sum (^&4) upto 5    NB. means sum the fourth powers of 1,..5
979
sum (2&^ ) upto 5    NB. means sum the series 2 to the powers 1,..5
62
```

You can have endless fun with hooks, but be careful of the differences between composition and hook. For example, `sum (%>: )..` means sum the series $k/k+1$, while `sum (%@>: )..` means sum the series $1/k+1$.

As another illustration `%^` and `%@^` define values according to the following table:

	monadic	dyadic
<code>%^</code>	$y.e^{-y}$	$x.e^{-y}$
<code>%@^</code>	e^{-y}	$x.e^{-y}$

Parentheses can also make subtle differences:

```
sum (+*:^:2) upto 5    NB. sum n+ n^4
994
sum ((+*: )^:2) upto 5    NB. sum ({n(n+1)})({n(n+1)}+1) = n(n+1)(n^2 +n+1)
1554
```

Standard mathematical series fit within this framework, for example:

```
sum (%!) upto 5        NB. series for e = 1 + 1/1! + /2! + ..
2.70833
(sum% upto 9)-0.57721    NB. sum of recip's->ln(n)+Euler's const
2.25176
^2.25176                NB. e to power 2.25176 is approx 9
9.50445
```

Or take a well known series approximation to pi using the sums of the reciprocals of the integers:

```
%: 6*sum ^&_2 upto 100
3.13208
```

While summing series with a computer is no substitute for analysis, it can become a very supportive, as well as a compulsive, activity.

Now let's come to earth and do the interesting bit! Suppose that the series to be summed calculates the present worth of future payments. For example, if the annuity rate is 8% what is the present value of an income of £1 per annum for the next ten years? This requires the summation of the series $\{1/(1+i)\}^k$ for $k=1..10$. Hence define the `1/1+i` verb described above

```
g=.%@^
```

which anticipates inputs of, for example 1.08 and 10, and follow the pattern established above:

```
sum (1.08&g) upto 10
6.71008
```

Now suppose that the income is to be paid more frequently. The appropriate root of the interest rate $1+i$ has to be taken, and the premium divided by the frequency of the payments.

For example if income is paid monthly:

```
12%~ sum (12%:1.08)&g upto 120
6.9527
```

or for quarterly income:

```
4%~ sum (4%:1.08)&g upto 40
6.90816
```

APR (Annual Percentage Return) calculations can be performed in the same fashion. For example borrowing £1 at an APR of 20% with monthly repayments results in a total repayment of

```
sum 12%~12%:1.2&^ upto 12
1.10532
```

showing that a nominal rate of 10.53% is equivalent to an APR of 20%.

Obtaining APRs from nominal rates requires the solution of polynomial equations. Writing a verb to perform Newton-Raphson root finding is greatly aided by the derivative adverb `D.`. A root-finding adverb is discussed in detail in [1], and is updated here to run in J for Windows:

```
newton=.1 : 'J-x.%x.D.1' (^:_)
```

The `1 :` indicates that the string following defines an adverb, whose relation to the formula for Newton-Raphson iteration will be immediately clear to anyone who is familiar with this topic. The final "power to infinity" adverb in parentheses means that iteration is to be continued until convergence.

A verb which models a polynomial uses `#.`, for example

```
ff=.#.&1 _1 _2
```

where `1 _1 _2` are the coefficients of powers in descending order, so that the polynomial represented above is $x^2 - x - 2$.

Everything is now in place to calculate APRs from nominal rates of interest. The polynomial to be solved is

APL Product Guide

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: 100331.644@Compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDMM and the full line of Interprocess APL2 enhancements.
Optima	IBM Compatible	poa	Complete PC-based station, APL Interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 385PC!
	Run-time Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Atlantis Software	Analytic Platform (K)	poa	K is an APL-like language
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	260	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v1.8	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Any user purchasing APL+Win during 1996 will receive free updates to Vn 1.8 and Vn 2.0 (user to pay carriage)
	Upgrade to Version 1.8	540	From earlier versions of APL+Win. Free update to Version 2.0 (user to pay carriage)

	Migration to APL+Win	620	from APL*PLUS II versions 4/5. Free update to Version 2.0 (user to pay carriage)
		750	from earlier versions of APL*PLUS II
	APL+DOS	1300	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	620 / 390	from APL*PLUS/PC or APL*PLUS II
	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft OY	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$165	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APIs; TIME facility
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J Professional (Inc runtime)	\$495	
	J Personal Edition	\$100	

	J Personal (disks only)	\$40	The text of the manuals is available as Windows Help, so the paper copies are no longer a necessity.
	APLWIN	\$30	For 386/PC under Windows 3.1
	APL Reference Manual	\$30	Documentation for all the above.
	J System Kit	\$24	J 6.2 diskette with manual "J:Introduction and Dictionary"
	J Source Code	\$90	Full C source code plus 100-page book
J Austria	J	poa	Distributor for Austria and Switzerland
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis Systems	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
RE Time Tracker Oy	APL*PLUS/PC	poa	Complete APL*PLUS and Statgraphics product range and user support for Finland
	APL*PLUS II/OS		
	APL*PLUS III/WIN		
	APL*PLUS/UNIX		
Solliton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.

Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
APL-385	APL-385 for APL*PLUS/PC FSM-385 DRAW-385 DB-385 GEN-385	50	Including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualed	\$1500-4000	Electronic Data interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	pee	The First APL compiler
(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	Causeway for Dyalog/W	30/\$50	Registration for the Causeway platform supplied with Dyalog APL/W as shareware.
	Rain Graphics (shareware)	30/\$50	Annual registration covers full on-line documentation (Windows Help) and handy-reference for the Rain graphics workspace supplied with Dyalog/W. Life registration is £150/\$250.
	Rain Graphics (professional)	250	Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Price reduced to £100 for registered users.
	NewLeaf for Dyalog/W	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom.

Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support system for top management. Developed in Italy over 7 years. Requires APL mainframe or APL*PLUS/II. Optional modules: EIS, Excel Interface, DTP output via LATEX, output on map background.
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	IAC/Graf	15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$199	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$99	As above for APL*PLUS/PC.
	(DLL Version 1)	\$199	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs
	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as SequelLink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
Interprocess Systems	APL2 Development Workstation	poa	
	IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger

(mainframe)	AFM	\$15300	High performance component and keyed file system (VS APL and APL2)
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	WSORG	\$1500	Full-screen Workspace Organizer for APL2.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lingo Allegro	FRESCO Business Graphics	poa	Fast and Easy Business Graphics DLL
	AP126/PC	poa	GDDM Interface for Dyalog APL/W
	AP127/PC	poa	ODBC Interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	TOPR	poa	APL Code and Application Management for Dyalog APL/W
Mercla	LOGOL 92	poa	Logistics management system for 386/486 & RISC computers, Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TW/GS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
RE Time Tracker Oy	UIT/W	poa	TMT-Team Oy's User Interface Toolkit for APL*PLUS II and PLUS III under Windows. Comprehensive spreadsheets, replicated fields, special field types, etc.
	DB+	poa	TMT-Team Oy's database interface for APL*PLUS II & PLUS III under Windows. Interfaces to almost twenty different databases.
Soliton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL
	STSC's SHAREFILE	poa	component files for IBM VSAPL and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
(for APL*PLUS II)	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)
	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)

	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
(for Dyalog 7.2 & 8)	DLL Parser version 2.0	FF1450	Auto-generates <i>DATA</i> instructions for DLLs and gets constant values and structures.
	DELPHI Forms Generator	FF950	Takes a Delphi form and turns it into a Dyalog APL program which builds an identical form.
	Unihware Toolkit/W Vn. 2.4	FF39000	Fast relational database system, and complete set of Windows tools for RAD in Dyalog APL/W.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9519067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	APL Programming	poa	Short or long-term consultant available.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, specialises in tree-processing algorithms.
APL People	Consultancy	poa	Consultants available at all levels. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics; PC and mainframe
Causeway	Consultancy and Training	poa	Management and upkeep of the Causeway development environment for individuals and large APL sites. On-site training for Causeway/Dyalog and Causeway/Plus III. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	poa	24-hr programmer: APL, C, SmallTalk, Graphics; PC, mini, mainframe and network.
David Crossley	Consultancy	poa	Broad experience in many APL environments
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Dinosoft OY	Consultancy	poa	Specialised in very large databases.
Dogon Research	Consultancy	poa	APL Systems consultancy, design, implementation, support, documentation and maintenance. All dialects with special emphasis on APL2 and Dyalog APL/W.
Dyalic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.

E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL.
Evestic AB	Consultancy	poa	Excellent track record from 10+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
Graymantle Assoc	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC, HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Kestrel	Consultancy	poa	All APLs, all environments. Design, analysis, coding, maintenance, documentation, training, interfacing.
Lingo Allegro USA	Consultancy	poa	General APL consulting: Migration and Downsizing; Performance Tuning.
Mackay Kinloch Ltd	Consultancy	170-320	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consultancy
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis Systems	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
RE Time Tracker Oy	Consultancy	poa	Specialised in comprehensive APL Windows user interfaces, APL Multimedia, APL to API level interfacing for Windows, Windows applications, DLLs & databases.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL

Snake Island Research Inc	Consultancy	poa	APL Interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation. APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms, 17-23 years experience.
Unware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickdiffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Employees placed at all levels.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	ComLog-Logger	\$25.95+p&p	APL*PLUS II combi-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — In English or French.
	The BBS\APL:	\$24 p.a.	703-528-7617, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Kestrel	Employment	poa	Permanent and contract, home and abroad. From individual placement to supply of complete project teams.
	Software Library	poa	Low-cost software distribution service; call for details.
Oasis Systems	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Solliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	International	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
APL Interest Group	South Africa	-	-	
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Chicago SIG	Chicago			
Hartford Group	Hartford, Connecticut, USA			
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc.	Tokyo, Japan			
Melbourne APLUG	Melbourne, Australia		Quarterly meetings	free
New York SIG	New York, USA			
Potomac SIG	Washington DC, USA		Free monthly meetings	
Rochester APL	Rochester, New York			
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SOCAL	Southern California		Seminars	\$15 (\$5 students)
SovAPL	Oblninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qdy SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: +1 (212) 626-0606 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX, UK. Tel: 01245-496647; Fax: 01245-496648.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
Adaytum Systems		13 Great Georges Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adlee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31-3474-2337 Fax: +31-3474-2342
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO6 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group		
APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 928-2059 Email: frgp21a@prodigy.com

APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Club Germany	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108
APL Interest Group, South Africa	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803-7200 Fax: +27 (11) 803-9134 Email: mikemont@spl.co.za
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS UK. Tel: 01225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: +33 1 69-08-79-63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
The Bloomsbury Software Co Ltd	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524; CompuServe: 100010,1467
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS8 6LS UK. Tel: 0117-9730036. email: acamacho@cx.compuink.co.uk Reuternet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874697 Email: 100430.740@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Causeway Graphical Systems Ltd	Adrian Smith, Duncan Pearson	5 The Maltings, Castlegate, MALTON, North Yorks YO17 0DP UK. Tel: 01653-696760 Fax: 01653-697719 Compuserve: 100265,1564
Chicago SIG	Larry Mys	836 Highland Drive, Chicago Heights, IL 60411 USA C'serve: 73040,3032
Cinerea AB	Rolf Komemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: +33 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: +46 31-45 37 40. Fax: +46 31-45 24 23.
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA UK. Tel: 0171-229 5344
Danish User Group	Per Gjerlof	Email: gjerper@inet.uni-c.dk
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 6EX UK. Tel: 01604-760241
Dinosoft OY	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 0 70028825 Fax: +358 0 70028824
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP UK Tel: 0181-520 6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orlington, Kent BR5 1JS UK. Tel: 01899-824741
Evestic AB	Ole Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx, UB5 4QS UK. Tel: 0181-864 9537
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca

Greymantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-876065 Email: 100412.1305@compuserve.com
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Mallprod F465, Springfield, MA 01111 USA Tel: +1 (413) 788-8411x2836
H.M.W.Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Email: dick.holt@aol.com
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-527190. Compuserve: 100021.3073
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS8 6LS UK. Tel: 0117-9760036 email: acamacho@cix.compulink.co.uk Reutemet (Share): ACAM
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (+2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD UK Tel: 01732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aim@infostroy.spb.su
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: +45 42 10 70 22 Fax: +45 42 10 75 74 Email: insight@inet.uni-c.dk
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 120 Wapping High St, London E1 9NH Tel: 0171-265 1120
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: +1 (404) 410-1700. Fax: +1 (404) 410-1773 Cserve: 70373,2676
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559
J Austria	Joachim Hoffmann	Münzgrabenstr. 68, A-8010 Graz, Austria. Tel: +43 (0)316 814529 Fax: +43 (0)316 816683 Email: JoHo@magnet.at
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Association		23-2-302 Hlromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessamer Drive, Stevenage, Herts SG1 2DX UK. Tel: 01438-310155 Fax: 01438-310131
Lingo Allegro USA Inc.	Victoria H. Fil	113 McHenry Road, Suite 161, Buffalo Grove, IL 60089 USA Tel: +1 (708) 459-7529 Fax: +1 (708) 459-8501 Email: info@lingo.com
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voice: 01426 98 3858 Compuserve: 100010.33
MasterWork Software Ltd	Fraser Jackson	PO Box 56-038, Tawa, Wellington, New Zealand. Tel and Fax: +64 (4) 232-4440 Email: 100242.2535@compuserve.com
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 588 7574 Fax: +61 3 588 7600 Email: hld@dar.csiro.au
Mercla Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants UK. Tel: 01730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Oasis Systems	Theo Zwart	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: 100447.431@compuserve.com
Optima Systems Ltd	Paul Grosvenor	Alport House, Purley Way, Croydon, Surrey CR0 0XY UK. Tel: 0181-781 1812 Fax: 0181-781 1999
Potomac APL SIG	John Martin	51 Monroe Street, Plaza East Two, Rockville MA 02850-2421 USA Tel: +1 (301) 762-9372 Fax: +1 (301) 762-9375 Email: jam@aol.com

QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB UK. Tel: 01276-855880 Fax: 01276-855301
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (212) 664-3078
RE Time Tracker Oy	Richard Eller	PO Box 363, FIN-00101 Helsinki, Finland. Tel: +358-0-400 2777
Rochester APL	Gary Dennis	Soliton Associates, 1100 University Avenue, Rochester, NY 14607 USA Email: gsd@lpsalab.tor.soliton.com
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14607-1828, USA. Tel: +1 (716) 454-4360. Fax: +1 (716) 454-5430
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30244, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks HP8 4QH UK.
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eecg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1062 AC Amsterdam, Netherlands Tel: +31 20 648 4475 Fax: +31 20 644 1206 Email:sales@soliton.com
SovAPL	Alexander Skomorokhov	PO Box 5081, Obninsk-5, Kaluga Region, Russia Email:askom@apl2.obninsk.su
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rswain@acm.org
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: +1 (817) 577-0165 Compuserve: 73700,2545 Email:syarus@unlcomp.net
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: sl@fl.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Ben Best	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2J5, Canada Email: benbest@io.org http://www.sigapl.mtlake.com/sigapl/welcome.html
Unlware	Eric Lescasse	Tour Neptune, Cedex 20, 92086 Paris la Defense 1, France. Tel: +33 (1) 47-78-78-00. Fax: +33 (1) 40-90-04-11
Wickliffe Computer Ltd	Nick Teffer	76 Victoria Rd, Whitehaven, Cumbria, CA28 6JD UK. Tel: 01946-692589
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL UK Tel: 01203-523369
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (203) 872-7805

WORLD WIDE WEB SITES

ORGANISATION	URL
APL2000	members.aol.com/APL2000
APL-385	www.demon.co.uk/apl385
Causeway	www.causeway.co.uk
Dyadic Systems Ltd	www.dyadic.com
IBM APL2	www.torolab.ibm.com/ap/apl/apl2.html
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch
MicroAPL Ltd	www.microapl.co.uk
SigAPL	www.acm.org/sigapl
Rex Swain	www.pconet.com/~rthswaln
Toronto SIG	www.sigapl.mtnlake.com/sigapl/welcome.html
Jim Weigang	www.chilton.com/~jimw

FTP SITES

ORGANISATION	DOMAIN NAME
IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplasdii

REVIEWS SECTION

J3 - J Grows Up!

by John D. Baker (bakerjd@limestone.kosone.com)
and Martin Neitzel (neitzel@gaertner.de)

The following was posted to the newsgroup comp.lang.apl:

As many readers of comp.lang.apl know a new version of J has been released. As a J3 beta-tester and longtime J'uggler (J user) I've had an opportunity to beat up on the new system. Here's a brief summary of what J3 has to offer.

Let's get straight to the main point. With version 3, J has come of age. If you have new projects, particularly on Windows systems, take a good look at J. Here is a sample of what you will find:

Better Documentation

Iverson Software has often been criticized for producing documentation that requires serious study. Fortunately, J3 continues this noble tradition.

J3 comes with three main documents (distributed as Windows *.hlp files and as printed manuals). They are: *The Dictionary*, *A User Manual* and *J Phrases*. The dictionary has changed very little from previous editions. It's still the deep classic we've all read and cherish. Changes and editions have been added and the formatting of the document has improved but the dictionary is still "*The Dictionary*." The User manual is very similar to J2's manual. New sections have been added and many changes have been made to reflect the facilities in the new system.

I found the J Phrase book the most interesting. For old, and I mean old, APL'ers that remember the FinnAPL Idiom booklet printed way back in the early 80's the J Phrase book will seem familiar. The phrase book is a collection of over 1,000 tacit J definitions. Many phrases are direct translations of FinnAPL idioms but many are pure J and can not be easily expressed in APL. This is a fabulous resource that any serious J programmer should not be without. As a plus the phrase source code is distributed with J3.

Performance Improvements

Prior to release 3, J often came up short on standard APL benchmarks. Scalar (atomic) function processing has been reworked and J is now as fast, and often faster, than most APL systems on scalar integer and floating point algorithms. The improvements are in the (3 to 5 times faster) range for scalar functions.

The speed boost runs through the entire system. For example 32-bit ODBC is faster in J3 than APL+Win. The power conjunction, (not available in APL systems), is also much faster. Performance is no longer a valid reason for not using J.

Language Enhancements

New "Level" words **L.** and **L:** have been added to J.

- **L.** reports the maximum nesting level of a noun. It's similar to APL2's depth primitive.
- **L:** the "level" conjunction applies its left argument at the levels specified by its right argument.

The **L:** conjunction makes it easy to do APL2-style pervasive scalar function processing in J. **L:**'s fine level control also makes it possible to do all sorts of "at depth" operations that are not easily coded in APL2.

n. **m.** **u.** **v.** argument designators have been added to make it easier to define explicit adverbs and conjunctions in J.

The grade primitives **/:** **\:** have been extended to all arrays. This is a very welcome extension. Don't groan, but J now makes the grade!

OLE automation

The professional editions of J support OLE server automation. You can run a full version of J as an EXE server or you can run a stripped down version as an in process DLL server. It's now possible, and indeed easy, to farm out array calculations to J from any Windows application that can serve as an OLE automation controller. Right now the list of OLE controllers is short: Excel 7, VB 4.0, MS C/C++, but this is such a significant technology that within a year every serious Windows tool will support OLE automation.

Personally, OLE automation support in J3 has changed the way I program and how I attack problems. It's now easy to doll up a user interface with the dumb

blondes of software "Powerless Blunder" and "Visionless Bs" yet do the real data processing with J under the hood and out of sight.

Improved Debugging Facilities

A new set of 13! : n debugging verbs have been added. It's now possible to examine the state of the execution stack under J program control and to set "latent" debugging sentences that execute upon debugger breaks.

The new debugging facilities make it easy to write all sorts of "tactic" tools, like code profilers, space monitors, nice tracers, et cetera.

J3 debugging is still command-line and programmer-oriented which frankly is a blessing.

32-bit ODBC

The data driver 14! : n verbs have been reworked in the professional 95/NT version to provide 32-bit ODBC. A new column-oriented fetch primitive has also been added.

When using "good" 32-bit ODBC database drivers the new ODBC sub-system is much faster, and able to handle much larger data objects than 16-bit ODBC. 16-bit ODBC limited J to BLOBS of 64K. Now it's up to a respectable 2 Gigs.

New Window Driver

The J3 window driver 11! : 0 has been completely reworked to support new 95/NT controls and to provide better event processing. The new driver is not compatible with J2 and scripts using the old driver will have to be changed. One change that I particularly like is treating all Window driver errors as J domain errors.

The form editor has also been improved to take advantage of new controls and to exploit J locales to improve the modularity of J GUI code.

Improved utility scripts

The standard scripts distributed with J have been revised to take advantage of improvements in the language and window driver.

I have used J for years to solve problems. It's always been a good ad-hoc problem solving tool. J3 is now a good system-component tool. I'm no longer going to be shy about including J as a part of client systems. In fact I plan to focus a large

part of my programming efforts in the next year on exploiting J's powers via OLE/DLL in 32-bit Delphi and VB environments.

A Review of J3

by Martin Neitzel (neitzel@gaertner.de)

My previous Release of J was 2.01. I bought it mostly because I was just curious how the development system with its GUI support would look like. I had mixed feelings about the GUI engine I found there, the so-called "window driver". I liked the level of abstraction and simplicity and in particular the approach to access the window driver via a clean, text-based protocol instead of an API. However, I never liked to do the tedious house-keeping necessary for the proper dispatching of events, not in any other window system and not in J, either.

My interest in the GUI facilities only became revived when I saw a short demonstration of the interface builder which was part of the Professional Edition. It made it not only easy to lay out the forms, it also took care of the association of events and the corresponding J code. This was exactly what I had missed. But since I am usually not running Microsoft Windows — I only bought it for running J- I did not bother to acquire the Professional Edition back then.

I was therefore very pleased to learn that this form editor would also be distributed with the Release 3 Freeware version. I initially got that to have a closer look at the form editor, but in the end I was pleased by a lot more things than just that. This review relates to the Professional Edition of Release 3.

GUI development

After installing J (it's a breeze) I went directly to chapter 7 in the *User Manual* which introduces the form editor. This chapter is a very well written tutorial which guides you through the development of a small application from beginning to end. It really pays to sit in front of the computer and to follow it step by step. I got so carried away that it was well past midnight when I switched off the computer.

It is not just that you now have an interactive tool that helps to lay out your forms and takes care of all the coordinates involved. Two other things that are not so prominently visible at first sight but nonetheless just as significant have been changed, too.

The system comes now with a default event handler that does all the hard work for you. The association between events and J code under that scheme is not any more through case tables but implicitly by name. If your form `f` has a button called `foo`, a verb called `f_foo_button` will automatically be found and invoked as a handler for it. (In fact, there is some search scheme to locate a handler.)

The other new thing is that all controls are now regarded as maintainers for some value. The value of an edit control is the text, the value of a checkbox is a boolean, and so on. The new `set` command is used to change those values under program control. Reading the values is even easier. The default `wdhandler` will atomically extract all information from the event structure and assign them to ordinary J variables. At the same time, many `wd` commands have been simplified to return the result directly. The overall effect is that there is no need any more to use `wddata`.

Good use of locales is made to prevent a cluttered namespace. It helps here that J locales are very simply structured. You don't have any trouble to access the data from several controls at the same time. As a UNIX hacker I am just familiar with the X Window System and its various offsprings. `Tk/Tcl` there and J's `wd` are now very comparable with respect to coding style and ease of use. `Tk` has a slightly more sophisticated widget set, but J is clearly more mature than `Tcl` as a frame language and better suited to manage an entire application.

In terms of features, the current developments of Microsoft Windows seem to be closely tracked. J's window driver has new controls inherited from the Windows95 and NT environment, in case you happen to run that. As long as portability is not an issue for you, you can now make use of some new native Windows95 controls. Progress, status and trackbars, and spin buttons are the simpler new features, but tabbed panes, the RTF text control, and toolbars can have a big influence on the look and feel of your application.

While this strong support is all good news for Windows users, there is however still no port of the window driver for the X11 system available yet. I am still holding my breath for it and am hoping that more and more dependency on MS-Windows goodies will not prohibit that goal sometime. It would be very nice to write portable applications and GUIs in J.

Documentation

The two books *Introduction and Dictionary* and *User Manual* are by and large the same as in Release 2. Of course, they have been extended to cover the new

facilities and the language and the system. As before, beginners will appreciate the usage examples that accompany the terse *Dictionary* entries since Release 2.

The documentation kit now also contains the new *Phrases* book. Don't be fooled by the tabular layout that is mostly used. *Phrases* is much more than just a large collection of idioms ordered by topic.

It also serves as an introduction to several programming pragmatics related to tacit programming which is used throughout the book. I personally find that many typical programmer's questions are addressed here which I felt uncovered in the earlier *Programming in J*. Introductory notes and many simple examples guide you in a very gentle pace through the flexibility and facets of a basic concept before it is used in combinations with other elements. For example, more than two pages are dedicated to simple uses of `v&n` which prepare you to recognize such mini-groups quickly in larger contexts. Another page demonstrates just the various indexing capabilities of `{` and corresponding APL expressions using the bracket notation.

About half of the book consists of side-by-side listings of J phrases and their descriptions, e.g. `0 1&#:` and "whole and fractional parts of y".

Language changes

As always, it is advisable to have a look at the `status.doc` file and read the related entries in the *Dictionary* anew.

There are neither radical nor incompatible changes in the language this time. It appears that most of the work done was concentrated on performance enhancements, the smoothing out of some finer nooks and crannies and on new facilities for J as a development system.

The language itself has been quite stable and the only modifications you might have to apply to your existing code would be changes related to the window driver which are well worth such a inconvenience. Since the new default handler is just a utility routine in J and the older hand-coded `wait;` loops can still be made, you should be able to continue with the old scheme and just a few changes here and there. But all in all, you should benefit from converting to the new scheme of associating events and code.

Dyalog APL/W 7.2 for Windows 3.1, OS/2 and APL/W 8.0 for Windows 95 and NT

reviewed by Bob Byers (73043.1745@compuserve.com)

(This review was first published in Gimme Arrays)

This article reviews Dyalog APL/W version eight for Windows 95 and APL/W version 7.2 for DOS/Windows 3.1 and WIN-OS/2 from Dyadic Systems. Version 8.0 is a new product developed for the Windows 95 and Windows NT platforms. It is a full-fledged, 32-bit, Win32 compliant application. Concurrent with the release of version eight, Dyadic released an upgrade for DOS/Windows 3.1 or WIN-OS/2 users, version 7.2. Both products are close in terms of features, and workspaces are interchangeable between them. Version 8.0 offers additional features that are available only on the Windows 95 and NT operating systems.

Dyadic first announced plans of version 8.0 last fall just after the release of Windows 95. At that time, Dyadic was offering a pre-release preview program that could be subscribed to. As a self-confessed software "junkie" and APL/W user, since version 6.3, I took advantage of the offer and received a stream of updates over that past eight months. Each update I received contained something new and it was enjoyable to see the various stages of development take shape. Dyadic began shipping final releases of both versions in the second week of May.

I tested version eight on a 486-33Mhz system running Windows 95, and version 7.2 on a Compaq Pentium-75Mhz and on a PS/2 386-25Mhz, systems both running OS/2 Warp. The 486 and PS/2 have 16MB of RAM and the Pentium has 24MB of RAM.

Last October, members of the Toronto APL SIG were given an early preview of version 8.0 by Peter Donnelly, Managing Director of Dyadic Systems.

The original shipping date was scheduled for late 1995. However, due to unexpected delays the release date was pushed forward a few times until it finally shipped in May. As we all know, delays in the software industry are common. Remember, Microsoft was more than 2 years late with "Chicago" — Windows 95. Delays can be annoying but they are better than shipping buggy code.

This is another ambitious upgrade of APL/W. A quick review of history shows just how far APL/W has evolved over the last four years. Dyadic was the first

APL vendor to sell a true Windows APL development environment, version 6.2, back in 1992. In addition to offering an excellent APL interpreter, version 6.2 came with high level Windows GUI building tools that allowed the APL developer to do event-driven Windows programming. A runtime was included so APL/W applications could be distributed royalty free. In 1993, Dyadic released version 6.3 which directly supported 31 Windows objects, 77 properties and 31 events. In 1994, Dyadic released version 7.0 that came with a new user interface, customizable sessions by using the session object `SE`, support for additional Windows objects, closer APL2 compatibility, ODBC support and introduced namespaces to APL. In 1995, version 7.1 was released which further refined, added new features and stabilized this leading edge product.

Now, version 7.2 builds on this solid 7.1 foundation. This release supports 43 Windows objects, 152 properties and 85 events. Version 8.0 has all the 7.2 features plus Windows 95 enhancements, thereby boosting the number of Windows objects to 54, 186 properties and 129 events. Dyadic has now reclassified some events to be called "methods" which is consistent with other programming environments like Visual Basic and Delphi. The distinction between events and methods is that methods can only be invoked via program control and not by the user. APL/W can also be extended and use outside, powerful non-APL programs through the use of auxiliary processors (APs), by name association (i.e. `NA`) to access any compiled function in a Windows DLL file, by defining a VBX class in 7.2 and an OCX/ActiveX control class in 8.0. DDE support has been included since version 6.2.

Over the past four years APL/W leaped ahead of its closest competitor APL*PLUS in terms of innovation and features. That leap occurred with the debut of version 6.2 as a true Windows application. This leadership has continued with each subsequent new release. In contrast, back in 1992, APL*PLUS introduced a cumbersome Windows interface in versions 4.5, 5.0 and 5.1. Furthermore, the Windows extensions did not work under OS/2. In 1994, APL*PLUS did introduce a native Windows APL that also works under OS/2, but it does not support the number of Window objects or have the features that APL/W offers. Manugistics was also trying to sell the APL*PLUS product and was withdrawing development resources and sales support. A new release of APL*PLUS is due out this year and it is good to see that the new owners, APL2000, are breathing new life into APL*PLUS.

A revamped APL*PLUS will lead to better competition in the marketplace which can be viewed as a positive development. To point out the positive influence of competition let's examine the very visible OS/2 and Windows battle for the desktop. If IBM had listened to industry pundits and given up on developing

OS/2 our choices for operating systems would probably be between only DOS and Windows NT. OS/2 is a technical success story that runs DOS and Windows programs extremely well. Microsoft responded to the OS/2 competitive threat by developing Windows 95 supported by an aggressive and ingenious marketing campaign. Today, we have four operating systems to choose from for the Intel platform, Microsoft or IBM DOS, IBM OS/2, Microsoft Windows 95 and Windows NT. OS/2 is due for a new release by the fall (code name *Merlin*) and Windows 95 and NT will be updated this year as well. Microsoft has overall market share but OS/2 has made its mark. The point here is that, because of competition, we as users have choice and benefit from fast-paced improvements and lower prices. Hopefully, the same analogy will be the case between Dyadic and APL2000. It is important, however, that both companies along with IBM can grow the APL market and not just gain greater market share from each other.

Now back to my review. Requirements to run version 7.2 are DOS/Windows 3.1 or 3.11, or OS/2 2.1, or 3.0 "Warp" with at least 6MB of RAM. Version 8.0 requires Windows 95 or Windows NT. For Windows 95 a minimum of 8.0 MB of RAM is required. For Windows NT there is no minimum specified but, because Windows NT itself needs 12MB for the operating system alone, 18 or 20MB is my estimated minimum for an NT workstation. Version 7.2 runs very well under Windows 95 and should run under NT.

Why Two Versions? Why Continue Windows 3.1 Development?

As stated earlier, version 8.0 is intended for Windows 95 and Windows NT. It will not work in Windows 3.1 or OS/2. Version eight is intended to be the future platform for major enhancements to APL/W. Microsoft's vision of the world is that all PCs should be running Windows 95 or NT. To date, Windows 95 has been more successful in the home PC market than the corporate one. Windows NT, which is due for a new release shortly with the Windows 95 interface, shows potential for the future, but because of its heavy hardware requirements it is not a common operating system found on workstations. Today, most companies still rely on DOS/Windows 3.11 environments or OS/2 workstations which are many times more commonly installed than NT. Thus, until the business market moves in greater numbers to Windows 95 or NT it makes good business sense for Dyadic to continue to update version 7.x. Both versions share common C++ base code which makes it easier for Dyadic to update both versions at the same time.

Version 8.0.8 Features New Objects

Version 8.0 supports 11 new objects that are part of the Windows 95 environment. The names of these objects are *ImageList*, *ListView*, *OCXClass*,

ProgressBar, PropertyPage, PropertySheet, RichEdit, Spinner, TrackBar, TreeView and UpDown. Associated with these objects are a host of new properties and events. These new objects are commonly found in other Windows 95 applications and they can all be built into APL/W applications as well. Each of these objects can enhance the visual appeal of your applications. There is a separate tutor workspace, *WTUTOR95*, that shows how each object works.

In APL/W, GUI objects are namespaces, which means you can embed any callback functions and data inside each object if you wish. You can also create non-GUI namespaces. Namespaces were first introduced back in version 7.0 and are very important in APL/W. Namespaces can be viewed as mini-workspaces within your workspace. Namespaces can be copied between workspaces and can also be saved on component files offering the possibility of building APL class libraries. Details of namespaces have been the focus of previous articles and I will not cover namespaces in detail here.

OLE Controls (OCX support) and ActiveX

OCXs are 32-bit, third party add-in DLLs, software components developed for Visual Basic, Visual C++ and any other development platform that can support them. Now OCX support has been introduced in version eight. Version 7.x continues to support 16-bit VBXs, version 1.0 controls. Support for VBX has been improved since version 7.0 and most VBXs do work, but you might come across some that do not, and inadequate documentation can make them difficult to use. The situation for OCXs is much better. To date, Dyadic claims that all OCXs tested work in Version 8.0 and because each OCX control must contain embedded documentation in the control OCXs are easier to figure out how to use. VBXs are not supported in version eight. Almost all companies that produce controls are concentrating on OCX/ActiveX controls.

Full OLE 2.0 technology is still being developed. Improvements planned for later this year include being able to use version 8.0 as an OLE server.

A new variant on OLE is what Microsoft calls ActiveX controls. ActiveX controls extend OLE to include Internet hooks. APL/W should be able to use ActiveX controls. If Dyadic has to update APL/W for ActiveX support a free upgrade will be issued.

New Explorer and Find Objects Tools

The Dyalog APL/W Explorer is based closely on the Explorer tool in Windows 95. This is a cool tool, it is hard to live without it. It allows you view your APL

workspace in a visual way. There are two parts to the Explorer. On the left side, an outline of your workspace is presented in a visual format via the TreeView object. If you are using namespaces or GUI objects this is extremely helpful in viewing how the workspace has been structured. The right side of the Explorer shows icon or detailed level information of your functions, variables and operators in the namespace you want to focus on. You have your choice of large icon, small icon or detail view. The detail view, which I find the most useful, uses the listview object and includes columns that show information on the object name, type, header line, size of object, date of the last edited change and who did the last edited change to a function — finally you can tell who did that last change. By clicking on any object in the Explorer you can edit it or display even more information via property sheets. You can, as well, perform namespace management operations such as creating/deleting namespaces and copying APL objects to and from namespaces either with keystrokes, or with menus, or you can drag & drop with the mouse. Some operations are available by using the right mouse button on a selected object.

The Explorer can be left on during your session. If you load another workspace the Explorer updates itself with the new workspace objects. If you load two or more APL/W sessions you can invoke separate Explorers.

Another new tool, the Find object tool, can locate functions, variables and operators that are in your workspace but may be buried in deeply nested namespaces.

Long File Names

Windows 95 applications, like APL/W, support long file names. If you can type, you can include up to 256 characters in a workspace name, so you can get quite descriptive. If, however, you want to load your saved version eight workspaces under version 7.2, you will have to truncate the name back to eight characters.

Windows 95 Logo

Dyadic's intent for version 8.0 is to qualify for the official Microsoft Windows 95 logo and for any applications written in APL/W to, in turn, also be eligible to qualify. The process of qualifying is now underway. Dyadic's position is that version eight, as is, qualifies for the logo, but it is up to an independent third party to decide. If version 8.0 fails to qualify, Dyadic has stated they will make any necessary changes and issue a free update.

Common New Features Found in Both Version 7.2 and 8.0

Development Environment

Both versions have a similar look and feel. Dyadic introduced a new look back in 7.0 and have retained the same interface. Version eight has new buttons on the toolbar for the new Explorer and Find utilities. Both versions have introduced a pop-up menu which is invoked by the right mouse button when you click anywhere in the session.

Control Structures

Control structures were first introduced to APL by APL*PLUS (then Manugistics — now APL2000). At the time, APL*PLUS developers encouraged other vendors to adopt control structures. As a result, APL/W now supports control structures with the exact same syntax as APL*PLUS.

Control structures are not intended to replace good APL programming techniques that reduce the need to do looping but they do provide an accepted alternative, that other programming languages have had for a long time. If you are porting from APL*PLUS to APL/W and are using control structures that porting has just become easier to do.

Native File System

Like control structures, $\square N$ functions have been part of the APL*PLUS environment for many years. APL/W has now adopted these native functions thereby eliminating the need to emulate native file read and write functions with user-defined functions.

Enhancements to the Grid Object

Since version 7.0, APL/W has shipped with a native Grid object for displaying 2-dimensional arrays in a spreadsheet-like interface. Other APL and J vendors do not have a native grid and you have to purchase a separate VBX or OCX control to get one. In APL/W you get the grid built in and it is probably one of the objects you will use most. It is designed to work as a child object to a form, sub-form, property sheet and group objects and is fully documented in the manuals and in the on-line help.

Dyadic has enhanced the grid capabilities with each new release of APL/W. This latest enhancement allows you to develop even more sophisticated applications with the grid. There are so many new features that the *WTUTOR* workspace now dedicates five lessons to the grid object. Of note, multi-level column and row titles are now available, along with displaying bitmaps, Windows metafiles and other graphics in each cell or across the entire grid.

Formatting cells via `□FMT` specification has been added thus making it easier to format cells. The enhanced grid now supports 69 separate properties and 39 events and methods.

New GUI Resource Editor (*WDESIGN* Beta at Time of Writing)

Since version 7.0 shipped, Dyadic has promised to update their *WDESIGN* workspace. *WDESIGN* is a workspace that first shipped with 6.2. It is a visual resource editor for developing and maintaining GUIs and gives APL/W a Visual Basic look. It has not changed since version 6.3. A new *WDESIGN* was not ready back when 7.0 was released and therefore did not incorporate new objects such as MDI, the grid object, namespaces, tab buttons etc.

The final version is still not ready but the latest beta version of *WDESIGN* was shipped to show it off and to get feedback. It looks very promising. All the new features are now supported or soon will be. Also, under version eight OCX controls can be incorporated. Developing a GUI interface is much easier if you can just point and click the objects you want from a toolbar onto your form and update properties via a property list. *WDESIGN* will generate all the code necessary to recreate your GUI or, alternatively, save the GUI objects in your workspace. The final version of this workspace can't be shipped too soon.

An alternative resource editor, that ships as shareware with APL/W, is the *Causeway* workspace from Causeway Graphical Systems Ltd., written by Adrian Smith and Duncan Pearson. This is a new version of *Causeway* and it takes advantage of namespaces and the other new features. It is worth your while exploring *Causeway*. An updated on-line help file is provided.

Enhancements to the Tracer/Debugger

The tracer has been enhanced to allow you to insert multiple breakpoints in functions. Step by step tracing and continuous running after breakpoints are useful new features. The new tracer/debugger now sports a suite of new icon buttons with tooltips available to let you know what each action button does.

Distributed Dialog Workspaces & Shareware

APL/W ships with the usual Dyadic utility workspaces, a revised *WTUTOR* workspace, a new *WTUTOR95* workspace to help building GUI objects, demo workspaces including an OCX browser, a VBX or OCX example and the new *WDESIGN* beta workspace. In addition, Dyadic includes shareware versions of workspaces donated by other APL developers. Included under the subdirectory for "Outer Products" is *Causeway*, RAIN Graphics, WSFIND, COMPARE and others. I find each of these workspaces useful. RAIN, in particular, is a good

graphics package that now uses namespaces. *Causeway* was featured at a recent SIG meeting by Adrian Smith along with the new NEWLEAF (not distributed as shareware) reporting workspace. Another workspace deserving honourable mention is COMPARE, by Rex Swain, which contains functions that will compare and highlight changes in functions from different workspaces. Some of these utilities, like COMPARE, can be copied into `□SE` and are available to you from any workspace.

Other Enhancements

Other useful enhancements and new features are included and are too numerous to mention. Two new system variables of note are `□PATH` and `□EXPORT`. They are useful when referencing utility functions within a workspace but contained in protected namespaces. `□PATH` is similar to the PATH command found in DOS.

Documentation

Good documentation is critical, especially for products such as APL, because you are not going to find any books at your local computer bookstore with names like "Dyalog for Dummies" or "Learn APL/W in 21 Days". You have to rely on what the vendor provides.

The documentation that ships with APL/W is the same for versions 7.2 and eight. The manuals are similar in style to those introduced with version 7.0. New material has been added and some re-structuring has been done. The number of manuals has been reduced from four to three but, overall, about 200 new pages have been added and there are about 1,400 pages in total. Features specific to version eight are well highlighted. Overall, the manuals are good, are well organized and include plenty of small examples.

What is lacking is a "how to" applications manual. By this, I mean, perhaps a separate manual showing complete and practical programming examples on application development. This manual would of course show off the benefits of using APL and Windows programming and the effective use of namespaces. Also detailed examples on each of the most popular OCXs and VBXs, with example code, and how to adapt any sample VB code to APL/W would be a great help.

Three Windows help files are also distributed. They are separated into "Latest Enhancements", "Language Help" and "GUI help". These are all well done and are an invaluable source of on-line help.

What APL/W Still Needs

Like all software, there is always room for improvement. In addition to the documentation suggestions, here are some suggestions for things I would like to see. Relatively speaking, these are minor. I am sure all current users of APL/W have their own suggestions.

First, it is easy to clutter up your screen with too many edit windows. This makes it awkward to get to the APL session and you either have to move those edit windows around or minimize them individually. There should be a way to get to the APL session more easily. Other Windows applications have a Window menu that allows to you do standard window management operations (i.e. cascade, tile or minimize open windows). APL/W should have a similar menu or else some other method to get the APL session "on top".

I would like to see the resource editor workspace *WDESIGN* more easily accessible. Perhaps the workspace could become part of the *USE* session object and available via the click of a button from the toolbar rather than loading or copying this workspace.

Importing workspaces from APL2 (mainframe) or APL*PLUS including component files can be done but exporting APL/W to other environments is not included. I would like to see APL2's *IN* and *OUT* transfer file facilities adopted. Transfers to and from APL2/PC and APL2/2 should also be possible. Under "Outer Products", Jim Weigang's shareware APLASCII transliteration workspace is included. This workspace can import and export workspaces to and from other APLs by using ASCII translation. This is a workable interim solution but should not be a replacement for formal importing and exporting tools.

The grid object, as is, is excellent for working with tables. I think an enhancement to the grid would be to display 3-dimensional arrays automatically by specifying the grid values property.

The new versions are quite stable but, given all the changes, some bugs do exist. By the time you read this a bug fix will probably be in the mail. Dyadic is quite responsive to responding to any problems. In the past, they have provided free updates that fix reported problems and sometimes add new features. In order to ease the distribution of updates Dyadic needs to extend its new Internet Web site and include an ftp facility. Improving the Web site is in the works.

Conclusion

Reviewing and working with a product like APL/W gives you faith in the future of APL. There is a lot of functionality for the APL/W user at any level. It also narrows the gap that exists between APL and what other development languages have. Even if you are not into developing GUIs there is still reason to consider APL/W. If you are developing GUIs APL/W supports the most Windows objects in the marketplace. Also, namespaces and all the benefits associated with them are only available in APL/W.

Should you buy, upgrade or convert to APL/W? The answer is "it all depends." This review has attempted to identify most of the new features. In the end, you have to try it for yourself and decide. My experiences have been very positive. Where I work, we started with mainframe APL2, migrated to APL*PLUS for the PC and in 1993 migrated fully to APL/W and some APL2/2. Version eight is for you if you have Windows 95 or NT and most of the major future enhancements will be done here. If you want to develop for Windows 3.1 and Windows 95/NT platforms you will need both versions so you can package the correct runtime interpreter and associated files. If Windows 95 or NT is your sole direction then you can get by without version 7.2.

Everything is included in the one purchase price. There are no additional add-in products to buy from Dyadic, such as ODBC support. There are, of course, add-in products from independent companies. Specific prices are something you should ask Dyadic or your local distributor.

Need more Information?

Dyadic Systems' Internet Web page URL address is:
<http://www.dyadic.com>

Internet e-mail address is:
sales@dyadic.com

Fractals, Visualization, and J

*A hands-on course on mathematical visualization by Clifford A. Reiter
Iverson Software Inc., Toronto, 1995. viii+182 pp., 18 colour plates.*

reviewed by Nicholas J. Cox (n.j.cox@durham.ac.uk)

Cliff Reiter's book breaks new ground. Hitherto, all books on J have been written by Kenneth Iverson or others (Roger Hui or Chris Burke) working at Iverson Software Inc. If J is to take off in a big way, and many of us hope very much that it will, all sorts of enthusiasts, and especially experienced teachers, must write J books at a variety of levels. Moreover, this is the first book with a focus on J as a way of doing graphics, thus providing a welcome extension of the range of applications for the language that have been publicised in detail.

Target Audience

It is a little difficult to say for whom the book is written, except in the fatuous sense that it is for anyone interested. The reader is not expected to know J (or indeed APL). Indeed, the examples might well get a beginner in J suitably excited, because we move fairly briskly to things that are fun to do, while also immediately getting a strong sense of the power and elegance of J. The reader is plotting polygons by p.5 and drawing a Sierpinski triangle on p.9. There are also several drill exercises for the learner, although they tend to be mixed up with much more substantial projects without overt discrimination. My guess, however, is that most readers would be better off cutting their teeth on other material introducing J.

Similarly, it is said that readers who have studied matrix algebra ought to find the mathematical level comfortable. That is about right with the proviso, presumably implicit, that they should have studied several other parts of mathematics to roughly the same level, say at least to a beginning university level. Any readers who have not yet come across fractals will also find some ideas introduced as if they were already broadly familiar.

Obtaining the Software

Access to J for Windows is required. Other files and programs produced by Reiter or available in the public domain are also essential for practical work. (They are now bundled with J Release 3.) A freeware version of the J interpreter

or the other material may be obtained by anonymous ftp from the archive at the University of Waterloo.

Cliff Reiter has stuffed this book full of neat and clever things that you can do. They range from reproducing standard introductory examples to producing original pieces of mathematical research. To do it full justice would require many sessions on the computer, and perhaps also several bursts of reading to make up the reader's mathematical deficiencies. I found that the story jumped around in a rather unpredictable manner. Indeed, most books touching on fractals seem consciously or unconsciously to mimic the nonlinear form of their subject. Major themes such as fractals, cellular automata, image processing and ray tracing tend to occur as attractors experiencing multiple visits during the reader's trajectory. (Inevitably?)

It is difficult to imagine this book being used as a course text, unless the teacher were Cliff Reiter himself, or were to have exceptional verve and enthusiasm in projecting the material and helping the students at tricky points. Rather, this is primarily a book for private study. Here are some examples from among the many intriguing details. Gerunds are used to implement random transformations. Thus suppose one of the transformations t_0 , t_1 or t_2 is to be chosen with equal probability. The verb `rt = . t0't1't2@.(?@3:)` chooses randomly from 0, 1 and 2 and then correspondingly from the three transformations. Most of the examples of gerunds I have seen previously struck me as somewhat contrived ('OK, but I can't imagine actually wanting to do that'), so I like this example in which the construct appears naturally and neatly.

Sample Fractals

More substantial is the example of the Lorenz attractor ('strange' according to some), now a staple of chaotic dynamics, that arises from three coupled differential equations, originally a caricature of atmospheric convection. We get results with a dozen lines of code. This should be a good way to impress an applied mathematician or numerate scientist that J can be used to move quickly on serious projects. In my persona as a geomorphologist, I must comment on the fractal simulation of topography, familiar to most of us from movies or posters if not from Mandelbrot's mathematics.

Reiter follows previous workers in drowning the lower levels on the resultant image. A good reason for such a graphical flood is that it looks more attractive to most of us, because of the primal or aesthetic appeal of water masses. A bad reason less often remarked is that it hides the least accurate characteristic of fractal-simulated topography, namely that the number of pits (local minima) is

expected to be the same as the number of peaks (local maxima). Very few natural landscapes satisfy this equality: for a variety of physical reasons, not only are lowlands much smoother than highlands, but they also possess very few pits.

J reference sections in the Appendices are useful in their own right. In particular, there is a very useful summary of function composition that nicely complements Kenneth Iverson's expositions. Commenting on a more pedantic level, I note a horror of hyphens (as in 'the long term behavior of a real valued quadratic map'), and a generous sprinkling of typos: Reiter goes back and forth between 'Mandelbrot' (right) and 'Mandlebrot' (wrong) and he several times treats 'automata' as a singular noun. I did not spot any serious errors, although I did not always manage to get things working properly on my computer. It is a pity that the colour illustrations are all gathered at the end, presumably to keep the price down. Anyone with a strong interest in J, actual or potential, should get hold of this book and study it.

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

RECENT MEETINGS

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

Joint Meeting of IBM GSE and APL Club Germany Frankfurt 23rd - 24th April 1996

Notes by Adrian Smith

As always, this was a well-organised and well-attended meeting (around 40 people were at most of the sessions). It was primarily an APL2 group, and it is also worth noting that Germany is a real stronghold for OS/2 so there is genuine interest in the progress of APL2 for OS/2.

I have detailed notes on two of the sessions: Timo Laurmaa demonstrated a number of TCP possibilities with APL and Netscape, and Dave Liebttag (IBM) showed a fully working APL2 interpreter under Windows 95.

Timo Laurmaa on TCP/IP and APL2

Timo began by showing how Netscape can be used as a front-end to an APL mainframe system. He made an early entry for the 1996 bad pun contest by demonstrating that the HTML construct `<lieb> some text </lieb>` has no effect — Netscape does not understand *Liebtags* (groan). He was very much focussed on the idea of an 'IntraNet' — an internal company network where there are no security problems and everyone has a high-bandwidth connection. Also corporate users are very happy to register for mass copies of Netscape which they see as virtually free. It gives them the possibility of standardising the user interface across a huge variety of applications.

He continued with some background on TCP/IP which he compared to shared variables except that anyone can talk to anyone once they understand the low level calls. Once you have established a connection to a TCP/IP node address, you find the HTTP 'port' is conventionally numbered '80' and the combination of address and port forms a 'socket' which the server opens and listens to. If it is an APL application (using AP119) it looks for a stream of characters like `'|GET|~~~~~|HTTP1.0|return'` which is how Netscape begins a request. APL can now parse the remainder of the request which is essentially the string you see in the Netscape top line when you submit a query to a server such as Alta Vista. It can do whatever processing is required, format the results and write them to the socket wrapped in standard HTML.

Note that the server will close the socket after every request, which allows thousands of 'simultaneous' users, as sockets are occupied for a very short time — very different from an FTP connection where a user keeps the link active all the time. This 'sign-in-sign-out' model makes it impossible to ask for passwords as the server has no means of knowing that a following transaction has anything to do with a previous sign-up!

Timo ended with a demonstration of Netscape 'evaluating' APL expressions — simply type `HTTP://.../aplserver/2 3p16` into the top window (of course the APL characters present some problems) and see the result displayed in the browser window below. He handed out a couple of pages of quite simple APL code which implemented this facility on an APL2 mainframe.

Dave Liebttag on APL2 Future Directions

Dave began with a summary of the improvements made at service-level 7 of the OS/2 product. Some of the highlights were:

- some new `□` functions such as `□WSID`, and also you can now fix functions with 'illegal' `□fns` in the header line — a major migration aid!
- you get some warnings when closing, for example, the session manager.
- the editor now understands APL tokens.
- cross-system `□SYO` is much easier to set up as you can now use the domain names and AP119 will translate these to addresses for you.
- some new GUI objects such as timers and DDE support.

In the afternoon, Dave approached the podium again, with a considerable show of reluctance "Well, I hate to say it ... this is my machine and it's running Microsoft!"

- *I think this is what you want — let's see what happens if I click on it!*

Dave gave us a quick tour of APL2 running under Windows 95, essentially he was able to show us a complete interpreter running in a very reasonable session window, with a function editor also fully operational. He then paused to ask the audience a series of questions, designed to find out how much more is needed to make this a marketable product, and whether there is sufficient market to justify the work.

1. *I need to know about the Gui facilities. Providing AP124 all these years has been fairly expensive. Gui applications are not going to fly in AP124 anyway!*

2. AP207 — the general graphics processor. This works now, but comes up as a separate window. We could do more — events and so on — but do you really need AP207 the old way?
3. Co-operative processing with $\square$ SVO. The vast majority of customers are not using it — do we need to do it?
4. What about AP127? DB2 is not there any more, so what do you want. Do we connect specifically to Oracle or do you need 'everything' via ODBC?
5. Communications inside the machine: DDE, OpenDoc, OLE. What do you need?
6. Language questions. Do you want true, blue APL2 or a system that would be easy to migrate to? Do we need namespaces, control structures, commute, rank, set-union and so on? What do you want in the interpreter?

So far the project has taken 7 weeks from ground-zero to what you see. Do you need a Gui interface at all? We can already call the interpreter as a DLL, so why not use another tool for the interface — I will never be able to catch up on Visual Basic and your finished application will never look 'quite right!'

Currently this is headed for Windows 95 and Windows NT. If you only care about one of the above, let me know. Windows 3.x is not an alternative.

Dave got some quite clear answers, but of course this was a very stratified and self-selected survey! Yes, they want AP124 (people want to do off-line mainframe development) but no one needs AP207. Database access should be ODBC, but a reasonable minority wanted high-throughput Oracle access as well. True-blue APL2 is fine. Unfortunately for Dave, there was a solid vote for carrying on chasing after VB — virtually everyone wants to do their Gui programming in APL and no-one was willing to risk the alternative of programming in VB, Delphi or C++ and just calling APL to do the hard stuff. As you would expect, both 95 and NT are requirements.

I really hope this happens, but I have seen APL2 interpreters under Windows before (at least 5 years ago) and IBM politics always killed them at birth. If you need to downsize a mainframe system, what more APL2-compatible interpreter is there? Why should you have to struggle with all the gotchas listed by Rex Swain in the last Vector? Just)out, head for your Windows NT box and)in. There has to be a market out there, and it obviously isn't on OS/2. It would be a terrible pity if APL2 died just as APL+Win and Dyalog finally approach full compatibility with it!

Eric Iverson at the BAA 19th April 1996 (at the RSS)

notes by Adrian Smith

Introduction

This was a special meeting, arranged at short notice to take advantage of Eric Iverson's presence in the UK. It was one of the best attended meetings for years, with a really good mix of APL old-timers and new faces. Eric talked a little about the background to J, then spent the bulk of the talk demonstrating the power of calling J as an OLE2 server. Finally he showed some of the new graphics facilities and demonstrated the power of the language in calculating some interesting statistics on a ridiculous number of primes, and then plotting the resulting chart with upwards of a million data points.

J — Background and Motivation

J is produced by Iverson Software Limited, which was formed as a software development company in 1988. In 1989 Ken Iverson and Roger Hui had been working on a first implementation of J, and they decided to join forces with ISI and to make J the primary ISI product — it was first shown at APL90 in Copenhagen. About a year ago, Kirk Iverson joined the team, so there are now 4 people focussed on the design and development of the language.

All marketing and sales support is handled by Strand Software, which leaves ISI free to concentrate totally on the interpreter.

Eric drew a very interesting timeline on the board, which helped to explain the original motivation for J, and also to put it in the context of the general development of array-processing languages.

1964 The APL notepad epoch. APL came into being as the Iverson Notation at Harvard.

1966 The first machine-implementations of APL by IBM. There was a rapid evolution of ideas, carried on by STSC and IP Sharp until around ...

1984 No-one was funding language changes any more. Sharp added box and rank, but the language was no longer moving forward.

- 1984 The J notepad epoch. Ken took the APL design philosophy and began sketching ideas for an 'ideal world' language ignoring all that had accumulated over the previous 20 years.
- 1989 The first machine implementation led to the same dynamic explosion of ideas as had occurred in APL. Very early in the cycle, J acquired some very fanatical users.
- 1993 J became a serious commercial product as the language definition stabilised.
- 1996 J3 is now a *really* serious commercial product!

So — how do you go about getting the best productivity from a high-level language like J? The crucial thing is the language itself, not the interfaces which are driven by fad and fashion and changes in the hardware.

J and APL would have been useful to the Babylonians, are useful now, and will still be useful in 1000 years time. Array problems don't change a great deal, but the interface is 'flavour of the month' and has been changing radically every 5 years and it's getting faster. Look at the timeline — there has been a huge change in the computing environment over this period, and most of it has been totally unpredictable!

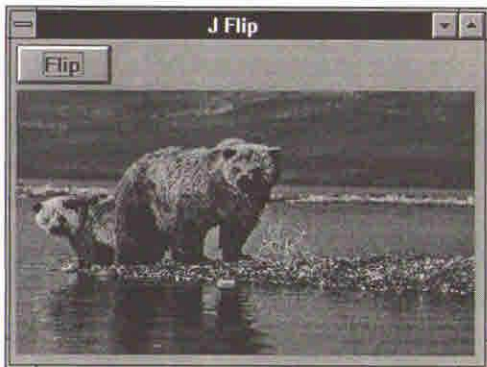
"ISI are strictly J language people — we are concerned with data-processing problems — we are J folk, not Gui folk.

To make a living, you have to sell something perceived as useful by other people! There are plenty of problems with no Gui — problems in Physics or Astronomy or weather-forecasting. These are good targets for APL or J. Then there are problems which are nearly all Gui — games and most spreadsheets have minimal DP, maximum user-interface — use the best tools available! Probably this is whatever tool the corporate IT department has given its stamp of approval to!

The interesting challenge is where you have high DP and high Gui content! In the time-sharing days APL had both the best language **and** the best user-interface. For many years in the PC world everybody else was miles ahead on the user-interface; Dyalog and APL2000 have done a phenomenally good job of nearly catching up! They don't dictate where the Gui is going, they are doomed to failure — they are always going to be behind!

So what about ISI? Given that we gave up the Gui chase, how do we remain viable? All that architectural framework that comes with the operating system has finally made it possible to realise the dream of 10 years ago — we can prototype in pure J, build the Gui in the tool of choice and ship the result as a single seamless package."

Eric continued with a sequence of 3 very impressive demonstrations. He produced 3 very similar-looking applications which simply had a main form, with a button on it to reflect the classic 'bears' picture left-to-right.



The first was in pure J, using the J window-driver to build the Gui. Eric used an existing script to do the 'data-processing' but built the Gui on the fly and showed how to save it as a runtime application which could be started from a Windows95 shortcut. This took around 15 minutes from start to finished production application.

"Now let's hypothesise that we need a much heavier duty Gui component, or perhaps your organisation says 'you shall develop in VB'. You ask 'Can I use some OCX controls and OLE objects in my VB application?' Naturally, they say yes, so here goes ..."



Eric proceeded to build the exact same system, using a Visual Basic interpreter installed off the CD that morning. The Gui design phase was similar, and he needed to add a line of code to declare the J EXE server. The *onClick* handler for the 'Flip' button called the J script and forced a repaint of the bears picture.

```

Public js As New jexeserver

Private Sub Command1_Click()
js.Do ("flip 'c:\b.bmp'")
picture1.Picture = LoadPicture("c:\b.bmp")
End Sub

Private Sub Form_Load()
js.Show (1)
js.Log (1)
js.Do ("0!;0<'c:\flipbmp.js'")
End Sub

```

Again this took less than 20 minutes to set up and involved no weird stuff at all in the VB code. To make it ready for distribution, Eric switched to the J DLL server so that on calling J, VB just loaded the J engine (no session) as an in-process DLL. He noted that it is as efficient to do 'JDO("expression")' as any other DLL call. It is simply a call-return.

Eric concluded this part of the talk by taking on the same problem in Borland Delphi and arriving at a visually identical solution in less than 10 minutes (well, we were running late for the tea break!).

He now had 3 icons lined up in a row, each of which could load and flip a 256-colour picture. In each case, the same J script was doing the data-processing (non-trivial, by the way) but the interface was built in a quite different way. It would have been fun to try it in Dyalog 8 too (J is perfectly usable as an OLE server from APL) but we really had run out of time.

```

// Delphi flip code

uses oleauto

var js:variant;

// formcreate
js:=createoleobject('jexeserver')
js.show(1);
js.log(1);
js.do('0!;0<'c:\flipbmp.js'');

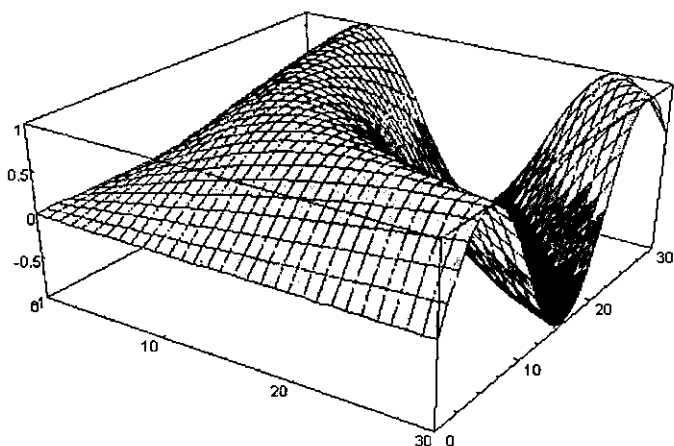
// buttonclick
js.do('flip 'c:\b.bmp'');
image1.picture.loadfromfile('c:\b.bmp');

```

A very rough byte-count showed Delphi well ahead at 600K for the entire system, native J second at 920K, and VB a long way last at 2.5M. However Eric pointed out that the J and VB numbers would change very little as the system became more complex, whereas Delphi grows quite quickly to around 1.5M where it too seems to level out.

Graphics in J

After the break, Eric gave us a quick demonstration of the new graphics capabilities in J. He very generously acknowledged the debt to 'Rain' which Chris Burke had used as a model, and to some extent as a target in terms of quality and flexibility.



The charts were very impressive, in particular the plotting speed for large volumes of data. There is also the option of some of the 'shaded contour' style plots beloved of mathematicians and advertising copywriters!

Questions and Answers

The meeting ended with a lively discussion, with most interest on the future plans for the J system. Eric stressed the importance of the Dec Alpha and other Unix platforms, and also introduced the possibility of adding 'infinite precision' arithmetic to the language at an early date.

Coverage in Byte Magazine

Eric strongly recommended any J novices to read the article "*The Joy of J*" in the August 95 issue of Byte magazine. This is an extremely positive summary of the language by Dick Pountain, who has been a Byte staff writer for many years. It was a particular pleasure to meet Dick for an hour or so after the meeting had ended (unfortunately he was unable to attend due to other commitments) and to hear his views on the likely future of the OLE technology and the 'array server' approach adopted by ISI.

BAA Vendor Forum

reviewed by Jon Sandles

The vendor forum was made up of four presentations – by MicroAPL, Causeway, Dyadic and The Bloomsbury Software Company. The venue was the Royal Statistical Society which proved to be quite adequate – although the hike downstairs to get tea and back up again probably took ten minutes longer than expected and caused us to overrun somewhat. Fortunately we did not get thrown out and the last speakers had adequate time (in the end!).

MicroAPL – David Eastwood

David began by telling us that most of the last year had been spent on non-APL projects and hence there had not been much time for APL product development. To put this into context he gave us an overview of the services and products MicroAPL are now providing. He also apologised for the lack of any 'live' presentation material – all the MicroAPL portable kit had been taken to a show they were currently doing in the USA!

- **APL Products.** This is APL68000 Level II for the Apple Mac (PowerMac and 680x0 Mac) and IBMRS6000 (it has also been available in the past for other now defunct computers, e.g. Amiga). It is based on APL68000, is APL2 compatible, and uses an APL*PLUS III compatible $\square W I$ object definition interface. It has a packager to glue the finished application together (and hide the fact that it's written in APL!) and has APL*PLUS compatible native file handling using the $\square N$ family of functions. David is aware of 7–8 strong commercially available packages that have been written in the language including Biological and Astronomical programs. Its use is concentrated in the States, France and Italy. Although there has been little time for new developments the next release will be incorporating Apple's new Copeland Operating System.
- **Translation and Emulation Services.** This is where a lot of MicroAPL's resources have been going. It is also interesting in that all the work evolved from code that they wrote for their own use internally! In 1990 MicroAPL had the best selling APL implementation on the Mac – this was written entirely in 680x0 assembler – over 65,000 lines of seemingly unportable code. This became an issue when the 680x0 chip was effectively made redundant with the emergence of RISC IBM chips in the Mac. Thus, MicroAPL wrote their own translator that takes 680x0 assembler and converts it to RISC assembler. This means they only need one copy of the source code which has obvious advantages – same bug fixes, simplified support etc. Fortunately the rest of the world was wanting to do the same thing and hence three products evolved:

- **PortASM** – converts Mac to PowerMac – WordPerfect is their biggest customer
- **PortASM/68K** – Mac to PowerPC – Phillips is a big customer
- **PortASM/86** – Intel assembler to PowerPC – Motorola is a big customer

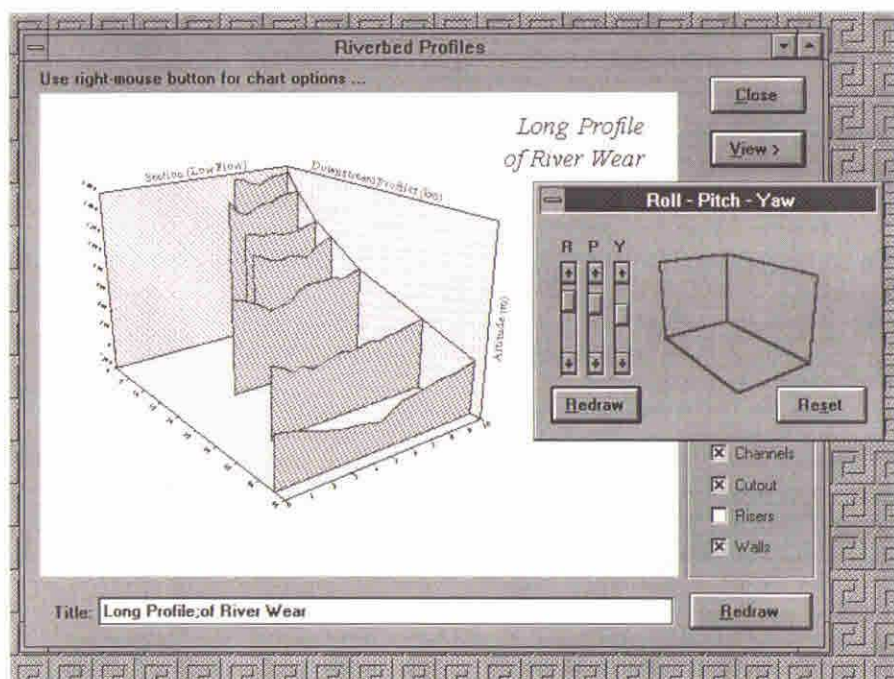
This experience in the assembler translation field also led MicroAPL to produce a number of similar 680x0 emulators which allow 680x0 software to run unchanged on a non-native platform like the PowerPC.

- **Consultancy.** Mac – C++. PC – Powerbuilder, C, C++ and Oracle. To demonstrate the kind of work they have done recently in this field David showed us slides of a system they have recently written for Reuters to access Reuters' news sources and pictures. This allows you to search for news by topic and then by content. Amazingly a search on APL had led to a hit on the publicity for the very event we were sitting in – presumably via the BCS publicity in Computing magazine. This project had taken 30% of their resources.

Causeway – Adrian Smith

Adrian introduced himself and Causeway. They are a two-man team of himself and Duncan Pearson. Much like MicroAPL the products that Causeway have made/are about to make available are things that they have found useful to themselves in their day-to-day consultancy work. The bulk of this work has been for SAP – who produce business management software for large companies.

Adrian started by putting up the following slide from the *Rain* graphics package that Causeway are now selling. For some time Adrian has stayed well clear of the modern vogue for fancy 3-d graphics which tend to distort rather than enhance the image you are trying to portray. But the above 3-d image is easily interpreted by the brain especially when the user can tilt its orientation using the cursor keys. Adrian didn't demo this live – apparently it's rather slow at the moment!



The majority of Adrian's talk was then focused on the *Newleaf* package – which is still in Beta test – but will be ready at APL96 and ported to APL+Win (not hard to do – just tedious). *Newleaf* makes it easy to do complex reports. You can have multiple columns – embedded graphs, embedded tables (these themselves can be highly complex – including hierarchical headers).

The underlying technology used is PostScript – and this itself has advantages – the output from *Newleaf* can easily be converted to PDF (via Acrobat Distiller) and hence viewed on the internet using Netscape 3 (via Adobe Acrobat). Although this is only one of many ways of viewing WYSIWYG documents on the net, because of its association with Netscape it's likely to become popular. It does mean that standard documents appear identical on-screen within the application as they appear on the printer, as they also appear when published on the internet.

Adrian then took us through a live demo of the use of the package – stating he had been toying with the code on the train on the way down so crashes were likely! In the end the only errors were his own typing!

There are effectively two stages to producing a report. The first is to create a page layout and define the different areas of the page – called frames – that the report needs to use. This is done using a graphical designer which allows you to drag frames about and give them different properties – like overflow, wrap etc. – which defines how the text will behave when it gets written into that frame. Once you've defined a page layout and all its frames you can move on to the second stage which is to actually create the report.

Assuming our page definition is held in a variable called *baa*. Also the Newleaf functions are in a Dyalog namespace called *leaf*.

```
leaf.Use baa           a Kicks off the report with baa
leaf.Place 'here is my address' a writes to the first frame
PG+leaf.Close         a Creates the postscript vector
leaf.View PG          a Puts the report on screen
```

Of course many more things are possible. Here is an overview of some of the other things Newleaf supports.

```
leaf.NewFrame ''      a Jumps to the next frame on the page

'Subhead' leaf.Place 'I am a subheading' a Creates a
      subheading in the current frame
```

The left argument is an example of a style. These themselves are user definable and are held in a variable *leaf.Δstyletab*. You can perform inline formatting of text (e.g. bold, italic, subscripts etc.); this is done using HTML-style tags – for example:

```
'Hello you <b>funny</b> guy'
```

Tables are just as easy to create – notice the use of namespaces to split off table specific code.

```
leaf.table.Spread 3 4 p112 a Sticks a table in the current
      frame
```

Graphs are simple to include as well. The *Rain* package uses the same underlying PostScript technology as Newleaf. Hence if we create a PostScript graph from *Rain* we can then just display it in the current frame.

Rain is in the namespace *ch*

```
ch.New 200 144
ch.Set 'style' 'boxed'
ch.Set 'Head' 'Here is a chart header'
ch.Bar +\?5 3p100
graph+ch.Close
```

and switching back to the Newleaf namespace:

```
leaf.IncludeLeft graph a Size as is, keep it on the left -
                        a write any text down the right hand side

leaf.Flow textvec      a Write text in the space remaining
PG+ch.Close           a Create the postscript vector
```

Adrian also demonstrated outputting the PostScript to PDF format as discussed above and also showed a direct conversion to HTML – although this isn't WYSIWYG it will view on any internet browser.

```
leaf.ToPDF PG
leaf.ToHTML PG
```

ISI – Eric Iverson

Adrian then introduced Eric Iverson who was on the last leg of his UK trip and just wanted to say a few words about J! Eric listed four sources where you can find out about J if you want to know more:

1. *Vector* magazine
2. *Byte* magazine (Sept 95)
3. On the website jsoftware.com.
4. The J conference in Toronto in June.

He then stated that one of the most exciting things about J release 3 is the ability to use it purely as a calculation engine. You can write your Windows front end in any other tool (VB, Delphi etc.) and call J from that as an OLE object.

Dyadic Systems – Peter Donnelly

Peter Donnelly of Dyadic systems came to demonstrate the new features within Dyalog 8. When he agreed to do this talk he had not quite realised that it was on the same date as the actual release date of Dyalog 8. Hence, he was feeling a bit tired! Peter explained that he had been up all night adding the finishing touches to the Dyalog 8 release. The last 2% always turns into the last 80%.

New features in Dyalog 8 include a native file system, control structures (identical to PLUS III), monadic use of `⌈WC`, improvements to the grid, changing printer orientation and of course OLE support. Also there is a major addition to the Dyadic namespace implementation via the system variable `⌈PATH` and the system function `⌈EXPORT`. Also, as a rule, you can draw a graphical object on ANY other object.

Namespace extensions

`⌈PATH` is a system global that allows you to set up a search path down which the interpreter will look for functions when executing a line of code. (I'm not sure whether it looks for variables as well – I suspect not?) To help manage this and presumably to prevent name-clashes the only functions that are picked off the path are those that have been explicitly exported via `⌈EXPORT`. `⌈EXPORT` in its monadic form allows you to query whether a function has been made visible to the path or not. The dyadic form sets and unsets the visibility of the object.

Improvements to the grid

These changes were shown via the tutorial workspace *WTUTOR* that comes free with DyalogAPL. The format of the grid can now be controlled by passing in a hierarchy (like with `msoutlin.vbx`) which allows you to represent multi-dimensional data in a two-dimensional grid. The formatting of the cells has been extended to allow the use of `⌈FMT`. Graphics are also allowed to be children of the grid – and rather conveniently you can use the cell co-ordinates to draw them. Thus, it becomes very easy to circle/highlight certain figures on the cell. (OK, it also becomes really easy to write your own minesweeper, but that's not what you should be thinking...)

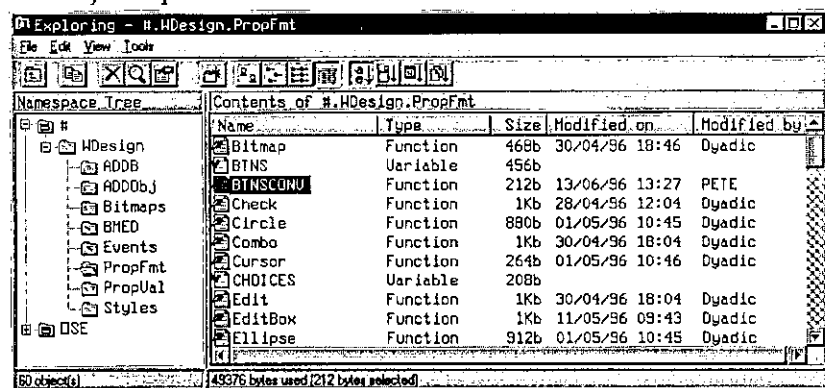
New Windows95 graphical objects

Peter showed us four new Windows95 objects, all of which have been implemented – these have been reviewed in *Vector* before so I won't go into any detail – but they are Imagelist, TreeView, Spinner and RichEdit.

OLE

Peter explained that OLE can be approached from many angles and the way you implement OLE can also be done in many different ways. Dyadic have implemented just one area of OLE which is the ability to call and use OLE controls. You can buy thousands of different types of OLE controls which do everything from temperature controls to complex spreadsheets. These get installed into the Windows95 registry and once installed are available to any application to use. The expression `'.' ⌵WG 'OLEControls'` gets all the names of the OLE controls and the details into DyalogAPL. These objects can then be used just like standard Dyalog objects. If you don't have a manual for them this shouldn't be a problem as Dyadic have implemented an OLE browser which queries the events and properties that each object supports and allows you to search through them. Dyadic have not implemented the king of OLE technology that is now available in J which allows you to call J from other applications. This is the other end of the OLE spectrum.

APL object explorer



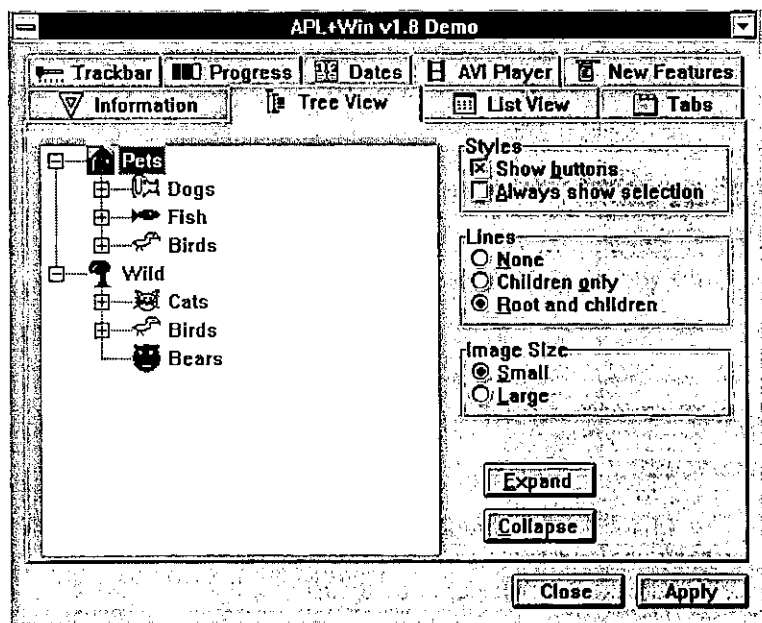
For me this was the highlight of the new features. Peter commented that since this had been implemented it was the first thing he brought up to view the contents of the workspace. In essence this allows you to browse your namespaces using a Windows95 Explorer interface. More interestingly, functions have a full set of properties – timestamps of last change, who changed it, size etc. Just like in Explorer there is a full 'find' facility where you could search for functions changed since a certain date by a certain person etc. You can also do search and replace. (A question asked at the end confirmed that these facilities are not yet available from within the function editor.)

The Bloomsbury Software Company – Peter Day & Dave Phillips

Faced with just ten minutes left in the meeting Peter Day rushed us through the history of the people behind last year's takeover of the APL*PLUS pc products from Manugistics by FRS and the formation of a new company, APL2000, headed by Eric Baelen. FRS's main products had been in APL*PLUS II but they did not feel they were getting the right level of support and development from Manugistics who seemed to have mothballed their development team. They had options – recode in C++, port to DyalogAPL – but these would lose them too much time – which they had none of in a competitive market. Thus, they bought the pc-APL products they required from Manugistics.

APL2000 are currently working on releases of the APL*PLUS products on both Windows platforms – as well as bringing the DOS level release up to speed with control structures and making improvements to the UNIX release.

Bloomsbury Software themselves are heavily involved with Smalltalk and are 'best team partners' with IBM for their *VisualAge* products.



Dave Phillips then took us through a demo using the alpha version of APL+Win. He took us through a demo of all the new objects (which matched the set that

Dyadic had just previewed). He showed an AVI control which supported drag and drop – i.e. drop an AVI file on it from file manager and it plays it back.

Amazingly some of the apparently Windows95-specific objects have also been made available under Windows 3.1. For example, the tree view and list view objects. I'm not sure how they've done this – but it could be quite a selling point for those people who are faced with supporting products under both operating systems, but need to take advantage of some of the new Windows95 features so that they don't lose out to the competition.

Some things have been much improved over the previous version, e.g. speed of the match operator. Some basic GUI features have also been much improved – e.g. Status bars.

OCX support will be provided – but not until version 2.1.

Another addition is the ability to query at any point in time which fields on the form had been modified. This means you can remove all the tedious (and potentially slow) callback code and just deal with changed fields when you feel good and ready. You also only need to run code for fields that have changed.

In the question and answer session an interesting debate occurred after Dave was asked whether the WED workspace was being improved. The user's problem was that it always crashed – particularly when a number of objects were involved. This seemed to make it unusable. Dave answered that the problems were related to the Win32s memory management (and hence the problem should go away under Windows95?) and not strictly a problem with the product. He also said that a product such as Causeway could be used to do this task. Causeway is a specific product designed to be used in place of WED. Another person added that this would only be possible if Causeway could produce a decent grid for PLUS III. Causeway answered that they could if they had the time. This debate was revealing to non-PLUS III users; if Win32s memory management is so unreliable, why build the whole interpreter on top of it? If WED crashed all the time then why ship the product? If Causeway is a better Windows designer then why not ship it with the product – after all Causeway is shareware. All these things need to be answered by the new owners of the PLUS III development team. Let's hope we see some results soon.

All in all the event was very successful. It was good to have another meeting in London after a prolonged break. Let's hope there are more to follow soon!

GENERAL ARTICLES

This section of Vector is intended for general readers who have a working knowledge of APL or J. Authors are encouraged to submit articles which illustrate effective APL applications.

Parallel Processing with J

by Gabriel F. Brisson (*gb2q@andrew.cmu.edu*) and
Clifford A. Reiter (*reiterc@lafcol.lafayette.edu*)

During the summer of 1995 a student research project at Lafayette College focused on the construction of attractors with the symmetry of the cube in 3 dimensions and later this was generalized to the symmetries of the n -cube in any dimension [1,7]. Experiments resulting in images of these attractors were implemented in J 2.06 [5] and run in parallel in order to be able to compute a large number of iterations in a timely fashion. This note describes the simple parallel processing scheme we used which is applicable to much more general problems.

The design of our distributed computing scheme was, of course, heavily influenced by the fact that we wanted to solve the particular problem of creating images of attractors. The study of nontrivial attractors is often traced back to Lorenz who discovered a "strange" attractor associated with a model he was using to study weather [6]. More recently there have been several examples of the construction of attractors in the plane that are forced to have certain symmetries [2,3,4]. The images they created typically showed the results of several million to billions of iterations of the functions they utilize. Since the attractors we planned to investigate were more complicated we wanted to have the flexibility of the programming language J but we expected to need to use a large number of iterations and so we planned to run the iterations in parallel in order to get results "rapidly".

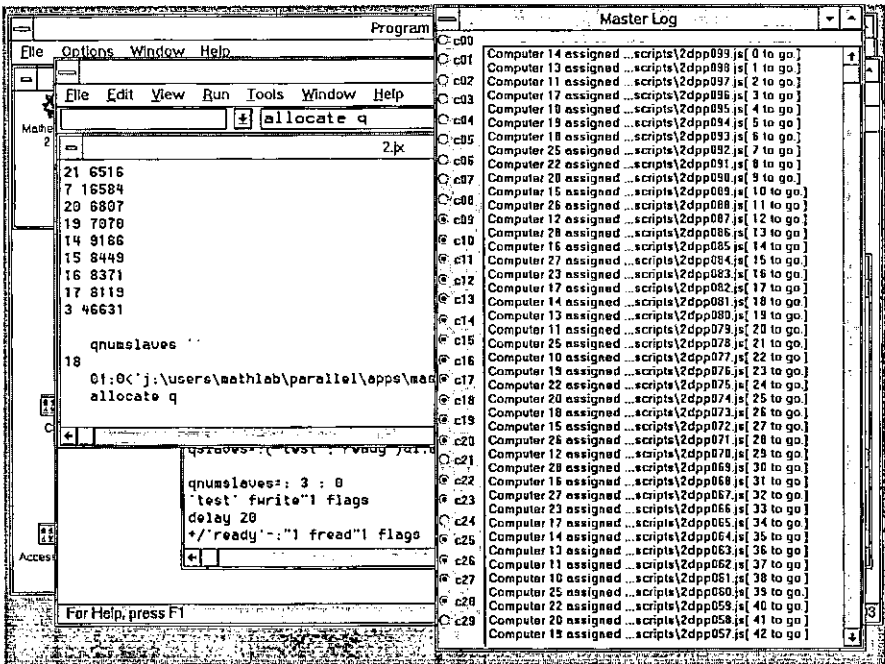
In addition to scripts that ran the experiments, we developed two J scripts to accomplish the parallel computing: one that creates a "master" computer which allocates J scripts that need to be run, the second creates slaves that run the allocated scripts. Below we will describe the scripts and then we will give a performance profile that shows how the system of parallel machines worked in practice as the number of slaves was varied.

A Simple Parallel Processing System

Our parallel processing system assumes that we can break our problem up into separate scripts and that there is a shared file system available to all the computers being used. The computers we used were all running Windows under DOS and hence the system had the full stability one would expect of that platform. We did not attempt to use any sophisticated network communication

— we simply read and wrote to the shared file system in a manner that avoided conflicts and minimized the impact of a single computer crashing.

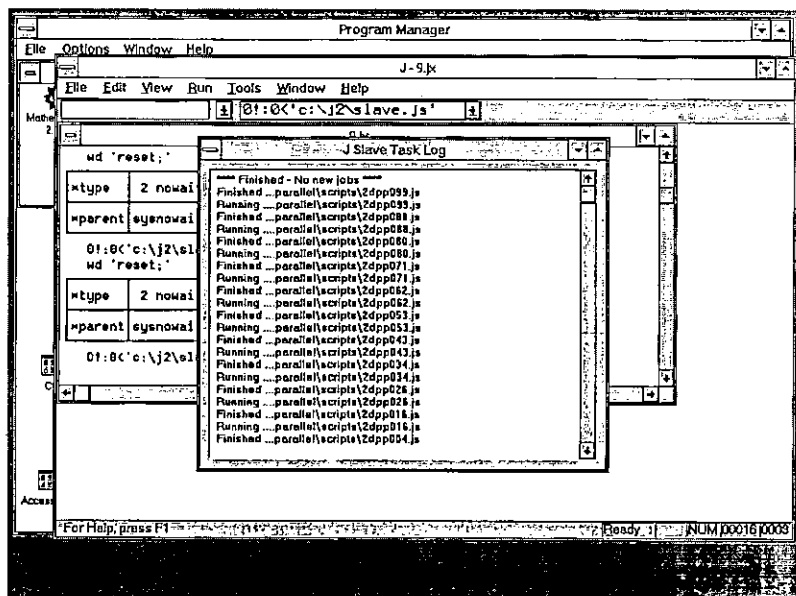
One computer is dedicated as the master computer and it simply polls certain files on the shared file system to see if any slave computer has indicated it is ready to accept a task. If so, the master assigns a script to be run to that slave. The script *master.js* is given in the appendix. Running that script creates a display window containing 30 radio buttons which are used to indicate which computers are active as slaves. The display also contains a log “stack” that indicates which tasks have been assigned to which computer and how many tasks remain to be assigned.



Note that since this is a stack, the most recent messages are on top. The main functions in the script are *createmasterlog* which creates that log window and *allocate* which is used to allocate tasks. In particular, if *tasklist* is an array whose items are character strings that are file names, then *allocate tasklist* would be the command used to allocate that list of scripts. Utilities for writing to the log stack, turning on/off specified radio buttons and giving shortened file

names are also included. With this design the tasks need not be related. Moreover, the master need not be running continuously; in particular, the master can be interrupted in order to add additional tasks to the task list without interrupting other processing.

The script *slave.js* also appears in the appendix. Running this script causes the computer to indicate it is ready for a task and then it polls for a task assignment. After running a script, the slave cleans the workspace of variables defined by running the assigned task and then indicates it is ready for another task. If no new task is assigned after a designated TimeOut period, the slave assumes no new tasks are forthcoming and halts polling. A sample screen from a slave computer is shown in Figure 2.



Organizing our parallel computations in this way provides a fairly general scheme that was suitable for solving our problem. The biggest shortcoming was the lack of error recovery capabilities. Most of the errors we encountered fell into 3 categories.

- Numbers blew up during the number crunching: the solution was to rerun the script with new initial conditions — we automated this as needed.
- The file server filled up: we could have avoided this by testing for free space before writing to the server though one needs to consider simultaneous writes from several slaves — in practice we developed experience that allowed us to time our writing/erasures to avoid this.

- Machines hung for no apparent reason (a feature of Windows) or were turned off by careless visitors to the laboratory: this meant that a script that had been allocated was never completed. We could usually recover manually since the allocation log provided adequate information to determine what script was running when a machine was prematurely terminated. However, it would have been more convenient to have a separate ordered display of completed tasks shown on the master.

While it is possible to trap errors in this version of J, it was difficult to distinguish between different errors that would require different responses. This kind of error analysis ought to be far easier in J3.

Performance Profile for an Application

In order to test the performance of this scheme as the number of slaves varies we selected one of the simpler images that we created and ran the same image with a varying number of slaves. In particular, the functions we want to iterate can be conveniently defined via a conjunction H that gives a function with certain symmetries of the n -cube where n is the number of parameters given on the right of H. For our test problem, we use the function f indicated that has the symmetries of the square.

```
H=:2 : 0
p=. (i. !#y.) A. i. #y.          NB. all permutations of i.n
s=. x. * _1 ^ (1 { y.) ** / @<: @: (#&>) @ C. "1 NB. signs of terms
k=. i. &0 "1 p                  NB. coordinate keys
(+ / . *) "1 & (k] /. s) @ ((*/ . ^) & (0 1 | : k] /. p { y.))
)
```

```
f=: 1 H 1 0 + 1 H 0 1 + _0.5 H 2 1 + _0.65 H 3 2
```

Each script iterates f one hundred thousand times and then creates a 600 by 600 bitmap image of the attractor where colour is used to indicate the number of times each pixel is visited. Other examples of creating bitmaps and images of attractors appear in [8] though the key to efficiently doing the frequency counting (#/.~) required for colouring does not appear there. For this application, each script results in one of these bitmaps which are stored on the shared file system. The final result is created by adding together the arrays in all these bitmaps. Our sample project consists of 100 scripts each running 100,000 iterations (amounting to 10 million total) along with a single script that accumulates the results of all the 100 others. The 100 task scripts are created in an automated fashion and they differ from one another only in the initial seed and the file name to be used for storing the image created by that script. The accumulator is assigned as the first task and runs the entire time the project is

running. Thus, we dedicate one slave to accumulating the results of the others. The accumulator times out after no new images have been created for a specified time and then creates a final image based on the accumulated results. Thus, our sample project consists of 101 scripts, one of which runs the entire time.

The timing tests we ran consisted of timing the accumulator's run time. This is the actual time to a final image and it includes about 25 minutes required for the accumulator to time out and create the final image. The ideal situation for this type of distributed computing would be for the run time to halve when the number of processors doubled. Thus one would expect the running time to be inversely proportional to the number of processors. Alternately, this means that the reciprocal of run time (cycles or number of "runs" per unit time) would be linear in the number of processors. However, because each process takes roughly a fixed amount of time and there are 100 tasks to be distributed on the $N-1$ nonaccumulating slaves we expect performance plateaus. For example, if 100 tasks are distributed on 20 processors, each processor is expected to run 5 tasks. If 100 tasks are distributed on 21 processors, each processor is expected to run 4 or 5 tasks, but the final result needs to wait for the processors that run 5 tasks and hence there is no benefit in increasing to 21 machines.

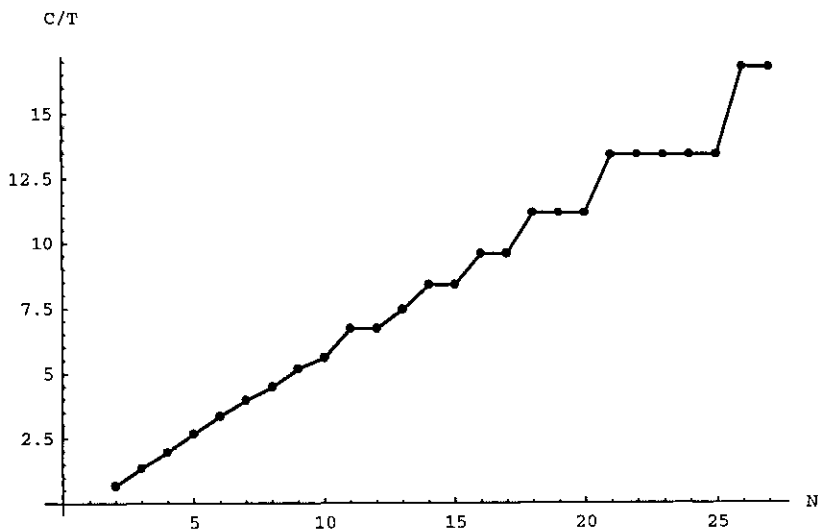
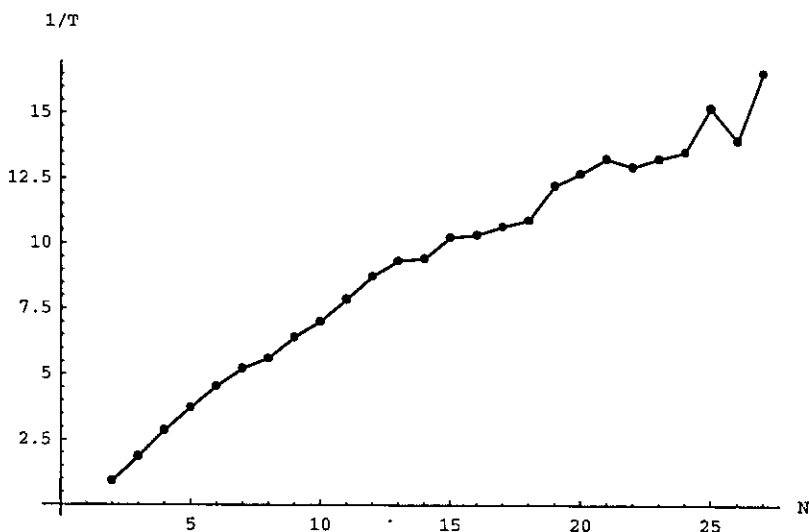


Figure 3 shows the expected performance profile taking this into account. It shows the number of cycles (runs of the 101 tasks) per unit time versus the number of slaves, where the vertical scale is chosen to be compatible with the actual measurements described next. Notice the plateaus.

Figure 4 shows the reciprocals of run time (cycles per day) for our sample project versus the number of slaves for actual experiments in our laboratory.

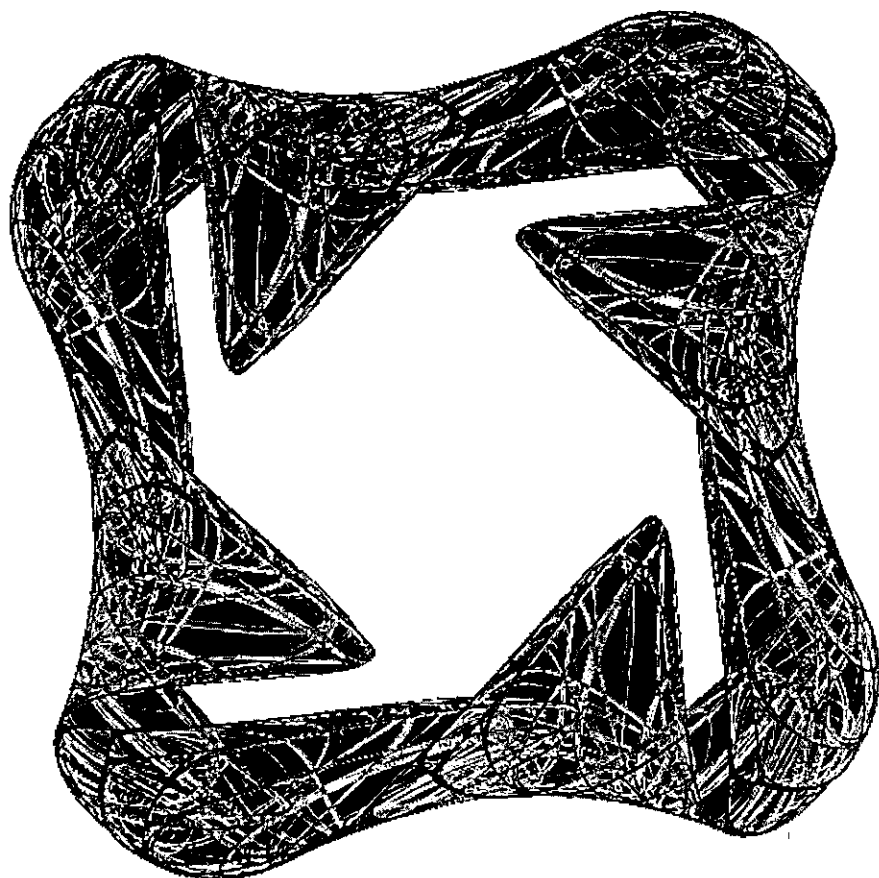


One expects there to be noise in the data since we had no control over network loading from other users (though we tried to mitigate this by running our experiments during holidays). Nonetheless, there is some evidence of plateaus and a roughly linear appearance though one might also observe some general concave down appearance that would be expected from network overloading. We take this performance profile to be surprisingly close to the optimal model shown in Figure 3.

The results of these timing experiments, along with some additional runs, were accumulated into a single image resulting from a billion iterations. The image is shown in Figure 5. Notice the two combatting tendencies in the image: the structure of the symmetry that is preserved by 90° rotations and chaotic appearance of the underlying attractor.

Acknowledgement:

This work was supported in part by NFS REU grant DMS-9424098.



References

- [1] G. Brissou, K. Gartz, B. McCune, K. O'Brien and C. Reiter, *Symmetric attractors in 3D space*, submitted.
- [2] M. Field and M. Golubitsky, *Symmetric chaos*, *Computers in Physics*, 4 (1990) 470-479.
- [3] M. Field and M. Golubitsky, *Symmetry in Chaos*, Oxford University Press, New York, 1992.
- [4] M. Field and M. Golubitsky, *Symmetric Chaos: How and why*, *Notices of the AMS* 42 (1995) 240-244.

- [5] K. Iverson, *J Introduction and Dictionary*. Iverson Software, Inc., Toronto 1995.
- [6] E. N. Lorenz, *Deterministic nonperiodic flow*, Journal of Atmospheric Science, 20 (1963) 130-141.
- [7] C. Reiter, *Attractors with the symmetry of the n-cube*, submitted.
- [8] C. Reiter, *Fractals, Visualization, and J*, Iverson Software, Inc., Toronto, 1995.

Gabriel F. Brisson
 94 Bullman Street,
 Phillipsburg, NJ 08865 USA
 email: gb2q@andrew.cmu.edu

Clifford A. Reiter
 Department of Mathematics, Lafayette College
 Easton, PA 18042 USA
 email: reiterc@lafcol.lafayette.edu

Appendix

The scripts *master.js* and *slave.js* are given below. These scripts, along with scripts that can be used to duplicate Figure 5 using this parallel processing system are available in electronic form from Prof. Reiter.

```
NB. %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% NB. The script
master.js
NB. Create the master log and prepare to allocate tasks
NB. Modify flagpath to point to a shared file system directory
NB. %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

flagpath=: 'j:\users\mathlab\parallel\flags'
fmt=:2&fmt : ({&'0123456789'@()#:- [ # 10"-))
flags=: (flagpath,'\flag'),'1 {fmt i.30}','1 '.txt'
NS=:30 NB. number of slaves

delay=:6!:3
shortfn=: 0&shortfn : ([ :);@; -@>:@[ {. <.:2@({&'\'')@))

log=: 3 : 0
logtext=: ' ',y.,CRLF,logtext
if. 20000<#logtext do. logtext=:_1000).logtext end.
vd 'psel Gabe;csl thelog;ctext "',logtext,'" ;'
)
```

```

createmasterlog=: 3 : 0
vd 'reset;' wd 'pco Gabe;pcloseok;'
vd 'pn "Master Log";'
lds=. fmt 1. NS NB. button numbers
pos=.":. 5*1.NS NB. button positions
vd 'xywh 1 ','1 pos ','1 ' 50 5;cc c','1 lds','1' button
                                bs_autoradiobutton;'
vd 'xywh 25 4 200 150;cc thelog edit es_multiline ws_vscroll;'
vd 'pas 0 0;pscale;pmove 200 0 200 300;pshow;'
logtext=: ''
log y.
)

```

```

hitbutton=: 3 : 0
1 hitbutton y.
:
vd 'csel c', (fmt y.), 'ccheck ', (fmt x.), 'pshow;'
)

```

```

allocate=: 3 : 0
log 'Seeking Slaves'
n=.0
while. (#y.)>0 do.
  status=.fread n{flags
  if. 'ready'-:status do.
    (>{y.) fwrite n{flags
    hitbutton n
    log 'Computer ',(fmt n),' assigned ...',(1 shortfn >{y.),'[
                                ',(":<:#{y.),' to go.]'
    y.=.}. y.
    end.
  if. 'quit' -: status do. 0 hitbutton n end.
  n=. NS|>: n
  delay 1
end.
y.
)

```

```

'test' fwrite"1 flags
createmasterlog ''

```

```

NB. ~~~~~~
NB. The script slave.js
NB. Create the slave Log and run allocated tasks
NB. Modify first line to give each computer a
NB. unique flag file in a shared file system directory
NB. ~~~~~~

```

```

flag=: 'j:\users\mathlab\parallel\flags\flag02.txt'
TimeOut=:60*15      NB. seconds without new job until timeout
vd 'reset;'
delay=:6!:3
shortfn=: 0ashortfn : ([ :):0; -@>:0[ {. <.:2@{,5'\'}@])

log=: 3 : 0
logtext=: ' ',y.,CRLF,logtext
vd 'psel Arno;casei thelog;ctext "',logtext,'"';
)

slave=. 3 : 0
logtext=: ''
t=.0
namesinuse=. 'status'; 'namesinuse'; ;, names ''
vd 'pco Arno;'
vd 'pn "J Slave Task Log";'
vd 'xywh 5 5 200 150;'
vd 'cc thelog edit es_multiline ws_vscroll ws_border;'
vd 'pas 5 5;pcenter;pshow;'
log y.
'ready' fwrite flag
while. t<TimeOut do.
  status=. fread flag
  if. 'js'-: _3{.( ' &~: # ] )status do.
    log 'Running ....',(2 shortfn status)
    0!:0 <status
    erase (;, names ' ')-.namesinuse
    'ready' fwrite flag
    log 'Finished ...',(2 shortfn status)
    t=.0
  end.
  if. 'test'-: status do.
    'ready' fwrite flag
  end.
  delay 10
  t=.t+10
end.
log '**** Finished - No new jobs ****'
)

slave 'Running.'
```

APL and J (1) – Function Rank

by Chris Burke

Though J shares many concepts with APL, in many respects it is radically different, and almost all APL constructions that are in J differ in some way. These differences can be a stumbling block to the experienced APL programmer who would like to learn a little about J.

In a series of articles entitled *APL and J*, we will try to explain the J way of thinking, by contrasting APL and J solutions to specific problems. The term APL will be used generically to refer to facilities found in most commercial implementations of APL.

We will start off by taking a look at function rank – one of J's most important new facilities – and also look at the related topics of J's agreement and assembly rules.

Housekeeping

We are going to need several arrays, and will use the letters *V*, *M*, and *A* followed by numbers to denote arrays and their shape. Thus *V*5 for a vector of length 5, *M*34 for a 3-by-4 matrix, *A*243 for a 2-by-4-by-3 array and so on. Where actual values are required, *V*5 will be defined as 15, *M*34 as 3 4 p 12, etc.

APL terminology will be used, except where J terminology is required for clarity. $\square_{io+1}$ is assumed throughout.

Agreement

Suppose we want to multiply two arrays together (or apply any scalar function to the arrays). What shapes must they have in order for the operation to be successful? In APL, the answer is straightforward: they must either have the same shape, or else one of them must be a scalar, in which case it is reshaped to match the other (this is known as *scalar extension*).

What happens if these conditions are not met, but the operation required is otherwise quite reasonable? For example, it is often the case that you want to multiply two arrays, where the shape of one array matches some leading elements, or trailing elements, of the shape of the other. Let's look at some simple examples, and typical APL solutions.

Example 1

The arrays are $M34$ and $V4$ and we want to multiply each row of the matrix by the vector. Straight multiplication fails:

```
M34 * V4
RANK ERROR
```

The error is a rank error, since APL requires two conforming non-scalar arrays to have the same rank. A standard solution in APL is to reshape the vector into a matrix:

```
M34 * (ρM34)ρV4
```

Another solution uses the following three-step process, which is of general utility. The data is split up into enclosed items, the function is applied to each item in turn, and the results are then re-assembled:

```
⊃(⊃[2]M34)×''⊃V4
```

Example 2

Now let's consider the case $M34$ times $V3$, where we want to multiply each column of the matrix by the vector. Reshape still works, but the shape of the matrix must be reversed, and the result must be transposed:

```
M34 * ⍋(⍋ρM34)ρV3
```

Similarly, three-step needs the result transposed:

```
⍋⊃(⊃[1]M34)×''⊃V3
```

Example 3

In the next example, we multiply each 3-by-4 block of the array $A234$ by the matrix $M34$. The methods of Example 1 can be used essentially unchanged:

```
A234 * (ρA234)ρM34
⊃(⊃[2 3]A234)×''⊃M34
```

Example 4

Finally, we multiply each 3-by-4 block of $A342$ by the matrix $M34$, as follows:

```
A342 * 3 1 2⍋(⌈1⍋ρA342)ρM34
3 1 2⍋⊃(⊃[1 2]A342)×''⊃M34
```

Solutions in J — Examples 2 and 4

In APL, the default axis is the last axis, whereas in J, it is the first axis. Therefore, while in APL, Examples 2 and 4 are more complex than Examples 1 and 3, in J they are simpler:

```
M34 * V3
```

Example 2

```
A342 * M34
```

Example 4

Why do these multiplications work in J? The answer is that two array arguments agree if the frame of one array is a prefix of the frame of the other. This is an extension of the rules of APL.

The term 'frame' will be explained later, but for now, you can substitute the word 'shape' with the same effect. Thus, in Example 2, the shape of *V3* is the first element of the shape of *M34*, while in Example 4, the shape of *M34* is the first two elements of the shape of *A342*. In such cases, J extends the smaller array to match the larger; this is known as *prefix extension*.

Solutions in J — Examples 1 and 3

Direct multiplication does not work this time in J, but note that the error is a length error, not a rank error:

```
M34 * V4
length error
```

```
A234 * M34
length error
```

The solutions use the rank operator:

```
M34 *"1 V4
```

Example 1

```
A234 *"2 M34
```

Example 3

You can get a rough idea of what is going on by thinking of the numeric argument to rank as being the dimension to which the function applies. Thus in J Example 1, the function is applied to the rank 1 arrays, which are the rows of *M34*, and the vector *V4* as a whole. In J Example 3, the function is applied to the rank 2 arrays, which are the matrices of *A234*, and the matrix *M34* as a whole.

Function Rank

In APL, data may be of arbitrary size and dimension, where the dimension of the data is referred to as its rank. J extends this notion to functions — all functions are assigned a rank which determines the behaviour of the function on certain subarrays called the cells.

The shape of an array can be split up into two parts, a leading part called the *frame*, and the remaining part called the *cells*. Either frame or cells can be empty. The frame can be thought of as the holding structure for the cells. The term *k-cell* specifies a cell of rank *k*.

For example, the shape of the matrix $M_{4\ 6}$ can be split up in three different ways:

- the size of the frame is empty, and size of the cell is 4 by 6. The cell is a 2-cell and is the entire matrix.
- the size of the frame is 4 and the size of the cells is 6. The cells are 1-cells and are the rows.
- the size of the frame is 4 by 6, and the size of the cells is empty. The cells are 0-cells and are the scalar elements of the matrix.

A function of *rank k* acts on the *k*-cells, with the following three-step process:

- the arguments are split up into *k*-cells.
- the function is applied between each pair of *k*-cells.
- the individual results are then reassembled.

Note that these steps are analogous to the three-step APL examples. However, they are not identical. In APL, the steps must be given explicitly, and the interpreter carries out each step in turn. In J, the steps are only notional; in general the interpreter knows how to evaluate functions with rank and carries out the evaluation directly.

The *rank operator* can be used to enforce different argument pairings, which are independent of the function to which it is applied. The function is then applied to each of the pairs, and in turn takes care of its own pairings.

Applying Function Rank

Now let's see how function rank is applied in the examples above. In J Examples 2 and 4 no function rank was specified, therefore the multiplications are carried out with the rank assigned to *, which is 0. This means that the cells are the scalars, and the frames are the entire argument shapes. This explains why, in the

discussion on J Examples 2 and 4, it was stated that the term 'frame' could be replaced by the word 'shape'. In J Example 1, the pairing is specified with rank 1. The three-step J process is thus:

Step 1. Break arguments into 1-cells. The 1-cells of $M34$ are the rows:

```
(1 2 3 4) (5 6 7 8) (9 10 11 12)
```

The 1-cell of $V4$ is the entire vector: (1 2 3 4)

Step 2. The function is then applied between the cell pairs:

```
(1 2 3 4*1 2 3 4) (5 6 7 8*1 2 3 4) (9 10 11 12*1 2 3 4)
```

Step 3. The individual results are then assembled into:

```
1 4 9 16
5 12 21 32
9 20 33 48
```

In J Example 3 the pairing is specified with rank 2. The three-step J process thus uses the 2-cells, which for $A234$ are the matrices:

```
1 2 3 4      13 14 15 16
5 6 7 8      17 18 19 20
9 10 11 12    21 22 23 24
```

The 2-cell of $M34$ is the entire matrix:

```
1 2 3 4
5 6 7 8
9 10 11 12
```

This cell is multiplied with the 2-cells of $A234$ and the results assembled into:

```
1 4 9 16
25 36 49 64
81 100 121 144

13 28 45 64
85 108 133 160
189 220 253 288
```

Rank vs Each

J's rank operator " and APL's each operator " are notationally similar, and are often confused. To understand the difference, note that the each operator is used for two essentially different expressions, for which J has two quite different solutions.

The first case is that discussed above, where APL each is used as part of the three-step solution to applying a function to non-conforming arrays; this usage corresponds to J's rank operator.

APL each also can be used to simply apply a function to each element of an array, typically an enclosed array. The corresponding expression in J is also called *each*, which is a standard utility defined as `&. >.` In such cases, the following are the same:

<code>f " A</code>	APL
<code>f each A</code>	J

Assembly

The third step of applying rank is to re-assemble the results. In the examples given here, assembly presented no problem since each item had the same shape. In practice, J can assemble items of different rank and shape as follows: the items are first brought to a common rank by introducing leading unit axes to any of lower rank, and are then brought to a common shape by padding with an appropriate fill element. This allows J to assemble results which in APL would signal rank or length errors:

<code>> (1 2 3) (2 2p10 11 12 13)</code>	APL
<i>RANK ERROR</i>	
<code>>1 2 3;2 2\$10 11 12 13</code>	J
<code>1 2 3</code>	
<code>0 0 0</code>	
<code>10 11 0</code>	
<code>12 13 0</code>	

The combination of J's more general agreement rules, function rank and permissive assembly allows many more calculations to be expressed in J without looping or reshaping.

References

1. Iverson, K.E. *J Introduction and Dictionary*, Iverson Software, 1996
2. Hui, Roger K.W. *Rank and Uniformity*, APL Quote Quad, June 1995 (APL95)
3. Neitzel, Martin. *Untying the Gordian Knot: Agreement in J*, APL Quote Quad, June 1995 (APL95)

however the clue is placed, the middle five squares must overlap and be black and can be placed in the solution. This process is continued for the down clues and the presence of existing black squares may well help place squares from certain down clues and possibly in the case of short single clues, some white squares. The initial grey background starts to fill in with black and white squares. The squares resulting from the down clues help fix further squares from the clues across and so on and so on, hopefully until completion. Artistic interpretation seems to have little to do with it.

The problem leads to an APL solution along the following lines.

The clues for the puzzle above are stored as a matrix as follows.

ACROSS						DOWN					
7	0	0	0	0	0	7	0	0	0	0	0
10	0	0	0	0	0	3	1	0	0	0	0
4	3	0	0	0	0	1	3	0	0	0	0
3	3	0	0	0	0	1	2	1	4	0	0
1	7	0	0	0	0	2	2	8	0	0	0
5	10	0	0	0	0	1	4	1	1	0	0
1	3	12	0	0	0	1	5	1	0	0	0
1	17	0	0	0	0	2	2	2	3	0	0
23	0	0	0	0	0	7	3	0	0	0	0
22	0	0	0	0	0	6	2	0	0	0	0
1	2	17	0	0	0	2	9	0	0	0	0
1	18	0	0	0	0	2	8	2	0	0	0
2	1	3	9	4	0	2	13	0	0	0	0
1	2	10	4	0	0	2	12	0	0	0	0
1	1	9	3	4	0	2	11	0	0	0	0
1	3	5	3	2	2	2	9	0	0	0	0
1	3	2	2	3	2	2	14	0	0	0	0
1	3	3	2	3	0	2	8	4	0	0	0
5	2	2	0	0	0	3	9	0	0	0	0
3	0	0	0	0	0	15	0	0	0	0	0
						14	2	0	0	0	0
						16	0	0	0	0	0
						6	5	0	0	0	0

Rows and columns are padded with zeros to accommodate the width of the longest clue. These arrays are stored in the nested array *TNA3* in *TNA3[num;1]* and *TNA3[num;2]* respectively where *num* is the position in *GAMES* of the particular problem.

The function *SOLVE* reads in the clues, sets up the array *SOL* and proceeds as follows:

See if any clues cause an overlap (function *OVERLAP*) and enter the result in the solution matrix *SOL*.

See if subsequent to the use of *OVERLAP* any unfinished line in the puzzle must have all unfilled squares as either all black or all white entries. (The function *TOTALS*).

Prior to the use of the different techniques for determining how to attack particular clues calculate how many combinations are possible for every clue and record the relevant statistics. This is the function *SETWC* which creates vectors (one position for each clue, so length is A+D where A is across and D is down) as follows:

MSS Number of possible combinations for each clue.

WC1 Clues in increasing order of number of combinations possible

WC2 Clues that have had a change made to them by *OVERLAP* or *TOTALS* but which are not completely solved.

WC3 Indicates whether a clue has more or less than 1000 possible solutions.

WC4 Holds the index to the position in the nested array *TNA* which holds the corresponding matrix of the possible clue combinations(provided it is not too big!)

The description of these matrices is set out in the Appendix.

When clues have a 1000 or more possible combinations they are dealt with in parts, the function *SHRINK* restricts the size of the combination matrix that will be used. Initially it is set to size 200 and this will only be increased (by steps of 400) if there is no other way to solve the nonogram.

The first ten clues, which are those which have the lowest number of possible solutions, and which also have had some entries made, are processed in increasing order of complexity (See line [8] of *SOLVE*), these clue numbers are put in *BUFFER*.

When the clue has been processed the clue change indicator *WC2[*NUM*]* is set to zero until a further change is made to that clue. *NUM* refers the position of the clue in the array, *num* is the corresponding position in either Clues Across *CLUEA* or Clues Down *CLUED*. If *NUM = num* we must be processing an across clue. This test is important in correctly indexing the solution matrix *SOL*

Clues Across, [1] Clues Down is called *CLUES*.

The function *CHOOSE* decides how we are to process a particular clue. This is determined by *WC3* which has entries initialised in *SETΔWC*, *WC3* entries however can be changed later.

Possible clue solving approaches are:

- | | |
|------------------------|--|
| <i>C0</i> | All clues with less than 1000 possible combinations. The Appendix explains how this works. |
| <i>C1</i> or <i>C3</i> | All clues with 1000 or more possible combinations. |
| <i>C4</i> | <i>C0</i> clues on second and subsequent processing. |
| <i>C2</i> | <i>C1</i> or <i>C3</i> clues near the end of the problem solving. |

The somewhat illogical naming of the clue solving approaches reflects the different times at which these techniques were devised.

Prior to using *C1* or *C3* the function *REDUCE* is used, this:

- I. Removes the second and other contiguous white squares.
- II. Chops off completed clues from the left or right edge.

following the use of *REDUCE* it may be possible to fully process the remaining clue and part of the *SOL* matrix in which case *C1* is used or if not the left and right edges are tackled using *C3*.

Eventually, when most (¾) of the solution matrix has been completed a further method is introduced replacing many *C1/C3* clues, this is *C2*.

This reflects the fact that at this stage it is usually more fruitful to calculate the possible combinations of all the outstanding squares rather than try to reduce the clue. The nested array *TNA2* is used in conjunction with this and the function *CHANGEΔWC* is the precursor to this.

The process continues with *BUFFER* being refreshed from time to time. If this is not possible at any stage slightly larger numbers of combinations are allowed in solving using *C1* and *C3*.

This approach solves virtually all nonograms. It is possible to use larger matrices than are stored in *TNA*, but to avoid overflowing the workspace these would be written out to file and brought in and out when required.

Other functions listed are:

COMB	Combination function.(page 159 of Gilman & Rose, 3rd edition)
FAULT	Indicates if an error has occurred when the nonogram is being solved providing the complete solution has been previously stored in TNA[;3], this is very useful when trying out new ideas.
INSTALL	Sets up TNA and TNA2.
PICTURE	Argument 0 or 1. Shows the nonogram white on black or black on white.

A typical 25 x 20 nonogram takes about 2 seconds to solve on a 486. Finally ...

CAT AND MOUSE



Appendix

The key to the nonogram solving routines is the use of matrices that store all possible ways in which the white squares can be arranged for a given clue.

Thus for a clue, 10,10 in a row of 25 squares the numbers stored would be:

0 1 4	1 1 3	2 2 1
0 2 3	1 2 2	2 3 0
0 3 2	1 3 1	3 1 1
0 4 1	1 4 0	3 2 0
0 5 0	2 1 2	4 1 0

The two ten black blocks being inserted between the first and second elements and the second and third elements.

The above matrix is derived from (2 COMB 6), 6 less 1, 2 COMB 6 where 2 COMB 6 refers to all combinations of two different numbers chosen from the numbers one to six and arranged in ascending order.

((2 COMB 6), 6)	(1, 2 COMB 6)	RESULT
1 2 6	1 1 2	0 1 4
1 3 6	1 1 3	0 2 3
1 4 6	1 1 4	0 3 2
1 5 6	1 1 5	0 4 1
1 6 6	1 1 6	0 5 0
2 3 6	1 2 3	1 1 3
2 4 6	1 2 4	1 2 2
2 5 6	1 2 5	1 3 1
2 6 6	1 2 6	1 4 0
3 4 6	1 3 4	2 1 2
3 5 6	1 3 5	2 2 1
3 6 6	1 3 6	2 3 0
4 5 6	1 4 5	3 1 1
4 6 6	1 4 6	3 2 0
5 6 6	1 5 6	4 1 0

This process of differencing combination matrices works for every possible clue, the problem being that they get too big and take too long to process! The matrix of all possible white square positions is interleaved with the clue e.g.

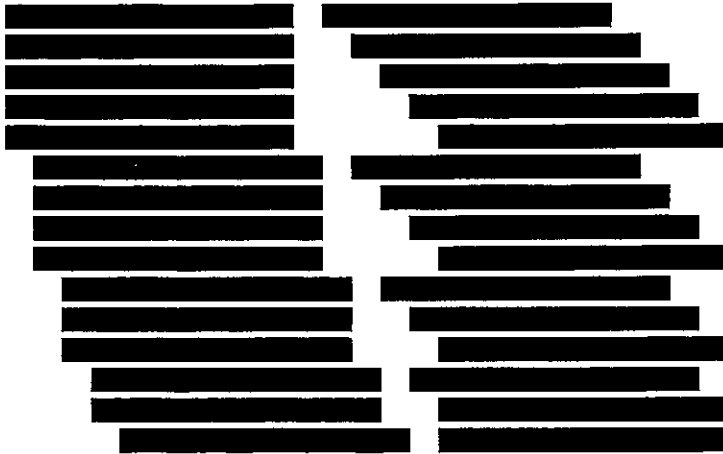
```

0  10  1  10  4
0  10  2  10  3
etc. etc.
```

This is then expanded, -1 for whites, 1 for blacks to:

1 1 1 1 1 1 1 1 1 1 -1 1 1 1 1 1 1 1 1 1 1 -1 -1 -1 -1
 1 1 1 1 1 1 1 1 1 1 -1 -1 1 1 1 1 1 1 1 1 1 -1 -1 -1

etc. which looks like :



The solution to date of the clue is compared with every row and those rows that have 1s and -1s in the same position are totalled. If *every* entry of a column is 1 or -1 then because all possible combinations have been tried any new 1 or -1 can be entered into the solution matrix.

```

▽ C0
[1] L+1+PM=>TNA[WC4[NUM]]
[2] PROCESS
[3] +4+NUM*num
[4] SOL[num;WP,WM]+*WP,-WM    0+6
[5] SOL[WP,WM;num]+*WP,-WM
[6] WC3[NUM]+8    MAT[NUM]+<M[W;]
▽

▽ C1
[1] L+1+PM=>TNA[WC4[NUM]]
[2] PROCESS
[3] +4+NUM*num
[4] SOL[num;MAP[WM]]+-1    SOL[num;MAP[WP]]+1    0+0
[5] SOL[MAP[WM];num]+-1    SOL[MAP[WP];num]+1
▽
    
```

▽ C2

```

[1] BNYD+((+/CLUE)-+/SOLN=1  oNOC+BNYD!NYD+/SOLN=0
[2] M+ (NOC, pSOLN) p+1+SOLN
[3] M[;WHERE SOLN=0]+>TNA2[BNYD+40*NYD]
[4] CLUE+,CLUE,[1.1]*CLUE
[5] CLUE+-1+CLUE/(pCLUE) p1 0
[6] M1+,M,9  oM1+9,((M1+0)v(1+M1+0),1)/M1
[7] W+ +f(WHERE M1=9) o.<WHERE CLUE≤ M1
[8] CUM+ +fM[W;]
[9] WM+WHERE CUM=0  o WP+WHERE CUM=pW
[10] +11+NUM*num
[11] SOL[num;WP,WM]+*WP,-WM  o+13
[12] SOL[WP,WM;num]+*WP,-WM
[13] WC3[ NUM]+B  o MAT[ NUM]+c*-.5+M[W;]

```

▽

▽ C3

```

[1] COMBINEL
[2] L+1+pM+>TNA[WC4[ NUM]]
[3] PROCESS
[4] +5+NUM*num
[5] SOL[num;MAP[(WM≤LHS)/WM]]+-1  o SOL[num;MAP[(WP≤LHS)/WP]]+1  o+7
[6] SOL[MAP[(WM≤LHS)/WM];num]+-1  o SOL[MAP[(WP≤LHS)/WP];num]+1
[7] COMBINER
[8] M+>TNA[WC4[ NUM]]
[9] PROCESS
[10] +11+NUM*num
[11] SOL[num;MAP[(WM≥I)/WM]]+-1  o SOL[num;MAP[(WP≥I)/WP]]+1  o+0
[12] SOL[MAP[(WM≥I)/WM];num]+-1  o SOL[MAP[(WP≥I)/WP];num]+1

```

▽

▽ C4

```

[1] SOLNNZ+SOLN(NZ+WHERE SOLN]
[2] M+>MAT[ NUM]
[3] W+WHERE,((1+pM),1)+SOLNNZ≤M[;NZ]
[4] CUM+ +fM[W;]
[5] WM+WHERE CUM=-pW  o WP+WHERE CUM=pW
[6] +7+NUM*num
[7] SOL[num;WP,WM]+*WP,-WM  o+9
[8] SOL[WP,WM;num]+*WP,-WM
[9] MAT[ NUM]+cM[W;]

```

▽

▽ CHANGEΔWC

```

[1] TEMP+WHERE WC3*B
[2] BNYD+((+/CLUES)-(+/SOL=1),+fSOL=1
[3] WNYD+((+/SOL=0),+fSOL=0)-BNYD
[4] WC1+(WC1-TEMP),TEMP[&NOC+BNYD[TEMP]:(BNYD+WNYD)[TEMP]]
[5] WC3[TEMP[WHERE (NOC<250 )^21>(BNYD+WNYD)[TEMP]]]+7
[6] COUNT+1

```

▽

```

▽ CHOOSE
[1] +2+14*NUM>num
[2] OS+SOLN+SOL[num;] ◊ CLUE+CLUES[NUM;]-0 ◊+((+/CLUE)=+/SOLN=1)/13
[3] →((D-+/CLUE)=+/SOLN=1)/14
[4] →WC3[NUM]
[5] AA
[6] C0 ◊ →15
[7] C2 ◊ →15
[8] C4 ◊ →15
[9] REDUCE
[10] →12-WC4[NUM]∈TMATRECRED
[11] C1 ◊ →15
[12] C3 ◊ →15
[13] SOL[num;]+x-.5+SOLN ◊ →15
[14] SOL[num;]+x.5+SOLN
[15] WC2+WC2v(Ap0), OS*SOL[num;]◊+0
[16] OS+SOLN+SOL[;num] ◊ CLUE+CLUES[NUM;]-0
[17] →((+/CLUE)=+/SOLN=1)/28
[18] →((A-+/CLUE)=+/SOLN=1)/29
[19] →WC3[NUM]+15
[20] AA
[21] C0 ◊ →30
[22] C2 ◊ →30
[23] C4 ◊ →30
[24] REDUCE
[25] →27-WC4[NUM]∈TMATRECRED
[26] C1 ◊ →30
[27] C3 ◊ →30
[28] SOL[;num]+x-.5+SOLN◊ →30
[29] SOL[;num]+x.5+SOLN
[30] WC2+WC2v(A+D)◊OS*SOL[;num]
▽

▽ R+M COMB N;M;N
[1] →(M=1,N)/1,r
[2] R+1+(0,(M-1)COMB N-1),[1]M COMB N-1 ◊ →0
[3] 1:R+(1N)◊.x11 ◊ →0
[4] r:R+(11)◊.x1N
▽

▽ COMBINEL
[1] OLDCLUE+CLUE ◊OLDMAP+MAP
[2] SOLN+(-1+1+CLUE)+SOLN◊CLUE+1+CLUE
[3] WC4[NUM]+((pSOLN)-+/CLUE)+50x-1+pCLUE
[4] →(~WC4[NUM]∈TMATRECRED)/2
[5] LHS++/CLUE+1 ◊SOLN[LHS+1pSOLN]+0 ◊ MAP+(pSOLN)+MAP
▽

```

```

V COMBINER
[1] CLUE+(-pCLUE)+OLDCLUE
[2] SOLN+((+/OLDCLUE+1)-+/CLUE+1)+OS[OLDMAP]
[3] RHS+(/CLUE+1 o MAP+(-pSOLN)+OLDMAP
[4] SOLN[(pSOLN)-RHS]+0 o I+1+(pSOLN)-RHS
V

V R+FAULT
[1] a RETURNS 1 IF WRONG
[2] R+0<SOL+TNA3[n;3]
V

V R+N FROM M;N;M
[1] R+(PM,M+1)-1,PM+N COMB M+1
V

V INSTALL
[1] TNA+520p0 oM+1
[2] (TNA[M])+c1 FROM M o+(50zM+M+1)/QLC oM+1
[3] (TNA[50+M])+c2 FROM M o+(44zM+M+1)/QLC oM+2
[4] (TNA[100+M])+c3 FROM M o+(18zM+M+1)/QLC oM+3
[5] (TNA[150+M])+c4 FROM M o+(12zM+M+1)/QLC oM+4
[6] (TNA[200+M])+c5 FROM M o+(11zM+M+1)/QLC oM+5
[7] (TNA[250+M])+c6 FROM M o+(11zM+M+1)/QLC oM+6
[8] (TNA[300+M])+c7 FROM M o+(11zM+M+1)/QLC oM+7
[9] (TNA[350+M])+c8 FROM M o+(11zM+M+1)/QLC oM+8
[10] (TNA[400+M])+c9 FROM M o+(12zM+M+1)/QLC oM+9
[11] (TNA[450+M])+c10 FROM M o+(12zM+M+1)/QLC
[12] a a a NOW INSTALL THE BINARY STUFF
[13] TNA2+1000p0 oB+2
[14] LOOP:INSTALLBIN2 b
[15] +(25>b+b+1)/LOOP
V

V INSTALLBIN2 b;a
[1] a+1
[2] LOOP:
[3] +(1000<a!b)/QLC+1o(TNA2[a+40*b])+ca INSTALLBIN3 b
[4] +(b>a+a+1)/LOOP
V

V R+a INSTALLBIN3 b ; acb;ab
[1] acb+a COMB b
[2] ab+((a!b),b)p0
[3] N+1
[4] LOOP:
[5] ab[N;acb[N;]]+1
[6] +(a!b)zN+N+1)/LOOP
[7] R+ab
V

```

```

▽ R←MATSIZE
[1] TEMP1←1+((A,D)/D,A)÷+/CLUES
[2] TEMP2÷+/×CLUES
[3] R←TEMP2:TEMP1
▽
▽ OVERLAP;NUM
[1] SOL[WHERE 0=+/CLUEA;]←-1
[2] NUM←0
[3] →(A<NUM+NUM+1)/9
[4] CLUE←CLUEA[;NUM;]÷0
[5] →(D≥2×+/CLUE)/3
[6] L←-1+(,CLUE,[1.1](ρCLUE)ρ1)/(2×ρCLUE)+,(12),[1.5]0
[7] R←(-D)+L ∘L+D+L
[8] SOLA[;NUM;WHERE(L=R)×L≠0]+1 ∘ →3
[9] NUM←0
[10] SOL[;WHERE 0=+/CLUED]+-1
[11] →(D<NUM+NUM+1)/17
[12] CLUE←CLUED[;NUM;]÷0
[13] →(A≥2×+/CLUE)/11
[14] L←-1+(,CLUE,[1.1](ρCLUE)ρ1)/(2×ρCLUE)+,(12),[1.5]0
[15] R←(-A)+L ∘L+A+L
[16] SOLD[WHERE(L=R)×L≠0;NUM]+1 ∘ →11
[17] SOL←SOLD∨SOLA ∘ SOLD∧SOLD

```

```

▽ PICTURE INDIC
[1] 2/□AV[33 178 220][□IO+1+SOL××INDIC-.5]
▽

```

PICTURE is not suitable for APL+PLUS III as its atomic vector does not have the grey scale characters.

```

▽ PROCESS
[1] WM←WP÷10
[2] SOLNNZ←SOLN[NZ+WHERE SOLN] ∘→(0=ρSOLNNZ)/E
[3] M←(,M),[1.1],(ρM)ρCLUE,0
[4] M←(L,ρSOLN)ρ(,M)/(×/ρM)ρ-1,1
[5] W←WHERE,(L,1)+SOLNNZεM[;NZ]
[6] CUM←+M[W;]
[7] WM←WHERE CUM=-ρW ∘ WP+WHERE CUM=ρW ∘→0
[8] E←→((ρSOLN)≥2×+/CLUE)/0
[9] Le←-1+(,CLUE,[1.1](ρCLUE)ρ1)/(2×ρCLUE)+,(12),[1.5]0
[10] R←(-ρSOLN)+Le ∘Le+(ρSOLN)+Le
[11] WP←WHERE Le×Le=R
▽

```

```

V REDUCE
[1] MAP←WHERE ~('1,'1) ∈ SOLN
[2] SOLN←SOLN[MAP]
[3] BW←WHERE (1,'1) ∈ SOLN ∘ WB←WHERE ('1,1) ∈ SOLN ∘ G←WHERE 0=SOLN
[4] BW←(0,BW)[1+~1+WHERE BW<1+G] ∘ WB←((1+ρSOLN),WB)[1+1+WHERE WB>~1+G]
[5] MAP←BW+WB+MAP
[6] BW←+/1=BW+SOLN ∘ WB←+/1=WB+SOLN
[7] BW←1+WHERE BW=+CLUE ∘ WB←1+WHERE WB=+CLUE
[8] CLUE←BW+(-WB)+CLUE ∘ SOLN←OS[MAP]
[9] WC4[NUM]+((ρSOLN)-+/CLUE)+50×~1+ρCLUE
V
V SETΔWC
[1] CLUES←CLUEA,[1]CLUED
[2] WC1←ΔMSS+MATSIZE
[3] WC2←*(+/SOLA*SOL),+/SOLD*ρSOL
[4] WC2[W2,A+W4]+0
[5] WC3←6+3×MSS>1000
[6] WC4←~51+TEMP1+50×TEMP2 A CALCS IN MATSIZE
V
V SHRINK ZZ
[1] TMATREC←TMATREC[WHERE TMATREC[;2]<TRED-ZZ;][;1]
V
V SOLVE GAME
[1] +2+TRED=200
[2] SHRINK 200
[3] START←SETΔWC ∘→8
[4] +(1∈WC2)/7+COUNT ∘ SHRINK TRED+TRED+400
[5] WG←WHERE (MSS>1000)×WC3≠8 aNUMBER CO CRITERIA SET TO
[6] WC2[WG[WHERE WG∈(WHERE-0Δ.=ρSOL),A+WHERE-0Δ.=SOL]]+1 aC1'S, C3'S
[7] +(QUARTERΔ+/+/SOL=0)/[LC+1] ∘ CHANGEΔWC
[8] BUFFER←φ{(10+(WC1×WC2[WC1])-0)-0 ∘ nb←BUFFER-A×A←BUFFER ∘→19-ρBUFFER
[9] NUM←BUFFER[10] ∘ num←nb[10] ∘ WC2[NUM]+0 ∘ CHOOSE
[10] NUM←BUFFER[9] ∘ num←nb[9] ∘ WC2[NUM]+0 ∘ CHOOSE
[11] NUM←BUFFER[8] ∘ num←nb[8] ∘ WC2[NUM]+0 ∘ CHOOSE
[12] NUM←BUFFER[7] ∘ num←nb[7] ∘ WC2[NUM]+0 ∘ CHOOSE
[13] NUM←BUFFER[6] ∘ num←nb[6] ∘ WC2[NUM]+0 ∘ CHOOSE
[14] NUM←BUFFER[5] ∘ num←nb[5] ∘ WC2[NUM]+0 ∘ CHOOSE
[15] NUM←BUFFER[4] ∘ num←nb[4] ∘ WC2[NUM]+0 ∘ CHOOSE
[16] NUM←BUFFER[3] ∘ num←nb[3] ∘ WC2[NUM]+0 ∘ CHOOSE
[17] NUM←BUFFER[2] ∘ num←nb[2] ∘ WC2[NUM]+0 ∘ CHOOSE
[18] NUM←BUFFER[1] ∘ num←nb[1] ∘ WC2[NUM]+0 ∘ CHOOSE
[19] +4×0∈SOL
V
V START
[1] MAT←130ρ0
[2] n←θρ(11+WHERE (,GAMES) [SS 12+,GAME])÷12
[3] A←θρ1+ρCLUEA→TNA3[n;1] ∘ D←θρ1+ρCLUED→TNA3[n;2]

```

```
[4] SOL←SOLA+SOLD+(A,D)ρCOUNT+0  oQUARTER+A×D÷4
```

```
[5] OVERLAP
```

```
[6] TOTALS
```

```
▽
```

```
▽ TOTALS
```

```
[1] W2←WHERE(+/CLUEA)=+/SOL=1
```

```
[2] W4←WHERE(+/CLUED)=+/[1]SOL=1
```

```
[3] →(0=ρW2,W4)/0
```

```
[4] SOL[W2;]←×SOL[W2;]-0.5
```

```
[5] SOL[;W4]←×SOL[;W4]-0.5
```

```
▽
```

```
▽ R← WHERE vec
```

```
[1] R←(×|vec)/!ρvec
```

```
▽
```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: A Word6 APL Keyboard	Ajay Askoolum	116
Technical Correspondence		119
Norman Thomson		
Dietrich Trenkler		
Advanced Windows Programming with APL and Delphi	Eric Lescasse	122
APL Operators and J	Norman Thomson	143
The J DLL and Extended Integers	Chris Burke	149
At Play with J: Riding a Unicycle	Gene McDonnell	154

Hackers' Corner: Keying APL Characters in WORD6

by Ajay Askoolum

It would be quite convenient to be able to key APL characters directly into a WINDOWS word processor in order to create, for instance, fax messages. With WORD6 this is possible in a few simple steps:

First step, copy the default WORD6 template NORMAL.DOT to another name in order that you can revert to it if necessary. In order to locate NORMAL.DOT, type DIR NORMAL.DOT /S followed by <ENTER> at the DOS prompt, on each of your logical/physical DOS drives. This will respond with the location of the file, perhaps as follows:

```
Volume in drive I is MS-DOS_6
Volume Serial Number is 3A2A-18E8
Directory of I:\MSOFFICE\WINWORD\TEMPLATE
NORMAL   DOT               11,776 04/05/96   20:37
        1 file(s)           11,776 bytes
Total files listed:
        1 file(s)           11,776 bytes
                          7,454,720 bytes free
```

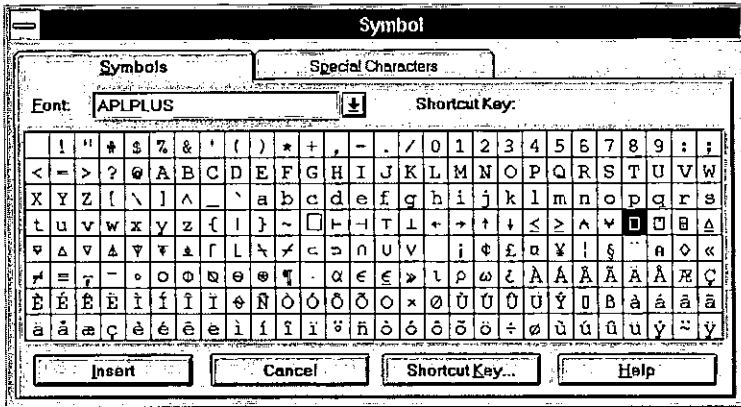
Then copy NORMAL.DOT to, say, XNORMAL.DOT as follows:

```
copy i:\msoffice\winword\normal.dot i:\ msoffice\winword\xnormal.dot
<ENTER>
```

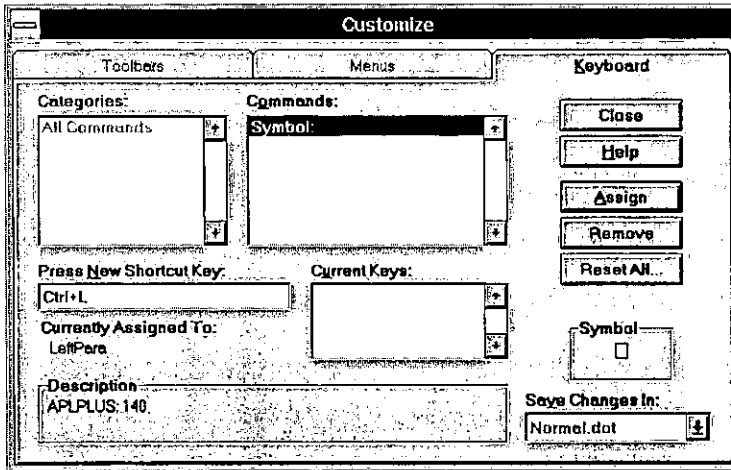
Remember to use the path reported by DIR NORMAL.DOT /S on your computer.

Second step, click on File, New to open a new document based on the NORMAL.DOT template.

Third step, click on Insert, Symbol and the following dialogue will be displayed.



Select the APLPLUS font, then highlight an APL character (⊞ is highlighted above), click on Shortcut key and the following dialogue will appear:



Note:

- I have chosen Ctrl+L as the short cut key. Shift+L will not work! For other characters such as 4 use Ctrl+Shift+3. The use of Ctrl is preferable to Alt since WINDOWS uses Alt.

- The short cut key is shown. If the character is already mapped, it will be shown in the Current Keys box. Click on Assign to make the assignment. An assignment made in error can be corrected by highlighting it and clicking on Remove.
- The changes will be made in NORMAL.DOT — This saves having to select the file in the Save Changes In box. After all the keys have been mapped, NORMAL.DOT can be re-named more appropriately to, say, APLPLUS.DOT and NORMAL.DOT can be restored from XNORMAL.DOT.
- Click on Close to save the assignment and then click on Close to close the Symbol dialogue.
- In order to test the assignment, press Ctrl+L: □ should appear; if not, review what you have done.
- There is no need to re-assign typewriter keys. Besides WORD6 would not allow it *in situ*.

Repeat step 3 for all the APL keys. The layout of the keyboard is found in APPENDIX E of the APL*PLUS III Reference Manual. Alternatively, type JKB within an APL*PLUS III session if the user command processor is installed for a display of the layout.

If you use multiple APL interpreters, use the font which comes with the interpreter. This enables APL characters to be mapped consistently for use within WORD6 and may confuse the meaning of Unified Keyboard yet further. The symbol $\circ$ in the ISIAPL font is the only one which I have come across which cannot be assigned for use within WORD6 since WORD6 uses it to mark paragraph end (character 0182).

If an APL character is assigned to a key combination that WORD6 already uses, WORD6 will re-assign its function to another key combination. For instance, WORD6 used Ctrl+A to 'Select All' text in a document and upon α . being assigned to the same key combination, WORD6 will reassign 'Select All' to Ctrl+Num5. This is done automatically.

Any APL expression thus entered into WORD6 (using whatever font for typewriter keys but APLPLUS, or another, for symbols) can be highlighted, copied to the clipboard and pasted into APL where it can be executed as though it had been entered within the session.

[Ed – except for a very few characters which Word thinks are for typesetting use. Beware of Φ Ψ in the DyalogAPL and APL+Win mapping – they paste as " into the APL session]

TECHNICAL CORRESPONDENCE

From: Norman Thomson

May 1996

I received a letter from Dietrich Trenkler, following our recent articles on confidence intervals [1], [2]. In [2] I observed that I did not understand what Dietrich meant by "coverage probabilities"; Dietrich has now explained this, and I am happy to translate his clarification into APL terms which follow on from my version of the Clopper-Pearson routines.

As a reminder, here is the point where, having observed R successes out of, say $N=10$, 95% confidence intervals are calculated for the true but unknown underlying binomial probability p . R can range from 0 to 10, and the specific values of the confidence interval boundaries are:

$T_{.,[10]0.95} \text{ clob}^{-1} 10, T+0, 110$		
0	0	0.3085
1	0.0025286	0.44502
2	0.025211	0.5561
3	0.066739	0.65245
4	0.12155	0.73762
5	0.18709	0.81291
6	0.26238	0.87845
7	0.34755	0.93326
8	0.44391	0.97479
9	0.55498	0.99747
10	0.6915	1

Now fix on a specific value P of p . Each of these 11 intervals either contains P , or it doesn't. For example, if $P=0.5$, the intervals corresponding to R in the range 2..8 contain P . These seven values of R can be regarded as "plausible observations" given this value of P . The sum of the binomial terms for these values of R is the probability that one of the plausible values in the above sense is observed. This is the coverage probability for P .

The confidence values boundaries form a finite set of values, which means that coverage probabilities as a function of p have discontinuities at these points. For example as p is reduced below 0.44502 the value $R=1$ comes into the plausible

set, and another term is added into the binomial sum. By extending the operations in [2] in the following way

```
[0] Z←N(C BinCOV)P;T
[1] * Z is coverage probability for given P; C is conf. level
[2] Z←N(P Bin)(P IN" C clb"N,T)/T+0,1N
```

```
[0] Z←N(P Bin)R
[1] * Z is sum of pR binom terms; R is a set of powers of P
[2] Z+/(P*R)*(1-P)*N-R)*R!N
```

```
[0] Z←L IN R
[1] * Z is 1 if l is strictly within interval R
[2] Z+0[-*×/R-L
```

```
10(0.95 BinCOV)"0.444 0.445 0.4451 0.4452
0.99315 0.99313 0.97092 0.97095
```

the coverage probabilities for various values of p can be obtained, and these discontinuities observed. This is what is illustrated in Dietrich's graph on p. 93, which was not, as I wrongly suggested, mis-transcribed.

The salient point, as Dietrich points out, is that the coverage probabilities can be proved mathematically to be consistently greater than $1-\alpha$, something which is not shared by other comparable techniques for calculating confidence intervals.

I hope these additions will further clarify Clopper-Pearson intervals, and associated techniques.

References

- [1] *Computing Clopper-Pearson Confidence Limits by the Illinois Method*, D. Trenkler, The Random Vector, Volume 12, No.2, pp. 87-94, October 1995
- [2] *Technical Note on Confidence Limits*, N. Thomson, The Random Vector, Volume 12, No.3, pp. 98-106, April 1996

From: Dietrich Trenkler

June 1996

Vector claims to be the best journal of APL and I dare not contradict. We are proud of its elegance and power but on the other hand I met some unintelligible resistance when I submitted articles written in LaTeX the other day (*Vector* 12,1 and 12,2).

We are working hard to convince people that APL is also very well suited to the mathematically unsophisticated user, but where is *Vector's* support for those who write maths? I hold, in this respect, *Vector* is unduly conservative, being an attitude we APLers always have to fight against. In the meantime (La)TeX has become an accepted standard in scientific journals and books and *Vector* resembles a firm fortress withstanding the breakers. I am not willing to believe that this is because the journal is British.

To say it frankly, to my eyes most formulas in *Vector* look horrible and I shrink back from submitting a paper being forced to use a word processing software (Word) which does not correspond to the standards I am used to and which deprives me of the freedom I enjoy using APL and LaTeX. Bernhard Strohmeier and I struggled hard to force TeX to deliver a format resembling that of *Vector's* standard and I felt a little bit let down on the part of *Vector* especially since articles in LaTeX had earlier been accepted without further ado and other formats have also been accepted. So I want to know what is your attitude concerning the layout of the articles. Is it *laissez faire* or is it the strict Word dogma? If it is neither some guidelines would be very welcome.

Ed/Production: Obviously we want to get Vector looking as good as possible! It would be helpful to know what Dietrich finds wrong with the Microsoft Equation Editor format; to our eyes there is little to choose between the formulae in his article (Vector 12.2 page 87) and Joseph de Kerf's which follows it on page 95.

The text of his article was received camera-ready and fitted to page as well as we could. In the first draft, all the font sizes were perfect, but in the copy as printed the body text was too small – the subheads and APL are correct. Unfortunately we did not have time to ask for another run as we were (as usual) running to very tight schedules.

In general we very much prefer to have editable copy, but we have no problems accepting mathematical content as .EPS files for inclusion. APL code can be accepted in most known workspace formats, and again we much prefer to have 'soft copy', particularly as most of the material will soon be made available on the BAA Web site for download.

Advanced Windows Programming with APL and Delphi

by Eric Lescasse (70731.3233@compuserve.com)

Introduction

Delphi is a Rapid (Windows) Application Development system developed by Borland. Its development environment is similar to Visual Basic. It lets you develop Windows applications with a minimum of code and headaches and produces fast compiled executables. Delphi exists in two flavours: Delphi 1, the 16-bit version of Delphi and Delphi 2, the 32-bit version of Delphi.

In this article we will use 32-bit products: Delphi 2, Dyalog APL/W 8 and APL+Win 1.8 running under Windows 95 (or NT). Everything which is presented in this article could be applied with little change to Delphi 1, Dyalog APL/W 7.2 and APL+Win running under Windows 3.1.

Delphi 2's main characteristics are:

- a true fast optimising native code 32-bit compiler
- compiled applications up to 50 times faster than PCode applications
- extremely powerful visual development environment
- self-contained compiled applications in one .EXE file
- ability to easily develop true Windows 32-bit DLLs
- complete support for Windows 95 and NT
- very powerful and easy to use multimedia capabilities
- complete object-oriented system
- OCX and OLE2 support
- Delphi 1 included in the Delphi 2 package
- executables with no royalties

Delphi can be of great help to the APL developer and we will show a few examples in this article. Here are a few things you can do with Delphi:

- build forms with the powerful Delphi form editor and auto-generate APL programs which recreate a functional and identical form. To do so, you would have to write a "Delphi Forms Translator" APL program to convert your Delphi forms into APL functions, as I did.
- write non-GUI Delphi DLLs and call them directly from APL using `⎕NA`
- write GUI Delphi DLLs and call them directly from APL using `⎕NA`
- solve some problems more easily with Delphi than with APL
- easily write multimedia applications
- add Web-surfing capabilities to your applications
- easily write forms (with no code) to access almost any database
- write starter Delphi applications for APL applications
- call Delphi executables directly from APL

Perhaps the most interesting and useful characteristic of Delphi for an APL developer is its ability to write 32-bit DLLs so easily. This article will concentrate on how to develop such DLLs and how to use them from APL.

However, Delphi has many other usages to complement APL, as listed above, that should not be overlooked, most of which are described in detail in my book *"Advanced Windows Programming with APL and Delphi"*.

Finally, we will only study one side of the APL to Delphi interface, i.e. the one where you develop in APL and call Delphi. Another interesting topic would be to develop an application in Delphi and call APL when necessary (mostly in Delphi event handler functions), but we will not talk about that here.

To make this article more readable, we will be using the following standard Dyalog APL/W and APL+Win icons as paragraph titles when showing APL code and examples. Moreover, we will use a different APL font to display examples written in APL+Win and in Dyalog APL/W.

Examples:



APL+Win

```
▼ A-CreateDirectory A;CreateDirectory
```

```
[1] 'DLL UTILSDLL.CreateDirectory(*C1,*I4*)'⎕na'CreateDirectory'
```

```
[2] A+CreateDirectory A 0
```

```
v
```



Dyalog APL/W

```
v A+CreateDirectory A;CreateDirectory
```

```
[1] 0NA'C:\800X\UTILSDLL.C32|CreateDirectory <0T =I1'
```

```
[2] A+CreateDirectory A 0
```

```
v
```

Delphi examples will be shown in a fixed Courier font. Example:

```
type
  pInt=^Integer;

procedure IntegerArg(X:pInt);stdcall;
begin
  X^:=X^+100;
end;
```

This article assumes that you have a basic knowledge of the Delphi programming environment, some notions of Pascal and that you understand how to use `0NA` to call DLL functions from APL. If you don't however, I guess you will still be able to follow me.

Designing your forms in Delphi

Delphi contains a wonderful forms editor, with all the bells and whistles you need to develop perfect forms most quickly.

You can:

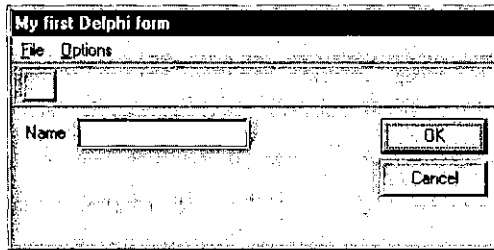
- select multiple controls and move them altogether with the mouse
- select multiple controls and move them pixel by pixel, with cursor keys, without risking a change to their vertical or horizontal positions
- use Delphi's Alignment Palette to perfectly align controls, evenly space them on the form, centre them in the form, ...
- copy/paste controls retaining all of the properties of the copied controls

All these features are missing from Dyalog APL/W *WDESIGN* workspace and from the APL+Win *WJED* editor and they are very important to the form's

designer. Without them, you both lose a lot of time when designing forms and almost cannot do a perfect job as far as controls alignment, centering, etc. is concerned.

Therefore the idea quickly comes to write an APL program which takes a Delphi form and translates it into an APL form. Although not obvious, this is quite feasible. I have developed such a tool called "*Delphi Forms Translator*". It would take too long to describe it here in detail, but a simple example will show the results.

Here is a form designed with Delphi:



Now run the Delphi Forms Translator (utility ParseDFM) in APL+Win, for example:

```
ParseDFM'fmTest'
```

```
fmTest_Make
```

The ParseDFM utility generates an APL program called fmTest_Make which is reproduced below:

```
v fmTest_Make;Dwself
[1]  APLfmTest_Make -- Created 6/01/96 at 10:57:36
[2]  'fmTest'Dwi'Delete'
[3]
[4]  Dwself+'fmTest'Dwi'New' 'Form' 'Close'
[5]  Dwself+'fmTest'Dwi'New' 'Form' 'Close'
[6]  Dwself+'fmTest'Dwi'New' 'Form' 'Close'
[7]  Dwself+'fmTest'Dwi'New' 'Form' 'Close'
[8]  Dwself+'fmTest'Dwi'New' 'Form' 'Close'
[9]
[10] Dwself+'fmTest.1Label1'Dwi'New' 'Label'
[11] Dwself+'fmTest.1Label1'Dwi'New' 'Label'
```

```
[12] Qwi'caption' 'Name'
[13]
[14] Qwself+ 'fmTest.bnButton1'Qwi'New' 'Button'
[15] Qwi'where' 38 261 25 75
[16] Qwi'caption' 'OK'
[17]
[18] Qwself+ 'fmTest.bnButton2'Qwi'New' 'Button'
[19] Qwi'where' 68 261 25 75
[20] Qwi'caption' 'Cancel'
[21]
[22] Qwself+ 'fmTest.edEdit1'Qwi'New' 'Edit'
[23] Qwi'where' 37 45 23 123
[24]
[25] Qwself+ 'fmTest.tbPanel1'Qwi'New' 'Frame'
[26] Qwi'caption' ''
[27] Qwi'scale' 5
[28] Qwi'style' 3
[29] Qwi'where' 0 0 30 343
[30]
[31] Qwself+ 'fmTest.tbPanel1.bnButton3'Qwi'New' 'Button'
[32] Qwi'caption' ''
[33] Qwi'where' 4 7 22 24
[34]
[35] Qwself+ 'fmTest.mFile1'Qwi'New' 'Menu'
[36] Qwi'onClick' 'Test_mFile1_Click'
[37] Qwi'caption' '&File'
[38]
[39] Qwself+ 'fmTest.mFile1.mOpen1'Qwi'New' 'Menu'
[40] Qwi'onClick' 'Test_mFile1_mOpen1_Click'
[41] Qwi'shortcut' 79 2
[42] Qwi'caption' ('&Open',Qtcht,'Ctrl+O')
[43]
[44] Qwself+ 'fmTest.mFile1.mN1'Qwi'New' 'Menu'
[45] Qwi'separator' 1
[46]
[47] Qwself+ 'fmTest.mFile1.mExit1'Qwi'New' 'Menu'
[48] Qwi'onClick' 'Test_mFile1_mExit1_Click'
[49] Qwi'shortcut' 115 4
[50] Qwi'caption' ('Exit',Qtcht,'Alt+F4')
[51]
[52] Qwself+ 'fmTest.mOptions1'Qwi'New' 'Menu'
```

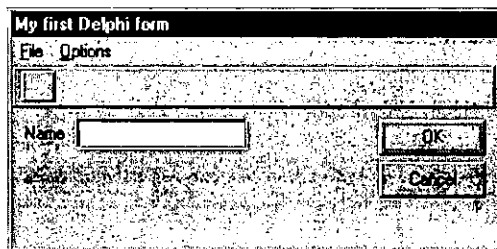
```

[53] Dw1'onClick' 'Test_mOptions1_Click'
[54] Dw1'caption' '&Options'
[55]
[56] Dwself+'fmTest.mOptions1.msortByDate1'Dw1'New' 'Menu'
[57] Dw1'onClick' 'Test_mOptions1_msortByDate1_Click'
[58] Dw1'caption' 'sort by &Date'
[59]
[60] Dwself+'fmTest.mOptions1.msortBySize1'Dw1'New' 'Menu'
[61] Dw1'onClick' 'Test_mOptions1_msortBySize1_Click'
[62] Dw1'enabled' 0
[63] Dw1'caption' 'sort by &Size'
[64]
[65] Dwself+'fmTest.mOptions1.msortByExtension1'Dw1'New' 'Menu'
[66] Dw1'onClick' 'Test_mOptions1_msortByExtension1_Click'
[67] Dw1'value' 1
[68] Dw1'caption' 'sort by &Extension'
[69]
[70] 0 Op'fmTest'Dw1'Wait'

```

▼

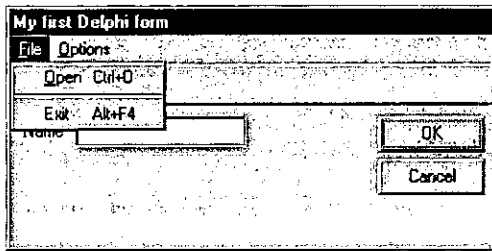
If we run fmTest_Make, here is the form we get in APL+Win:



The *ParseDFM* utility is also available for Dyalog APL/W 7.2 and 8 and would create an APL function that would build an identical form.

Using Delphi to design a form is a great time saver when you have a lot of controls in the form, or when the form has complex menus. The Delphi menu editor is exceptionally good and easy to use.

The Delphi Forms Translator has also converted the menus, accelerator keys, etc.:

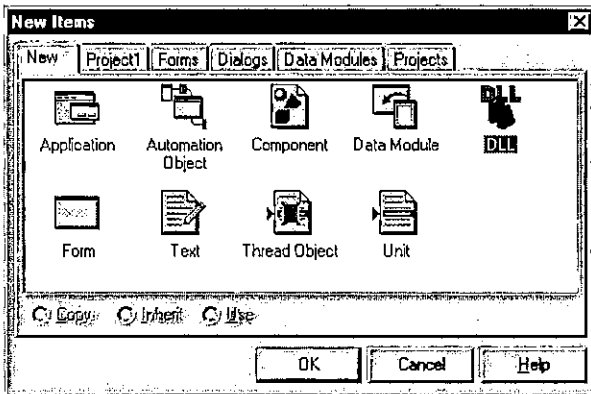


Writing 32-bit DLLs with Delphi 2

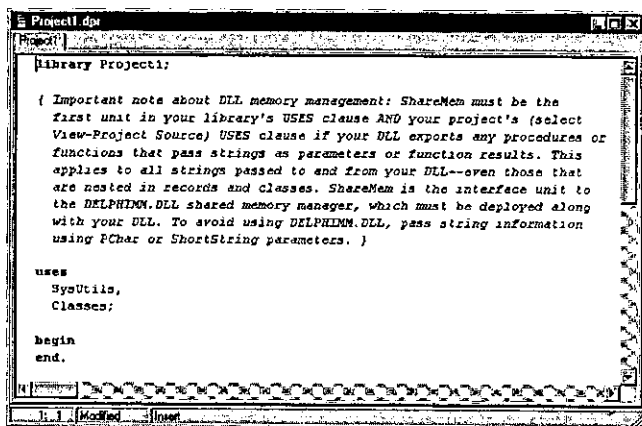
A Delphi DLL is just a Delphi program, except that it should start with the word **library** instead of the word **program**.

To create a DLL in Delphi 2, use the following simple steps:

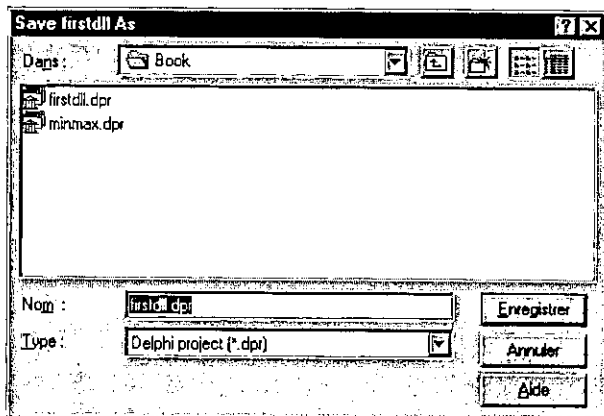
1. select **New...** from the **File** menu — the following dialog box shows up:



2. select the **DLL** icon and click **OK** or **Enter**. The editor informs you that you must install **ShareMem** in the **Uses** clause of your DLL and must use **DELPHIMM.DLL** if you want to pass strings as parameters or function results. This is not necessary if your strings are declared as **PChar** or **ShortString**. So we will use **PChar** arguments to pass character strings back and forth between APL and Delphi 2 and we can therefore forget about this comment.



3. save your DLL project by selecting **Save Project As...** from the Delphi File menu, and enter **firstdll.dpr** as the DLL project name.



Fine. You have saved your first DLL project. We now need to write the DLL.

Developing the DLL and calling it from APL

We may install either Object Pascal functions or procedures in a DLL. To add a function or procedure to the DLL, simply add the function code below the uses clause and add a reference to this function in an exports clause at the end of the DLL as shown in the source code below.

It is advisable that you always use procedures rather than functions in your DLLs.

First, we will remove the warning comment displayed at the top of the DLL. Then we will add 3 procedures called **IntegerArg**, **DoubleArg** and **CharString** which will respectively accept a Delphi **Integer** and a Delphi **Double** (floating point) and a **PChar** (=pointer to a null terminated string) as arguments and return different values to APL.

```
library firstdll;

uses
  SysUtils,
  Classes;

type
  pInt=^LongInt;
procedure IntegerArg(X:pInt); stdcall;
begin
  X^:=X^+100;
end;

type
  pDbl=^Double;
procedure DoubleArg(X:pDbl); stdcall;
begin
  X^:=X^+100.1;
end;

procedure CharString(X:PChar);stdcall;
begin
  X:=StrUpper(X);
end;

exports
  IntegerArg index 1,
  DoubleArg index 2,
  CharString index 3;

begin
end.
```

Note that we have declared 2 new types: **pInt** which is a pointer to a **LongInt** and **pDbl** which is a pointer to a **Double**. We need to do that because we want our procedures to modify their arguments and return a different value, through them, to APL. The **^** notation in Delphi is used to represent pointers. **PChar** are themselves pointers to null terminated strings, so we do not need to declare a new pointer type there.

Note also the **exports** clause at the end of the DLL: it is essential to make your routines visible from the outside world (namely APL here).

Now, choose **Project/Compile** or press **Ctrl+F9** to compile your DLL. If you have not mis-typed anything, it should compile OK.

Now that we have done the Delphi part, we need to perform the APL part of the work. If you have never seen or used **ANA** yet, please read the few documentation

pages in your APL manuals that describe its syntax: `QNA` lets you call a DLL function from a 16-bit or 32-bit DLL, pass to it arguments from APL and receive results back in APL. One of the key points to understand is that you always need to use **pointers** when you want your DLL procedure to return a value to APL different from the one you passed from APL.

Once a function is successfully associated through `QNA`, it appears as a locked function in your workspace and can be used like any other APL function.



Dyalog APL/W

We need to issue the following `QNA` statement in order to associate our 3 procedures:

```
QNA'C:\BOOK\FIRSTDLL.DLL.C32|IntegerArg =I4'  
QNA'C:\BOOK\FIRSTDLL.DLL.C32|DoubleArg =F8'  
QNA'C:\BOOK\FIRSTDLL.DLL.C32|CharString =0T'
```

A Delphi procedure does not return any direct result; in Dyalog APL/W, if you prefix a **procedure** argument with the `<`, `>` or `=` symbols, it means the following:

Prefix	Meaning for the argument
<	Input array or pointer to an input data is passed from APL to the DLL function/procedure as an array or pointer
>	Output array or pointer to an output data is passed from the DLL function/procedure back to APL as an array or pointer
=	Input/output array or pointer data is both passed from APL to the DLL function/procedure AND/OR returned from it as an array or pointer

Here, for example, we have used an `=F8` parameter code to indicate that we pass an eight-byte floating point value to the Delphi procedure AND that it should also return to us an eight-byte floating point value through this `F8` parameter.

For character strings, we must use the `=0T` code to denote a null-terminated string argument.

Let's try our new associated functions:

```
DoubleArg 3.14159
103.24159
```

```
CharString <'eric'
ERIC
```



APL+Win

We need to issue the following *DNA* statement in order to associate our 3 DLL procedures in APL+Win:

```
'DLL FIRSTDLL.IntegerArg(*I4+)' Dna 'IntegerArg'
'DLL FIRSTDLL.DoubleArg(*F8+)' Dna 'DoubleArg'
'DLL >FIRSTDLL.CharString(<C1+)' Dna 'CharString'
```

In APL+Win, you need to prefix the parameter code with *** to indicate you are using a pointer and suffix it with *+* to indicate that the DLL routine changes the parameter value and returns a different one to APL.

Therefore we use an **F8+* parameter code to indicate that we pass an eight-byte floating point value to the Delphi procedure AND that it should also return to us an eight-byte floating point value. Let's try our new associated function:

```
DoubleArg 3.14159
103.24159
```

It works fine.

For character strings, we must use the *C1* code in APL+Win.

```
'DLL >FIRSTDLL.CharString(<C1+)' Dna
'CharString'
```

You must call the associated APL function the following way:

```
CharString 'eric'
ERIC
```

Note that we are using the *<* additional prefix for the DLL routine parameter to indicate that it must be enclosed, and the *>* result prefix indicating that the result

must be disclosed. If we had not used $\Leftarrow$ and $\Rightarrow$ we would have had to make the following call from APL:

```
 $\Rightarrow$  CharString  $\Leftarrow$  'eric'
```

or:

```
 $\uparrow$  CharString  $\Leftarrow$  'eric'
```

First Example: Creating a Directory

Dyalog APL/W does not include any system function to create a new directory on your disk and if you need to do so in an application, you have a problem. Let's write a very simple Delphi DLL to solve the problem.

Here is the source code of the Delphi DLL:

```
library utilsdll;

uses
  SysUtils,
  Classes;

type
  pBool = ^Boolean;
procedure CreateDirectory(dir: PChar; b: pBool); stdcall;
begin
  b^ := CreateDir(dir);
end;

exports
  CreateDirectory index 1;

begin
end.
```

This example is pretty simple: we are just using the Delphi CreateDir function which returns True (i.e. 1) if the directory was successfully created and False (i.e. 0) if the directory could not be created.



Dyalog APL/W

Here is a sample function we could use:

```
▽ A←CreateDirectory A;CreateDirectory
[1]   ⍳NA'C:\BOOK\UTILSDLL.C32|CreateDirectory <0T =I1'
[2]   A←CreateDirectory A 0
▽
```

Note that the function returns a single boolean value (pointed by the `=I1` pointer); the first argument is only passed as an input pointer (using the `<` notation).



APL+Win

```

▽ A←CreateDirectory A;CreateDirectory
{1} 'DLL UTILSDLL.CreateDirectory(*C1,*I4+)'⌈na'CreateDirectory'
{2} A←CreateDirectory A 0
▽

```

Call this function the following way:

```

CreateDirectory 'C:\TEST',⌈tcnul

1

CreateDirectory 'C:\TEST',⌈tcnul

0

```

A return code of 1 indicates success: the `CreateDirectory` associated function fails the second time because the `C:\TEST` directory already exists.

Of course, in `APL+Win` you could have used the `⌈mkdir` system function to get the same result.

Including a Delphi form in a DLL

It is almost as simple to include a Delphi form in a DLL.

We will simply take an example. Let's assume that you need a form to select a directory from your hard disk. You do not want to use the standard Windows File Open box because you are not interested in selecting a file, but simply in selecting a directory.

To solve this problem you could select one of 3 techniques:

- try to create your own APL form, but that would be pretty hard since you would have to deal with such things as find existing drives, find directory hierarchy, use owner draw list boxes and combo boxes to display nice bitmaps to the left of drives and directories, ... a task clearly out of reach of most APL users.

- try to “customize” the standard Windows File Open box by using a hook function: this is not easy programming either and requires quite a number of low level calls plus the ability to receive low level callbacks from Windows which only APL+Win permits. APL2000 has solved this problem with APL+Win and, with their permission, I have included hereafter the code of the functions they wrote, because these functions are extremely interesting to study and teach a lot about Windows programming. You can also download these functions from the APL 2000 Web site.

<http://members.aol.com/apl2000>

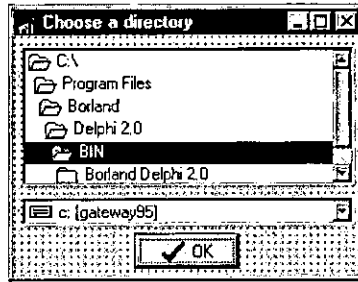
selecting **APL+World** on their page, then **Demonstration APL+Win workspaces**, then selecting **GETDIR.W3** to download.

- write a simple Delphi DLL with a simple Delphi form.

Let's write a simple Delphi DLL to solve this problem.

1. select **File/New...** and double click the **DLL** icon
2. remove the long comment that shows up in the code
3. select **File/New Form** (this creates a new unit: **Unit1**)
4. click the **Save All** toolbar button and save your project as **SelDir.pas** and **SelDir.dpr**
5. change the form's **Caption** property to **Choose a directory**
6. click the **System** tab on the component palette
7. double click the **DriveComboBox** object to drop it on the form
8. double click the **DirectoryListBox** object to drop it on the form
9. click the **Standard** tab on the component palette
10. double click the **BitBtn** object to drop it on the form
11. change its **Kind** property to **bkOK**
12. click once on the **DriveComboBox1** object in your form to select it
13. change its **DirList** property to **DirectoryListBox1**

Your Delphi form should look like the following, in Delphi design mode:



Most of the code you need has been written for you by Delphi. In the source code reproduced below, I used a grey background to show lines which you have to write yourself.

Here is the source code of your DLL project:

```
library seldir;

uses
  SysUtils,
  Classes,
  seldir_ in 'seldir_.pas' {Form1};

exports
  ChooseDir index 1;

begin
end.
```

and here is the code handling the form:

```
unit seldir_;

interface

uses
  Windows, Messages, SysUtils, Classes, Graphics, Controls,
  Forms, Dialogs, StdCtrls, Buttons, FileCtrl;

type
  TForm1 = class(TForm)
    DriveComboBox1: TDriveComboBox;
    DirectoryListBox1: TDirectoryListBox;
    BitBtn1: TBitBtn;
  private
    { Private declarations }
  public
    { Public declarations }
  end;

end;
```

```

var
  Form1: TForm1;

procedure ChooseDir(s:pChar);stdcall;export;

implementation

{$R *.DPM}

procedure ChooseDir(s:pChar);stdcall;export;
begin
  Form1:=TForm1.Create(Application);
  try
    Form1.ShowModal;
    StrPCopy(s,Form1.DirectoryListBox1.Directory);
  finally
    Form1.Free;
  end;
end;
end.

```

Whenever you want to add a form to a Delphi DLL, you must write code similar to that reproduced below. Your code should include the following lines:

```

Form1:=TForm1.Create(Application);
try
  Form1.ShowModal;
finally
  Form1.Free;
end;

```

where you first dynamically create an instance of your form (thus allocating memory for it), then use the Delphi error handling structure **try...finally...end** which ensures that in case of error in the **try** section, the **finally** section is always executed, thus being sure to release the dynamically allocated memory.

The form is displayed by invoking its **ShowModal** method (same as **APL+Win Wait** method or same as using **DDQ** in Dyalog APL/W).

When the user closes the form, the **Directory** property of the **DirectoryListBox1** object is read. We need to use Delphi **StrPCopy** function to copy the **Directory** property which is a **String** to variable **s** which is declared as a **PChar** (=null terminated string).

Now we need to call the **ChooseDir** function from APL.



Dyalog APL/W

Write a simple Dyalog APL/W function similar to the following one:

```

▽ A+ChooseDir;ChooseDir
[1]  Dna'c:\temp\selDir.032 |ChooseDir =0?'
[2]  A+ChooseDir C256p' '
▽

```



APL+Win

```

'c'\Dchdir'c:\temp'
C:\temp

```

We first set the current directory to the one containing our new compiled DLL, so that APL finds it. Then we write the following simple APL function:

```

▽ A+ChooseDir;ChooseDir
[1]  A+'DLL' >SELDIR.ChooseDir (C*Cl+)'Dna'ChooseDir'
[2]  A+ChooseDir 256p' '
▽

```

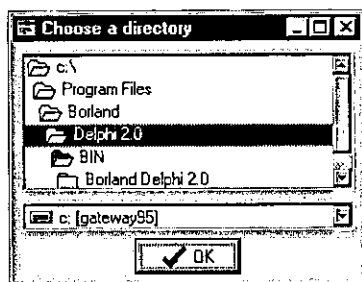
and finally we call it.

```

ChooseDir
c:\Program Files\Borland\Delphi 2.0\BIN

```

The form which is displayed is the following.



As you have noticed, Delphi not only helped solve our problem but also allowed us to return directories with long file names and got a bitmap button as a little bonus!

The APL 2000 GetDir solution to selecting a directory

I am reproducing below the 2 functions that you may find in the GetDir.W3 workspace downloadable from APL2000's Web page.

The GetDir workspace demonstrates APL+Win capabilities to tap into Windows low level API calls and callbacks when necessary. They show exactly what a C programmer must do to perform the same action.

Basically the idea is to intercept and handle a message which occurs when the File Open common dialog is about to be displayed on the screen.

This message is called WM_INITDIALOG in Windows.

The structure of the GetDir function is basically the same as the WGetOpenFileName function distributed in the APL+Win Windows.W3 workspace.

The only important differences are lines 24, 26, 41, 49 and 51.

On line 24 a new pattern is added to the APLW.INI file, using the W_Ini internal utility, in order to add a WM_INITDIALOG_GetDir message to the system. In fact, the WM_INITDIALOG_GetDir message is defined using the Windows constant WM_INITDIALOG as the message type, so that WM_INITDIALOG_GetDir be recognized as a kind of a WM_INITDIALOG message.

Then, on line 26, the function creates a filter for the WM_INITDIALOG_GetDir message, so that the APL instruction: GetDir_Init ⍵warg will be executed when GetOpenFileName is called, just prior to displaying the File Open common dialog. The value found in ⍵warg will be the WM_INITDIALOG_GetDir message, i.e. the File Open common dialog handle (as a matter of fact, because of the ~ symbols in the message declaration, only the first message parameter, which is the dialog handle, is passed to APL in ⍵warg).

In Windows terminology, the GetDir_Init function is called a hook function.

Line 41 includes the pointer to the hook function in the OPENFILENAME data structure which is then passed as an argument to GetOpenFileName.

Calling GetOpenFileName generates a WM_INITDIALOG_GetDir message which in turn fires the GetDir_Init APL hook function.

Now, this function does all the job of customizing the File Open dialog box just before it gets displayed (that hardly takes a few milliseconds, so all this process is absolutely transparent to the user).

The GetDir_Init function basically runs two loops. The first one is used to hide all of the File Open common dialog controls which are not useful to select a

directory, i.e. all those which are on the left of the dialog box (the file name edit control, the file names list box, ...).

The second loop is used to move all the remaining controls by the same offset to the left of the dialog box, retaining their vertical positions.

Finally, the dialog box is resized to avoid displaying it with a large empty space on the right of the moved controls.

```

▼ R=Title GetDir Dir;File;OFN;Title;Data;pData;hF;lpfnHook;nMaxFile
[1]  A Use common dialog box to choose a directory
[2]  A Dir= initial directory
[3]  A Title= caption of dialog box
[4]  A R= new directory ('' if cancel or error)
[5]  A Copyright 1996 APL2000, Inc.
[6]
[7]  R='' 0 nMaxFile-256 A size of Dir and File
[8]
[9]  File='APL2000.RAD' A any name is fine, just a place holder
[10] File=File, (nMaxFile-pFile)@UTCNUL A pad with nulls
[11]
[12] Dir=(-('\'-1:Dir)^(3<pDir)):Dir A remove trail "\" if not root
[13] Dir=Dir, (nMaxFile-pDir)@UTCNUL A pad with nulls
[14]
[15] :if 0=Onc 'Title' 0 Title='Choose Directory' 0 :endif A default caption
[16] Title=Title,UTCNUL A null terminated
[17]
[18] Data=File,Dir,Title A build composite data
[19] -(0=hF=Dwcall 'GlobalAlloc' 'GMEM_MOVEABLE' (@pData))/Aexit A allocate mem
[20] -(0=pData=Dwcall 'GlobalLock' hF)/Aexit A get ptr to memory
[21] Data W_Mem pData A write data to memory
[22]
[23] A define pattern (only want init message, Dwarg=handle of dialog box)
[24] 0 0@Dwcall 'W_Ini' '[Pattern]WM_INITDIALOG_GetDir=B=0(H,~U=WM_INITDIALOG,
~U,~L)'
[25] A get pointer to hook function
[26] lpfnHook=Dwcall 'W_CreateFilter' ('WM_INITDIALOG_GetDir' 'GetDir_Init Dwa
rg')
[27]
[28] A build OPENFILENAME data structure (see Win32 SDK for documentation)
[29] OFN=D2C 76 A lStructSize - length of the structure
[30] OFN=OFN,D2C 0 0 A hwndOwner, hInstance
[31] OFN=OFN,D2C 0 0 A lpstrFilter, lpstrCustomFilter
[32] OFN=OFN,D2C 0 0 A nMaxCustFilter, nFilterIndex
[33] OFN=OFN,D2C pData+0 A lpstrFile (first piece of Data)
[34] OFN=OFN,D2C nMaxFile A max size of file name
[35] OFN=OFN,D2C 0 0 A lpstrFileTitle, nMaxFileTitle
[36] OFN=OFN,D2C pData+pFile A lpstrInitialDir (second piece of Data)
[37] OFN=OFN,D2C pData+pFile,Dir A lpstrTitle (third piece of Data)
[38] OFN=OFN,D2C +/4 8 32 2048 A Flags - hide read-only, don't change dir, e
nable hook, path must exist
[39] OFN=OFN,W2C 0 0 A nFileOffset, nFileExtension
[40] OFN=OFN,D2C 0 0 A lpstrDefExt, lCustData
[41] OFN=OFN,D2C lpfnHook A pointer to hook function
[42] OFN=OFN,D2C 0 A lpTemplateName
[43]
[44] -(0=Dwcall 'GetOpenFileName' OFN)/Aexit A exit if error or cancel
[45] Data=W_Mem pData,82,pData A read the composite data
[46] Data=(Data@UTCNUL)Data A file path and name is up to first null
[47] R=(0v0Data='\'')/Data A directory is up to the last backslash
[48]
[49] Aexit: 0 0@Dwcall 'W_DestroyFilter' lpfnHook A done with hook proc
[50] 0 0@Dwcall 'GlobalUnlock' hF 0 0 0@Dwcall 'GlobalFree' hF A free memory
[51] 0 0@Dwcall 'W_Ini' '[Pattern]WM_INITDIALOG_GetDir' A remove from INI
▼
▼ GetDir_Init hDlg;col;dlgsize;h;id;offset;row
[1] A hook function on init of OpenFileName for directory selection
[2] A Copyright 1996 APL2000, Inc.
[3]
[4] A control location is obtained relative client area of dialog

```

```

[5]  A but are set relative to the upper left of screen
[6]  dlgsize=323 Odr Dwcall 'GetWindowRect' hDlg (16pDtcnul) A col,row of wind ow
[7]  (col row)+2*dlgsize 0 dlgsize-(2*dlgsize)-(2*dlgsize)
[8]  col=col+Dwcall 'GetSystemMetrics' 'SM_CXDLGFRAME' A border width
[9]  row=row+Dwcall 'GetSystemMetrics' 'SM_CYDLGFRAME' A border height
[10] row=row+Dwcall 'GetSystemMetrics' 'SM_CYCAPTION' A caption height
[11] offset=col,row
[12]
[13] A hide controls in the first column
[14] A lFnm edFnm lsFiles lType cbType
[15] :for id :in 1090 1152 1120 1083 1136
[16] h=Dwcall 'GetDlgItem' hDlg id
[17] :if id=1090 A first time thru
[18] col=1323 Odr Dwcall 'GetWindowRect' h (16pDtcnul) A start col
[19] offset =offset -col,0
[20] dlgsize=dlgsize+col,0
[21] :endif
[22] 0 0pDwcall 'ShowWindow' h 'SW_HIDE'
[23] :endfor
[24]
[25] A move the controls in the second and third columns
[26] A lDir lCurDir lsDirs lDrv cbDrv lDOK lDCANCEL
[27] :for id :in 55535 1088 1121 1091 1137 1 2
[28] h=Dwcall 'GetDlgItem' hDlg id
[29] (col row)+2*323 Odr Dwcall 'GetWindowRect' h (16pDtcnul)
[30] :if id=55535 A first time thru
[31] offset =offset +col,0
[32] dlgsize=dlgsize-col,0
[33] :endif
[34] (col row)+(col,row)-offset
[35] 0 0pDwcall 'SetWindowPos' h 0 col row 0 0 'SWP_NOSIZE SWP_NOZORDER'
[36] :endfor
[37]
[38] A now resize the dialog box
[39] (col row)=dlgsize
[40] 0 0pDwcall 'SetWindowPos' hDlg 0 0 0 col row 'SWP_NOMOVE SWP_NOZORDER'

```

The technique shown here of using hook functions is the same you should use if you simply wanted to display a common Windows dialog box at a specific location on your screen, something otherwise impossible and which often bothers Windows programmers. I am sure that, from the examples given above, you can easily derive, as I could do now, the right hook function to do this if you need it.

Conclusion

This article is just a very simple introduction to interfacing APL and Delphi.

It shows a couple of simple examples proving that Delphi can be quite useful to the APL developer, particularly when you write Delphi 32-bit DLLs and use them from APL.

However there are many other tasks that you could easily perform with Delphi and which would perfectly solve some otherwise difficult problems.

Here are a few:

- a starter application, displaying a nice logo form immediately while APL and your application are loading and leaving this logo form displayed until your application's main form gets displayed

- create functional dialog boxes in Delphi and use them from APL
- write simple database forms or even more complex master/detail forms with lookup fields in order to browse/update traditional relational databases (believe it or not, you can do this with almost no code, in minutes, then install these forms in a DLL and use them from APL)
- call all common Windows dialog boxes easily
- create resources DLLs where you save most of your application resources (bitmaps, icons, cursors, strings, ...) thus reducing the amount of files delivered, speeding up application initialisation, ...
- extremely easily adding multimedia capabilities to your APL applications (videos, sounds including .MID, ...)
- even, surfing the Web from your APL application

What is especially nice is that most of these things are easy to do with Delphi, but, in order to fully exploit using Delphi from APL you will need to learn how to pass all kinds of data back and forth between APL and Delphi and you need to learn a bit about Delphi and Object Pascal.

I have spent quite some time exploring all this and ended up writing a 200 pages book titled *"Advanced Windows Programming with APL and Delphi"* which describes all types of arguments/results passing between APL and Delphi, and shows most of the above listed Delphi techniques and how to directly use them from APL.

This article is adapted from this book which is available from Uniware (CompuServe 70731,3233). The book includes a disk with all of the book examples' source code in Delphi 2, Dyalog APL/W 8 and APL+Win 1.8.

If you attend APL96 you may also want to attend my talk on the subject.

APL Operators and J

by Norman Thomson

J merits the attention of all serious APL programmers, because of the way in which it has brought the nature of APL operators into sharper focus.

Roughly speaking "function" in APL corresponds to "verb" in J. However J splits the concept of an APL operator in two, dividing it between that of "adverb", (strictly monadic), and "conjunction" (strictly dyadic). Conjunctions in J subdivide further into the types noun-noun, noun-verb and verb-verb, so that in total there are four identifiably different types of APL operators. The insight thus gained can have a profound effect on APL programming. For conciseness I shall call the resulting style "operator programming", and I claim that it can result in a great increase in generality of code produced.

Before considering the four classifications separately, I will illustrate what I mean by "operator programming", using what must be one of the first functions which most APL programmers write, namely *mean*:

```
[0] Z←mean R
[1] Z←(+/R)÷ρR

mean 1 4 5 6
4
```

Taking an operator view, and with a fork in J in mind, rewrite this by placing the $\div$ at the left hand end of the expression, where it comes as a climax following two transformations on R, the first ρ and the second $+/$. Connecting these two functions with an operator *AND* which applies both its function operands to the data:

```
[0] Z←L(P AND Q)R
[1] →(0=⊢NC 'L')/L1
[2] →0 Z←(L P R)(L Q R)
[3] L1:Z←(P R)(Q R)
```

leads to a revised definition of mean:

```
[0] Z←mean1 R
[1] Z←÷/+/ANDρR

      mean1 1 4 5 6
4
```

(The astute reader will have noticed the blank on the left which means that the result of *mean1* has depth two; this is a programming technicality which is readily dealt with in a production context.)

It is well known that the arithmetic mean is not the only form of average — indeed this is often the lead-in to the APL novice's second exercise which includes geometric and harmonic mean. Put in another way, there is a requirement to average values of other functions of the data, which in turn is an invitation to generalise *mean1* to an operator:

```
[0] Z←(P MEAN1)R
[1] Z←÷/+/ANDρP R

      +MEAN1 1 4 5 6
4
```

The harmonic mean is

```
      ÷÷MEAN1 1 4 5 6
2.4742
```

and the geometric mean is

```
      *⊙MEAN1 1 4 5 6
3.3098
```

The inverse function which is the leftmost token in each of the last two expressions is, to say the least, tiresome. Remember that a second operand is still available, so re-define *MEAN1* yet again:

```
[0] Z←(P MEAN Q)R
[1] Z←P÷/+/ANDρQ R

      +MEAN+ 1 4 5 6
4
```

leading to the following expressions for harmonic and geometric means:

```
      ÷MEAN÷ 1 4 5 6
      *MEAN⊙ 1 4 5 6
```

To program the absolute deviation from the mean, start with a function:

```
[0] Z←adev R
[1] Z←ε |R-+MEAN+R
```

which, used in conjunction with *MEAN*, gives the absolute deviation as:

```
+MEAN adev 1 4 5 6
1.5
```

Now stop and reflect — absolute deviation is only one form of deviation, another form is the squared deviation, whose average is the variance, so again “think operator”:

```
[0] Z←(P DEV)R
[1] Z←ε P R-+MEAN+R

[0] Z←sq R
[1] Z←R*2

+MEAN |DEV 1 4 5 6
1.5
+MEAN sq DEV 1 4 5 6
3.5
```

Next the function *sq* is itself just a particular case of something more general, so applying the same psychology define:

```
[0] Z←(P EXP)R
[1] Z←R*P

+MEAN(2 EXP)DEV 1 4 5 6
3.5
+MEAN(3 EXP)DEV 1 4 5 6
-4.5
```

the latter expression, for example, giving the third moment about the mean.

To sum up the essence of “operator programming” in a phrase — why write one function when you can write several for the same effort!

The next section describes instances of each of the four types of operator identified at the start.

1. Operators used adverbially

Primitive instances are reduction and outer product, (but NOT inner product which is a case of verb-verb). The discussion above includes a simple adverbial operator *EXP*. You may be tempted to think that there is little apparent difference between $2 \text{ EXP } 3$ and $2 * 3$. On the contrary, there is a very great deal. For one thing $(2 *) 3$ is a syntax error, whereas $(2 \text{ EXP}) 3$ is not only equivalent to $2 \text{ EXP } 3$, but also demonstrates an underlying device which, in computer science, is called "currying" after the American logician H.B.Curry. Roughly speaking, it involves replacing an operation with two arguments with an operation having only one, by binding one of the operands to the operation. Thus 2 EXP is a composite function which can be applied to each of $1 \ 2 \ 3$ to obtain $2 \ 4 \ 8$.

Another common example of currying is

```
[0] Z←(P LOG)R
[1] Z←P*R
```

```
10 LOG 2
0.30103
```

DEV above is a further example of an adverbial operator. It modifies its function operand by adverbially intruding "having subtract the mean" between the function operand and its argument.

2. Operations of type noun-verb

A simple example of this type of operator usage is *RPT1* which is designed to repeat the execution of a function a given number of times:

```
[0] Z←L(P RPT1)R;I
[1] I←0 ◊ Z←L
[2] L1:→(R≤I+I+1)/0
[3] →L1 Z←P Z
```

```
2(2 EXP)RPT1 5
65536
```

that is the fifth term of the series $2, 4, \dots$ where each term is the square of the preceding term.

A variation on this theme is the operator *RPT2* which executes a function repeatedly until two successive iterates are sufficiently close together as defined by a tolerance operand *Q*:

```

[0] Z←L(P RPT2 Q)R
[1] Z←L P R
[2] L1:→(Q>|Z-R)/0
[3] →L1 Z←L P R+Z

```

For example, a well known algorithm for finding the square root of a real number consists of iterating the following algorithmic step:

```

[0] Z←L sqrtstep R
[1] Z←0.5×R+L÷R

```

until an adequate approximation is obtained:

```

9 sqrtstep RPT2 0.001 1
3

```

Another example where an operand is a limit value is the operator *RPT* defined in [1].

3. Operations of type verb-verb

The discussion above contains two examples of this class, namely *AND* and *MEAN*. The principle is that the two functions concerned form a composition in a manner which is detailed within the operator definition. Another example, mirroring the adverb of the same name in J, is :

```

[0] Z←L(P WITH Q)R
[1] →(0=□NC 'L')/L1
[2] →0 Z←(Q L)P(Q R)
[3] L1:Z←(P R)(Q R)

10* 2 +WITH(10 LOG)3
6

```

which models the process of multiplication by using logarithms.

4. Operations of type noun-noun

These involve a different kind of operator usage. By using currying, they allow an extension of the number of function arguments to three or four. A simple example is the formula for compound interest, which is, in essence, $A(1+R)^N$, and involves three arguments. Currying the $(1+R)$ we have the operator

```
[0] Z+L(P CI)R
[1] Z+L*P*R

      100 (1.1 CI) 2 3
121 133.1
```

The formula for the area of a triangle, given the lengths of its sides is $\sqrt{s(s-a)(s-b)(s-c)}$ — again this has three arguments, so curry on one of them:

```
[0] Z+L(P TRIAREA)R
[1] Z+(×/(0.5×+/L,P,R)-0,L,P,R)*0.5

3(4 TRIAREA)5 6
```

Is operator programming a “better” form of programming? I believe so. The little suite of operations which were developed in the first part of this article included four operators:

AND, MEAN, DEV, EXP

and no functions.

By seeking opportunities to use operators, the APL programmer is forced to ask questions about what are (a) essential, and (b) generalisable about operations. For example, the content of the operators described above shows that:

the essence of *MEAN* is $\div$ $+/$ and ρ
 the essence of *DEV* is $\star$ and *MEAN*
 the essence of *EXP* is $\star$

and each of these operators is generalisable in ways which would not be apparent if functions alone had been employed.

The above discussion has tried to show how operators develop naturally when an APL programmer is receptive to opportunities for generalisation. Relatively simple examples were used in order that programming complexity should not interfere with the main thrust of the argument. A reader seeking an illustration of operator programming applied in greater earnest is referred to [1].

Reference

- [1] *Technical Note on Confidence Limits*, N. Thomson, The Random Vector, Volume 12, No.3, pp. 98-106, April 1996

The J DLL and Extended Integers

by Chris Burke

J DLL

With J Release 3, the J interpreter is packaged as a 32-bit DLL. Any program capable of calling a 32-bit DLL can now call a J subroutine directly.

The file J.DLL has the full J language, including external facilities such as the file system and ODBC interface, but without the Windows GUI functions and the Session Manager. The size is about 450K, compared to 950K for the full J development system. The DLL is freely available, and the current version can be downloaded from the J Web page.

The DLL is also an OLE in-process server and can be accessed via OLE.

The DLL will most likely be of interest to developers in languages such as Visual Basic, Delphi, C++ or Excel, who can now incorporate J as a calculation server in their applications. However it can also be called by Windows APL systems.

Why would anyone want to call J from APL? Well, for one thing it is a nice way to experiment with the DLL — since APL knows how to display arrays, there is no problem in displaying the results from J!

Also, J has some facilities that not are not in all APLs, such as ODBC data access support, complex numbers and extended integers. The DLL allows you to add such features to your own APL applications.

Finally, for those interested in experimenting with the use of J, or perhaps in migrating applications from APL to J, this allows you to call new functions written in J from an existing APL application.

An example using APL*PLUS III

The callable functions supported by the J DLL are a subset of those available using OLE. For the most part, you need only:

Do	execute a J sentence
GetM	get a J variable (returns the memory address)

To access the J DLL, you must first ensure the DLL is visible — the easiest way to do this is to copy the J.DLL file into your APL directory.

Next, you define *associated* functions to call J, using `⌈na`. These functions behave like user-defined, locked, monadic APL functions. For example, the following defines associated functions for Do and GetM:

```

v makejfn:r
[1]  ⌈na defines ⌈na fn:
[2]  ⌈na j_do
[3]  ⌈na j_getm
[4]
[5]  r←⌈lex 'j_do j_getm'
[6]
[7]  r←''
[8]  r←r,'DLL I4+J.JDo(*C1)' ⌈na 'j_do'
[9]  r←r,'DLL I4+J.JGetM(*C1,*I4+,*I4+,*I4+,*I4+)' ⌈na 'j_getm'
[10]
[11] ⌈error (0er)/'could not define ⌈na fn'
v

```

This function must be run first, to initialize the link with J. If it runs without error, the link is established.

Cover functions

The associated function `j_do` takes an argument of an enclosed character string terminated with an ASCII zero character, and sends it to J to be executed. The following cover function `jdo` ensures the argument is in the correct form, sends it to J, and signals any result error:

```

v r←jdo str;a0
[1]  ⌈na str is a character string
[2]
[3]  ⌈na ensure is zero terminated:
[4]  a0←0p⌈av
[5]  str←str,(a0*~1+str)/a0
[6]
[7]  :if 0=r+j_do cstr o :return o :end
[8]  ⌈error 'return code: ',r
v

```

The associated function `j_getm` takes an enclosed character string that represents a J variable name, terminated with an ASCII zero character, and sends it to J to be retrieved. The result from J is a set of pointers to the variable's type, rank, shape and data. A utility function `jfix` (not listed here) reads these pointers and creates the corresponding APL array.

The cover function `jget` is:

```

v r←jget name;a0;pdatt;psshape;rank;rc;type
[1]  A name is a character string
[2]
[3]  A ensure is zero terminated:
[4]  a0←0p⊆av
[5]  name←name,(a0≠1+name)/a0
[6]
[7]  (rc type rank pshape pdatt)+j_getm (<name),4p<,-1
[8]
[9]  :if 0≠rc ∘ ⊆error 'return code: ',*rc ∘ :return ∘ :end
[10]
[11]  r←type rank pshape jfix pdatt

```

The utility function *jcnd* calls *jdo* then *jget*:

```

v r←jcnd exp
[1]  r←jdo 'JXP=: ',exp
[2]  r←jget 'JXP'
v

```

Examples

```

jcnd 'i.2 3 4'          A same as: 2 3 4 p 124
0 1 2 3
4 5 6 7
8 9 10 11

12 13 14 15
16 17 18 19
20 21 22 23

jcnd 'q: 123456'        A prime factors of 123456
2 2 2 2 2 2 3 643

*/jcnd 'q: 123456'
123456

```

Boxed arrays are returned as APL nested arrays:

```

jcnd '''hi there''';i.2 3'
hi there      0 1 2
               3 4 5

ds jcnd '''hi there''';i.2 3'  A display structure
+-----+
| +-----+ +-----+ |
| |hi there| | 0 1 2| |
| +-----+ | 3 4 5| |
|               +-----+ |
+-----+

```

Complex numbers are returned as real and imaginary pairs:

```
jcmd '_1^(>:i.5)%5'      A fractional powers of -1
0.80901699 0.58778525
0.30901699 0.95105652
-0.30901699 0.95105652
-0.80901699 0.58778525
-1          0
```

Extended integers are returned as character strings:

```
jcmd '*/x:p:i.20'      A product of first 20 primes
557940830126698960967415390x
```

The overhead of using *jcmd* is around 2 milliseconds on a P133.

The J DLL and workspace JDLL.W3 can be downloaded from the J Web page www.jsoftware.com.

Extended Integers

J 3.02 includes support for extended precision integers. This is an upwards-compatible extension that leaves the existing treatment of integers unchanged.

If you want to add extended integers to APL, you need only call the J DLL!

An extended integer is entered and displayed with a suffix of 'x'. Also, the verb *x:* applies to integers and returns extended precision integers. Various primitives produce extended results if the arguments are extended. For example:

```
2^100          standard
1.26765e30

2^100x        extended
1267650600228229401496703205376x
```

The topic of Multiprecision Arithmetic was recently discussed in a series of articles by John Sullivan in *Vector* Vol 12, issues 1-4. In fact, John described an implementation for both multiprecision integers and floating point numbers. J3.02 supports only multiprecision integers, though this covers a lot of ground; moreover many floating point calculations can be emulated using extended integers. For example, the following illustrates the convergence of the Taylor series for *e*, the last value for 40 terms being correct to 48 digits:

```
p=. (10^50x)"_
e=. (p * [: +/ ! % !@i.) <.@% !
.. e &> 20 30 40x
271828182845904523492875272833519940029860437269664x
```

```
271828182845904523536028747135265860223807331507783x
271828182845904523536028747135266249775724709369870x
```

Having extended integers in the language is a big benefit in convenience and speed. Indeed, the performance is far better than for defined functions. Here are sample timings in milliseconds on a P133, comparing the new extended integers against the functions in `bignum.js`, a script distributed with J:

	N	1e2	1e3	1e4	1e5	1e6
2^N	bignum	93	335	7140	n/a	n/a
	extended	0.5	2.2	112	9050	n/a
2^N (mod N+1)	bignum	132	225	302	374	516
	extended	1.6	2.8	3.2	5.8	7.5

At Play with J: Riding a Unicycle

by Eugene McDonnell

This article deals with two topics dealing with permutations having a single cycle, which can be called unicycles. The first arises from a recent Internet inquiry, and the second resuscitates an obscure mathematician from two centuries ago to give him credit for having invented list processing.

We might ask how many unicycles there are for permutations of a given length. This number can be found by the use of Stirling numbers of the first kind, which came about precisely from a need to count the number of ways to arrange n objects into k cycles. In their APL95 paper, *Representations of Recursion*, Roger Hui and Ken Iverson give an efficient way to generate the table of values for these Stirling numbers for cycles:

```
S1v=. 1: `([S1r $:0<:) @. * " 0
S1r=. (0: ,]) + <:@[ * ],0:

S1v 4
0 6 11 6 1

S1v i.10

1      0      0      0      0      0      0      0      0
0      1      0      0      0      0      0      0      0
0      1      1      0      0      0      0      0      0
0      2      3      1      0      0      0      0      0
0      6      11     6      1      0      0      0      0
0      24     50     35     10     1      0      0      0
0     120     274     225     85     15     1      0      0
0     720     1764     1624     735     175     21     1      0
0    5040    13068    13132    6769    1960    322    28     1
0   40320   109584   118124   67284   22449   4536   546    36     1
```

As you can see, the number of ways that n objects can be arranged in a unicycle is $!n-1$.

I. The Bernecky Problem

Bob Bernecky, of Snake Island Research, in Toronto, sent a message on the Internet recently asking for help in deriving from the sequence of link fields in a

linked list of records the permutation which would put the records into order. That is, suppose the records looked like this:

No.	Name	Link	
0	Bee	6	
1	Zee	0	
2	Que	8	
3	Gee	5	
4	Pea	2	(A)
5	Jay	9	
6	Dee	3	
7	Vee	1	
8	Tea	7	
9	Key	4	

The first record in the list is assumed to be in the leading position, but the locations of the other records is arbitrary. The 'Link' field in (A) gives the number of the successor record, and is 0 if it is the last record. In (A), the record following the first is number 6, and this is followed by number 3, which is followed by number 5, and so forth. It should be evident that the list of links is a permutation in (non-standard) cycles form, in other words, a unicycle.

What was desired was to have the records arranged as follows:

No.	Name	Link	
0	Bee	6	
6	Dee	3	
3	Gee	5	
5	Jay	9	
9	Key	4	(B)
4	Pea	2	
2	Que	8	
8	Tea	7	
7	Vee	1	
1	Zee	0	

The desired permutation is given by the 'No.' field in display (B), that is, 0 6 3 5 9 4 2 8 7 1. My usual way of exploring such problems is to head in the general direction where I imagine a solution may be found, with no maps or guides or bearers, and just beat my way unaided through the jungle with a machete. To my satisfaction I found that I could obtain the solution by applying *raze* (;) to the *cycles-direct* (C.) of the link list, and rotating this result so that it begins with 0:

```
y=.6 0 8 5 2 9 3 1 7 4
(i.<0 |. ] ) ; C. y
0 6 3 5 9 4 2 8 7 1
```

This is somewhat mysterious to me, since `C.` applied to an open list is supposed to convert from the direct form of a permutation to the cycles form, and here it looks as if the reverse is happening. (See the *J Introduction and Dictionary* for a description of the cycles and direct forms of a permutation.) Roger Hui provided me with the following explanation:

```
rot=: 1.&0 |. ]

g0 =: 3 : '{&y.^:(i.#y.) 0'
f0 =: g0 { ]

fs =: {."1 @ ((/\) @ (i.@# , <:@# # ,:))
g1 =: rot@fs
f1 =: g1 { ]

g2 =: rot@;@C.
f2 =: g2 { ]
```

The `g`'s (and therefore the `f`'s) are equivalent on the vector `x`:

```
x=: 6 0 8 5 2 9 3 1 7 4
(g0 -: g1) x
1
(g0 -: g2) x
1
```

But they are not equivalent on arbitrary `x`:

```
g0 12?.12
0 10 3 4 6 1 2 5 7 9 8 0
g1 12?.12
0 10 3 4 6 1 2 5 7 9 8 0
g2 12?.12
0 11 10 3 4 6 1 2 5 7 9 8
```

In fact, the functions are equivalent exactly on those arguments that are a single cycle (`1: = #@C.`), and the explanation you seek lies in why `g2=: rot@;@C.` "works" on a single cycle.

The so-called "link list" representation of `x`:

```
0 1 2 3 4 5 6 7 8 9
x 6 0 8 5 2 9 3 1 7 4
```

specifies that 0 goes to 6, 6 to 3, 3 to 5, 5 to 9, etc., and that is what `C.` does in obtaining the cycle representation from the direct representation. If there is more than one cycle (if there is stuff left over from this process), `C.` then does it again on the remaining elements to get the next cycle.

Since the argument is a single cycle, the raze of what results simply removes the boxing, and the rotation converts from the J convention of starting a cycle by its maximal element to the alternative convention of starting a cycle from 0.

II. Crelle's Device

Histories of computing generally date the beginning of list-processing techniques to 1963 or so, with some possible smatterings of these techniques dating back to the days of Von Neumann, circa 1947. Imagine my surprise, then, to find that the date is off by over a hundred years.

The German engineer and mathematician August Leopold Crelle lived from 1780 to 1855. He made many minor contributions to mathematics, but is generally much better known as the founder and editor of the mathematical periodical *Crelle's Journal*. His foreshadowing of list processing is described in L. E. Dickson's monumental *History of the Theory of Numbers* (Vol. I, chap. VII, p. 185). First some discussion of primitive roots is necessary.

If we consider the powers of the positive numbers less than a given prime p , mod p , we note that some of these powers contain distinct elements, while others have repetitions. For example, when $p=7$ we get:

```
f=. ] | [ : ^/~ i. 6. <:
f 7
1 1 1 1 1 1
2 4 1 2 4 1
3 2 6 4 5 1   (C)
4 2 1 4 2 1
5 4 6 2 3 1
6 1 6 1 6 1
```

and we see that the numbers 1 2 4 6 give rise to rows with repetitions, but rows 3 and 5 contain distinct elements. This property of 3 and 5 is what characterizes them as primitive roots of 7. Dickson wrote (in 1918):

A. L. Crelle[s] ... device for finding the residues modulo p of the powers of a will be clear from the example $p = 7$, $a = 3$. Write under the natural numbers <7 the residues of the successive multiples of 3 formed by successive additions of 3; we get

```
1 2 3 4 5 6
3 6 2 5 1 4
```

Then the residues 3, 2, 6, of 3, 32, 33,... modulo 7 are found as follows: after 3 comes the number 2 below 3 in the table; after 2 comes the number 6 below 2 in the table; etc.

In other words, Crelle's device uses a linked list to convert from a list of multiples to a list of powers, mod some prime p . From the list

3 6 2 5 1 4

he produces

3 2 6 4 5 1

and this corresponds to the row beginning with 3 of table (C).

Crelle's list is clearly a single cycle, and thus a unicycle. His device works for each primitive root of a given prime. It is a way for converting from addition to multiplication, and is thus analogous to logarithms.

All hail Crelle, father of list processing!

Index to Advertisers

Causeway Graphical Systems Ltd	8
Dyadic Systems Ltd	16
APL Booklist (Renaissance Data Systems)	6
Soliton	2
Vector Back Numbers	66

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the new Editor:

Duncan Pearson,
Keeper's Cottage
Firby
YORK, YO6 7LH
Tel: 01653-618900
Email: 100265.1564@compuserve.com

Authors wishing to use Windows Write or Word for Windows may contact Vector Production for a copy of the Vector APL TrueType font, a suitable Winword template and the Vector APL typewriter.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Tel: 01439-788385
Compuserve: 100331,644.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

- Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐
- | | | |
|---|------|--------------------------|
| UK private membership | £12 | ☐ |
| Overseas private membership | £14 | ☐ |
| Airmail supplement (not needed for Europe) | £4 | ☐ |
| UK Corporate membership | £100 | ☐ |
| Corporate membership overseas | £135 | ☐ |
| Sustaining membership | £430 | ☐ |
| Non-voting UK member (student/OAP/unemployed only) £6 | | ☐ |

PAYMENT – in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1996/97 Committee

Chairman	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary	Sylvia Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Duncan Pearson 01653-618900 100265.1564@compuserve.com	Keeper's Cottage, Firby, YORK, YO6 7LH
Activities	Jon Sandles 01904-612882 100257.1756@compuserve.com	138 Burton Stone Lane, York YO3 6DF
Education	Dr Ian Clark 01388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Projects and Publicity	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Duncan Pearson	Keeper's Cottage, Firby, YORK (01653-618900)
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (01302-539761), Anthony and Sylvia Camacho, Ray Cannon (01252-874697), John Searle (0181-858 6811), David Ziemann (0181-348 4039), Jon Sandles (01904-612882), Ajay Askoolum	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for 'A Programming Language' — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway/Graphical Systems Ltd
The Maltings, Castlegate,
Milton North Yorks YO17 0DP
Tel: 01655-696760
Fax: 01655-696719
Email: 1002651584@compuserve.com
Web: www.causeway.co.uk

Dyadic Systems Ltd

Riverside View, Basing Road,
Oxford, OX4 1JN

Hants, RG24 0AT

Tel: 01256-314125

Fax: 01256-314130

Email: sales@dyadic.com

Web: www.dyadic.com

Insight Systems APS

North Strand, N19A

DK6 3SD, Halesowen

Tel: +45 42 10 70 22

Fax: +45 42 10 70 74

Email: insight@netnet-net.dk

MicroAFL Ltd

South Bank Technopark

90 London Road

LONDON SE1 6LN

Tel: 0171-922 8866

Fax: 0171-923 1006

Email: microapl@netnet-net.com

Web: www.microapl.co.uk

British APL Association

Postbus 1641

8430 BM Nieuwegein

Netherlands

Tel: 0347-42637

Compass RAD Ltd
10 Frederick Sanger Road
Surrey Research Park
Guildford Surrey GU2 5XD
Tel: 01483-602249
Fax: 01483-602279

HMW Trading Systems Ltd

Hamilton House,
1 Temple Avenue,
LONDON EC2A 0HA

Tel: 0171-553 6900

Fax: 0171-553 6925

Email: 1000202632@compuserve.com

APL2000

2 Independence Way

Suite 100

Princeton NJ 08540 USA

Tel: +1 (609) 784-9692

Fax: +1 (609) 784-9694

Email: sales@apl2000.com

Web: members.apl.com/APL2000

Solliton Associates Ltd

Greenfield Kennelway 59

1082 AG Amsterdam

Netherlands

Tel: +31 20 676 4476

Fax: +31 20 676 1206

Email: sales@solliton.com